

TRƯỜNG ĐẠI HỌC BÁCH KHOA TP.HCM
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỐT NGHIỆP KỸ THUẬT MÁY TÍNH

**Thiết kế SoC RISC-V tích hợp EdgeAI
cho ứng dụng IoT**

Báo cáo cuối kỳ 252
Hội đồng 3L Kỹ thuật Máy tính

GVHD: PGS. TS. Trần Ngọc Thịnh

ThS. Huỳnh Phúc Nghi

GVPB: ThS. Phạm Kiều Nhật Anh

STT	Họ và tên	MSSV	Ghi chú
1	Lâm Nữ Uyển Nhi	2212429	
2	Vũ Đức Lâm	2211824	

TP. Hồ Chí Minh, Tháng 5/2026

CAM KẾT

Chúng tôi cam kết báo cáo tốt nghiệp với đề tài *Thiết kế SoC RISC-V tích hợp EdgeAI cho ứng dụng IoT* là kết quả nghiên cứu, thiết kế, hiện thực và đánh giá của nhóm dưới sự hướng dẫn của PGS. TS. Trần Ngọc Thịnh và ThS. Huỳnh Phúc Nghị tại Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

Các nội dung về kiến trúc BK-AISoC, bộ gia tốc CNN, firmware bare-metal, tích hợp trên FPGA VC707 và kết quả đánh giá thực nghiệm được xây dựng từ quá trình làm việc của nhóm. Những tài liệu, hình ảnh, công trình nghiên cứu và nguồn tham khảo bên ngoài được sử dụng trong báo cáo đều được trích dẫn hoặc ghi nhận phù hợp.

Trong quá trình hoàn thiện báo cáo, nhóm có sử dụng công cụ trí tuệ nhân tạo sinh nội dung ở mức hỗ trợ như rà soát diễn đạt, định dạng văn bản và thảo luận hướng gỡ lỗi. Các công cụ này không thay thế cho quá trình thiết kế kỹ thuật, hiện thực hệ thống, phân tích kết quả và kết luận của nhóm.

Chúng tôi chịu trách nhiệm về tính trung thực của nội dung, số liệu và nhận định trong báo cáo. Nếu phát hiện sai phạm về học thuật hoặc bản quyền, nhóm xin chịu hoàn toàn trách nhiệm theo quy định của Nhà trường và Khoa.

Thành phố Hồ Chí Minh, tháng 5
năm 2026

Nhóm thực hiện

Lâm Nữ Uyển Nhi

Vũ Đức Lâm

LỜI CẢM ƠN

Trước hết, nhóm xin gửi lời cảm ơn chân thành đến PGS. TS. Trần Ngọc Thịnh và ThS. Huỳnh Phúc Nghị đã trực tiếp hướng dẫn, góp ý và hỗ trợ nhóm trong suốt quá trình thực hiện đề án tốt nghiệp. Những định hướng của thầy giúp nhóm xác định rõ phạm vi đề tài, lựa chọn kiến trúc SoC phù hợp, hoàn thiện thiết kế bộ gia tốc CNN và gắn kết phần hiện thực phần cứng với quá trình đánh giá thực tế trên FPGA.

Nhóm cũng xin cảm ơn Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh, đã tạo môi trường học tập và nghiên cứu để nhóm tích lũy nền tảng về kiến trúc máy tính, thiết kế logic số, hệ thống nhúng, giao tiếp phần cứng và tích hợp phần cứng–phần mềm. Những kiến thức này là cơ sở quan trọng để nhóm có thể xây dựng hệ thống BK-AISoC, từ thiết kế datapath của accelerator đến firmware điều khiển và kiểm thử trên bo mạch VC707.

Cuối cùng, nhóm xin cảm ơn gia đình và bạn bè đã luôn động viên, chia sẻ và hỗ trợ trong thời gian thực hiện đề tài. Sự tin tưởng và khích lệ đó giúp nhóm vượt qua nhiều khó khăn trong quá trình mô phỏng RTL, tổng hợp thiết kế, khởi tạo firmware, nạp dữ liệu, chạy thử nghiệm và phân tích kết quả.

TÓM TẮT

Sự phát triển của EdgeAI và IoT đặt ra nhu cầu xử lý mô hình học sâu tại thiết bị biên để giảm độ trễ và dữ liệu truyền về máy chủ [1], [2]. Các nghiên cứu về bộ gia tốc CNN cho thấy chi phí di chuyển dữ liệu và tổ chức bộ nhớ ảnh hưởng lớn đến hiệu năng hệ thống [3], [4], trong khi hướng tích hợp CPU RISC-V với accelerator chuyên dụng phù hợp với thiết bị IoT có tài nguyên giới hạn [2]. Báo cáo trình bày thiết kế và hiện thực BK-AISoC, một SoC RISC-V tích hợp bộ gia tốc CNN cho ứng dụng EdgeAI trên FPGA.

Bộ gia tốc tập trung vào lớp tích chập chuẩn trong CNN, với VGG16 và ALL-CNN-C được dùng như các mô hình khảo sát đại diện [5], [6]. Kiến trúc phân rã filter theo hàng và theo kênh, tổ chức ifmap theo từng pass, cộng dồn partial sum trong buffer và dùng ping-pong FIFO, ping-pong circle FIFO để tăng tái sử dụng dữ liệu. Datapath hỗ trợ INT8, bias, scale và activation ReLU/Leaky ReLU, phù hợp với hướng suy luận số nguyên trên thiết bị tài nguyên giới hạn [7]. Accelerator được tích hợp với PicoRV32, AXI, DMAC, HyperRAM, Video Streaming và các ngoại vi SPI, UART, I2C, GPIO, Timer; firmware bare-metal đảm nhiệm boot, HAL và điều khiển accelerator.

Hệ thống được kiểm chứng bằng mô phỏng RTL và thực nghiệm trên Xilinx VC707. Trong mô phỏng, Conv1 của VGG16 đạt khoảng 6.74 GOPS, còn Conv1 của ALL CNN C đạt khoảng 3.63 GOPS. Trên VC707, 13 lớp convolution của VGG16 chạy ổn định với tổng thời gian khoảng 2,613 ms. Với ALL CNN C, CPU kết hợp CNN accelerator giảm thời gian compute/model từ 154,155 ms xuống 103 ms, tăng tốc xấp xỉ 1496 lần; nếu tính wall section, mức tăng tốc vẫn đạt khoảng 695 lần. Kết quả cho thấy BK-AISoC tăng tốc đáng kể workload CNN, đồng thời chỉ ra giới hạn hiện tại chủ yếu nằm ở chi phí DMA, truy cập bộ nhớ ngoài và điều phối hệ thống.

Từ khóa: EdgeAI, RISC-V, SoC, bộ gia tốc CNN, FPGA, VC707, INT8,

IoT.

Mục lục

Danh mục Ký hiệu và Chữ viết tắt	xv
1 Giới thiệu	1
1.1 Tổng quan về đề tài	1
1.2 Mục tiêu và Nhiệm vụ nghiên cứu	2
1.3 Phạm vi đề tài	3
1.3.1 Phạm vi và Giới hạn đề tài	3
1.3.2 Đối tượng và Công cụ nghiên cứu	3
1.4 Phân chia công việc	3
2 Cơ sở lý thuyết	5
2.1 Tổng quan về Mạng nơ-ron tích chập (CNN)	5
2.1.1 Trí tuệ nhân tạo và Học sâu	5
2.1.2 Xu hướng chuyển dịch tính toán xuống biên (Edge AI)	6
2.1.3 Cơ sở toán học của Mạng Nơ-ron Tích chập (CNN) .	7
2.1.4 Kỹ thuật Gập Batch Normalization (BN Folding) . .	7
2.2 Công trình nghiên cứu liên quan kiến trúc bộ gia tốc CNN .	8
2.2.1 Kiến trúc tham chiếu: Hệ thống Eyeriss	8
2.2.2 Kiến trúc tham chiếu: Bộ tăng tốc Pixel-Level Fully Pipelined	8
2.2.3 Kiến trúc tham chiếu: Tăng tốc CNN trên FPGA dựa trên OpenCL	9

2.2.4	Kiến trúc tham chiếu: Hệ thống xử lý dữ thể trên nền tảng RISC-V cho IoT	10
2.2.5	Kiến trúc tham chiếu: Bộ tăng tốc luồng cấu hình lại (RSA) cho IoT	10
3	Kiến trúc đề xuất	12
3.1	Tổng quan kiến trúc đề xuất của BK-AISoC	12
3.1.1	Nguyên tắc phân tách control path và data path . . .	14
3.1.2	Vai trò của các khối chính trong kiến trúc	14
3.1.3	Luồng xử lý suy luận trên BK-AISoC	15
3.2	Phân tích và thiết kế kiến trúc tính toán cho lớp tích chập chuẩn	16
3.2.1	Tổ chức lại dữ liệu filter theo hàng và theo kênh . . .	17
3.2.2	Tổ chức lại dữ liệu ifmap theo thứ tự quét của kiến trúc	18
3.2.3	Cơ chế tính toán tuần tự theo pass và cộng dồn trong buffer	19
3.2.4	Mô hình hóa phần tử tính toán và điều kiện cộng dồn	21
3.2.5	Khả năng mở rộng mức song song của kiến trúc . . .	23
3.2.6	Tối ưu bộ nhớ bằng cơ chế tile filter	25
3.3	Kiến trúc tổng quan của bộ gia tốc CNN	27
3.4	Quy trình điều khiển và luồng dữ liệu	28
3.5	Các thành phần logic cơ sở	29
3.5.1	Cấu trúc Ping-pong FIFO	29
3.5.2	Cấu trúc Ping-pong Circle FIFO	30
3.6	Kiến trúc chi tiết các khối chức năng	31
3.6.1	Khối Instruction Table	31
3.6.2	Khối Layer Information	32
3.6.3	Khối Buffer Filter	33
3.6.4	Khối Buffer Ifmap	34
3.6.5	Khối Computation	35

3.6.5.1	Mảng xử lý Processing Array (PA)	36
3.6.5.2	Đơn vị xử lý Processing Unit (PU)	37
3.6.5.3	Kiến trúc khối Scale & ReLU	38
3.6.6	Khối Buffer Bias	39
3.6.7	Khối Buffer Ofmap	40
3.7	Mô hình hóa thời gian và bộ nhớ của bộ gia tốc	40
3.7.1	Giả thiết mô hình	40
3.7.2	Ký hiệu và tham số mô hình	41
3.7.3	Tổ chức dữ liệu theo tile và theo cụm bộ lọc	42
3.7.4	Mô hình thời gian của một pass	43
3.7.5	Mô hình nạp Ifmap và ghi Ofmap	43
3.7.6	Mô hình thời gian của một tile và của một lớp	44
3.7.7	Mô hình bộ nhớ vật lý	45
3.7.8	Hàm mục tiêu tối ưu	47
3.7.9	Đánh giá cấu hình bộ nhớ vật lý	47
3.7.10	Xác định kích thước vật lý	48
4	Hiện thực kiến trúc	50
4.1	Tổng quan kiến trúc BK-AISoC	50
4.2	Processor core và on-chip memory	52
4.3	Bus và address space	52
4.4	AXI4-Lite Interconnect	53
4.5	Peripheral controller	59
4.5.1	UART	59
4.5.2	SPI	60
4.5.3	I2C	62
4.5.4	Timer, Timer Interrupt và GPIO	64
4.6	External memory và data path	66
4.7	CNN accelerator trong hệ thống	66

4.8	DMAC cho CNN accelerator	67
4.8.1	Kiến trúc tổng thể	67
4.8.2	Phân xử truy cập đọc/ghi	69
4.8.3	Trình tự hoạt động của phần cứng	70
4.9	Video Streaming	72
4.9.1	Kiến trúc tổng thể	72
4.9.2	Thu nhận dữ liệu ảnh	74
4.9.3	Frame Buffer và điều phối khung hình	74
4.9.4	Display output path	74
4.9.5	Streaming path về phía accelerator	75
4.10	Kết luận chương	76
4.11	Thiết kế Firmware BK-AISoC	77
4.11.1	Mục tiêu và vai trò của Firmware	77
4.11.2	Tổ chức mã nguồn Firmware	77
4.11.3	Boot Flow và Memory Map	81
4.11.3.1	Memory map của application	81
4.11.3.2	Segmented boot image BAI1	82
4.11.4	Bootloader và công cụ nạp firmware	84
4.11.4.1	Giao thức flash application qua UART	84
4.11.4.2	Quá trình chuyển quyền điều khiển	86
4.11.5	SoC Hardware Abstraction Layer	87
4.11.5.1	Mục tiêu thiết kế HAL	87
4.11.5.2	Mô hình truy cập register	88
4.11.5.3	Tổ chức chức năng của HAL	88
4.11.6	Application mẫu kiểm chứng hệ thống	91
4.11.6.1	Các hướng chức năng của application	91
4.11.6.2	Kiểm chứng hoạt động end-to-end	92
4.11.7	Kết quả hiện thực Firmware	93
4.12	Triển khai hệ thống trên FPGA và kiểm chứng thực nghiệm	95

4.12.1	Nền tảng phần cứng thực nghiệm	95
4.12.2	Thiết kế PCB và Signal Integrity	97
4.12.3	Quy trình build phần cứng trên FPGA	99
4.12.4	Thực nghiệm trên mạch thật	105
5	Đánh giá	110
5.1	Thiết lập đánh giá	111
5.2	Phân tích kết quả mô phỏng	112
5.2.1	Kết quả mô phỏng lớp tích chập đầu của VGG16 . .	112
5.2.2	Kết quả mô phỏng lớp Conv1 của ALL CNN C . . .	114
5.2.3	So sánh với thời gian mô hình hóa	115
5.2.4	Đánh giá tổng hợp kết quả mô phỏng RTL	119
5.3	Phân tích kết quả thực tế VGG16 trên board VC707	120
5.3.1	Kết quả thực nghiệm và mô hình hoá thời gian chạy VGG16	120
5.3.1.1	Phân tích layer Conv1_2	123
5.3.1.2	Phân tích layer Conv5_1	124
5.3.2	Nhận xét và đánh giá chung đối với VGG16	125
5.4	Phân tích kết quả thực tế ALL CNN C trên board VC707 . .	126
5.5	Phân tích sự đánh đổi khi triển khai mô hình INT8 trên CNN Accelerator	131
6	Kết luận và hướng phát triển	135
6.1	Kết luận	135
6.2	Hướng phát triển	135
6.2.1	Hoàn thiện kiến trúc phần cứng	136
6.2.2	Tối ưu firmware và application	136
6.2.3	Định hướng đánh giá thực nghiệm	136

Tài liệu tham khảo	138
---------------------------	------------

Danh sách hình vẽ

Hình 3.1:	Kiến trúc tổng quan đề xuất của hệ thống BK-AISoC	13
Hình 3.2:	Ví dụ minh họa input feature và các filter trên hai channel .	17
Hình 3.3:	Tổ chức lại filter và ifmap theo các pass tính toán	19
Hình 3.4:	Cơ chế cộng dồn partial sum trong các buffer qua từng pass .	20
Hình 3.5:	Mô hình hóa datapath và logic cộng dồn partial sum	23
Hình 3.6:	Minh họa cách tổ chức dữ liệu filter và bias trong bộ nhớ ngoài	26
Hình 3.7:	Kiến trúc tổng quan của bộ gia tốc CNN	27
Hình 3.8:	Cấu trúc Ping-pong FIFO	29
Hình 3.9:	Cấu trúc Ping-pong Circle FIFO	30
Hình 3.10:	Khối Instruction Table	31
Hình 3.11:	Khối Layer Information	32
Hình 3.12:	Kiến trúc khối Buffer Filter	33
Hình 3.13:	Kiến trúc khối Buffer Ifmap	34
Hình 3.14:	Kiến trúc tổng quan khối Computation	35
Hình 3.15:	Kiến trúc Mảng xử lý Processing Array (PA)	36
Hình 3.16:	Kiến trúc Đơn vị xử lý (PU)	37
Hình 3.17:	Kiến trúc pipeline của khối Scale & ReLU	38

Hình 3.18: Kiến trúc khối Buffer Bias	39
Hình 3.19: Kiến trúc khối Buffer Ofmap	40
Hình 4.1: Sơ đồ khối kiến trúc tổng quan của hệ thống BK-AISoC . .	51
Hình 4.2: Các mô hình kết nối cơ bản của AXI4-Lite Interconnect . .	54
Hình 4.3: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $1 \times N$	55
Hình 4.4: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $N \times 1$	56
Hình 4.5: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $N \times M$	57
Hình 4.6: Cơ chế Arbiter Round-Robin trong AXI4-Lite Interconnect .	58
Hình 4.7: Hardware architecture của khối UART	60
Hình 4.8: Register map của khối UART qua AXI4-Lite interface . . .	60
Hình 4.9: SPI timing và các mode truyền dữ liệu	61
Hình 4.10: Control FSM của khối SPI	62
Hình 4.11: Timing và control FSM của khối I2C	63
Hình 4.12: Phân tách Timer ngoại vi và Timer Interrupt trong PicoRV32	65
Hình 4.13: Kiến trúc tổng thể của DMAC kết nối CNN accelerator và HyperRAM	68
Hình 4.14: Kiến trúc chi tiết bên trong khối DMAC	69
Hình 4.15: Vị trí của DMA0/DMA1 trong CAU và kết nối với Video Streaming	71
Hình 4.16: Kiến trúc của khối Video Streaming có tích hợp Resize Image	73
Hình 4.17: Kiến trúc phân lớp của firmware BK-AISoC	80
Hình 4.18: Định dạng segmented boot image của application	84

Hình 4.19: Luồng flash application qua UART bootloader	86
Hình 4.20: Sơ đồ không gian địa chỉ 32-bit được HAL sử dụng	89
Hình 4.21: Các hướng chức năng của application mẫu trên BK-AISoC .	92
Hình 4.22: Virtex-7 FPGA VC707 Evaluation Kit	96
Hình 4.23: Mô hình 3D tổng thể của bo mạch mở rộng VC707 I/O Ex- tension Board	96
Hình 4.24: Bo mạch mở rộng cho camera OV5640 với giao diện DVP .	97
Hình 4.25: Layout PCB thể hiện kỹ thuật Meander routing và mặt phẳng Ground	98
Hình 4.26: Via Shielding được áp dụng cho đường Clock nhằm kiểm soát EMI	99
Hình 4.27: Phân bố tài nguyên theo phân cấp BK-AISoC	100
Hình 4.28: Phân bố tài nguyên thực tế của BK-AISoC sau Implementation	104
Hình 4.29: Mô hình thực nghiệm gồm VC707, mạch mở rộng I/O, cam- era OV5640 và HDMI	105
Hình 4.30: Quá trình nạp boot image từ host computer xuống BK-AISoC thông qua UART bootloader	106
Hình 4.31: Thực nghiệm chạy application thật với UART bootloader và truyền dữ liệu trực tiếp giữa laptop và BK-AISoC	107
Hình 4.32: Hệ thống BK-AISoC chạy thực tế với camera, màn hình hiển thị và host application	107
Hình 4.33: Kết quả host application nhận luồng ảnh và so sánh dự đoán giữa host computer và BK-AISoC	108
Hình 4.34: Kết quả xuất tín hiệu video theo thời gian thực lên màn hình	108

Hình 5.1:	Waveform mô phỏng lớp tích chập đầu của VGG16	113
Hình 5.2:	Waveform mô phỏng lớp Conv1 của ALL CNN C	114
Hình 5.3:	Thời gian chờ DMA cấu hình memory	122

Danh sách bảng biểu

Bảng 1.1: Bảng phân chia công việc của các thành viên	4
Bảng 3.1: Vai trò của các khối chính trong kiến trúc BK-AISoC .	15
Bảng 4.1: Vai trò của các thành phần mã nguồn trong firmware .	79
Bảng 4.2: Phân vùng bộ nhớ chính của firmware	81
Bảng 4.3: Các miền Clock được sử dụng trong thiết kế sau Implementation	101
Bảng 4.4: Tổng hợp tài nguyên sau Synthesis và Implementation .	101
Bảng 4.5: Tài nguyên sử dụng theo từng IP sau Implementation .	103
Bảng 5.1: So sánh thời gian mô hình hóa và thời gian simulation .	118
Bảng 5.2: Tổng hợp kết quả mô phỏng RTL trên hai lớp tích chập	119
Bảng 5.3: Thời gian chạy thực tế của các lớp convolution trong VGG16 trên board VC707	121
Bảng 5.4: So sánh thời gian thực thi mô hình ALL CNN C giữa CPU-only và CPU kết hợp CNN accelerator	127
Bảng 5.5: So sánh hiệu suất xử lý theo cùng workload end-to-end của mô hình ALL CNN C	128
Bảng 5.6: So sánh độ chính xác của mô hình trên các môi trường thực thi	131

Danh mục Ký hiệu và Chữ viết tắt

Ký hiệu	Ý nghĩa
H, W	Chiều cao và chiều rộng của đặc trưng đầu vào (Input Feature Map)
C	Số lượng kênh đầu vào (Input Channels)
N_f	Số lượng bộ lọc / Số kênh đầu ra (Number of Filters / Output Channels)
H_{out}, W_{out}	Chiều cao và chiều rộng của đặc trưng đầu ra (Output Feature Map)
R, S	Chiều cao và chiều rộng của bộ lọc (Kernel Height, Kernel Width)
P	Kích thước vùng đệm (Padding)
Str (hoặc U)	Bước trượt (Stride)
T_h	Chiều cao của mảnh dữ liệu đầu vào trong một Pass (Tile Height)
T_c	Số kênh đầu vào được xử lý song song trong một Pass (Tile Input Channels)
T_m	Số bộ lọc được tính toán song song trong một Pass (Tile Output Channels)
T_{ho}	Chiều cao hợp lệ của mảnh dữ liệu đầu ra trong một Pass
b	Số chu kỳ đồng hồ để truyền một giá trị dữ liệu (Cycles per Data Transfer)
T_{comp}	Thời gian tính toán (Computation time)

Ký hiệu	Ý nghĩa
T_{load}	Thời gian nạp dữ liệu (Load time)
T_{store}	Thời gian ghi dữ liệu (Store time)
T_{pass}	Thời gian hoàn thành một Pass
I	Tensor dữ liệu đầu vào
O	Tensor dữ liệu đầu ra
W	Tensor trọng số (Weights)
B	Vector hệ số chệch (Bias)
μ, σ	Giá trị trung bình (Mean) và Phương sai (Variance) trong Batch Norm
γ, β	Tham số tỉ lệ (Scale) và dịch chuyển (Shift) trong Batch Norm

Viết tắt	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
CNN	Mạng nơ-ron tích chập (Convolutional Neural Network)
DNN	Mạng nơ-ron sâu (Deep Neural Network)
FPGA	Mảng cổng lập trình được dạng trường (Field-Programmable Gate Array)
SoC	Hệ thống trên chip (System-on-Chip)
RTL	Mức chuyển giao thanh ghi (Register Transfer Level)
IFM	Đặc trưng đầu vào (Input Feature Map)
OFM	Đặc trưng đầu ra (Output Feature Map)
PE	Phần tử xử lý (Processing Element)
PU	Đơn vị xử lý (Processing Unit - Chứa nhiều PE)
PA	Mảng xử lý (Process Array - Chứa nhiều PU)
MAC	Phép tính Nhân-Cộng tích lũy (Multiply-Accumulate)
DMA	Truy cập bộ nhớ trực tiếp (Direct Memory Access)
AXI-Lite	Giao diện mở rộng nâng cao rút gọn (Advanced eXtensible Interface Lite)

Ký hiệu	Ý nghĩa
AXI-Stream	Giao diện luồng dữ liệu mở rộng nâng cao (Advanced eXtensible Interface Stream)
BRAM	Block RAM (Bộ nhớ nội trên FPGA)
DMA	Truy cập bộ nhớ trực tiếp (Direct Memory Access)
AXI	Giao diện mở rộng nâng cao (Advanced eXtensible Interface)
DSP	Digital Signal Processing (Khối xử lý tín hiệu số trên FPGA)
LUT	Bảng tra (Look-Up Table)
FF	Flip-Flop
OSPI	Giao diện ngoại vi nối tiếp 8 kênh (Octal Serial Peripheral Interface)
SPI	Giao diện ngoại vi nối tiếp (Serial Peripheral Interface)
UART	Bộ truyền nhận dữ liệu nối tiếp bất đồng bộ (Universal Asynchronous Receiver-Transmitter)
I2C	Giao thức giao tiếp giữa các vi mạch (Inter-Integrated Circuit)
DVP	Cổng dữ liệu hình ảnh kỹ thuật số (Digital Video Port)
GPIO	Cổng vào/ra đa dụng (General Purpose Input/Output)

Chương 1

Giới thiệu

Chương này trình bày tổng quan về bối cảnh nghiên cứu, xác định mục tiêu cụ thể, phạm vi thực hiện.

1.1 Tổng quan về đề tài

Tên đề tài: Thiết kế SoC RISC-V tích hợp EdgeAI cho ứng dụng IoT.

Đề tài tập trung vào việc thiết kế và phát triển một hệ thống trên chip (SoC) dựa trên vi xử lý RISC-V tích hợp phần tăng tốc EdgeAI (Accelerator), nhằm xử lý các tác vụ trí tuệ nhân tạo ngay tại biên [2], [8]. Hệ thống sẽ được triển khai thử nghiệm trên nền tảng FPGA, với kiến trúc được tối ưu hóa nhằm hướng tới khả năng chuyển đổi sang thiết kế ASIC (Application-Specific Integrated Circuit) trong tương lai [9].

Bên cạnh việc thiết kế phần cứng, đề tài cũng bao gồm quá trình thử nghiệm hiệu suất hệ thống với một tập dữ liệu cố định và triển khai một số ứng dụng IoT thực tế làm "case study" (ví dụ: nhận diện hình ảnh từ camera) nhằm đánh giá tính khả thi và hiệu quả hoạt động của hệ thống trong môi trường thực tế [4].

Hệ thống hoàn chỉnh sẽ bao gồm các thành phần chính:

- Lõi vi xử lý RISC-V (CPU Core) [8], [10].
- Bộ tăng tốc mạng nơ-ron tích chập (CNN Accelerator) [3].
- Hệ thống Bus giao tiếp nội bộ (AXI-Lite, AXI-Stream) [11], [12].
- Bộ truy cập bộ nhớ trực tiếp (DMA).
- Các giao tiếp I/O với ngoại vi (HYPERRAM, SPI, UART, I2C, DVP, GPIO, TIMER,...) [13], [14], [15].

1.2 Mục tiêu và Nhiệm vụ nghiên cứu

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và hiện thực một hệ thống trên chip (SoC) hoàn chỉnh tích hợp lõi vi xử lý RISC-V và bộ tăng tốc phần cứng (Hardware Accelerator) cho các tác vụ trí tuệ nhân tạo tại biên (EdgeAI) [2], [8]. Cụ thể, đề tài hướng tới các mục tiêu sau:

Về kiến trúc hệ thống: Xây dựng kiến trúc SoC tối ưu năng lượng, sử dụng chuẩn giao tiếp AXI để kết nối giữa vi xử lý trung tâm và khối tăng tốc tính toán [11].

Về xử lý AI: Thiết kế khối Accelerator chuyên dụng hỗ trợ các phép toán trọng yếu của mạng nơ-ron tích chập (CNN) như ALL CNN C, nhằm giảm tải cho CPU và tăng tốc độ xử lý thực tế [5], [6].

Về ứng dụng thực tế: Tích hợp đầy đủ các giao tiếp ngoại vi (Camera/HDMI DVP, UART, SPI, HYPERRAM, I2C, GPIO, TIMER) để xây dựng một ứng dụng IoT trọn vẹn (ví dụ: nhận diện vật thể hoặc phân loại ảnh) chạy trực tiếp trên nền tảng FPGA và hướng tới ASIC.

Về quy trình thiết kế: Làm chủ quy trình thiết kế từ mức RTL (Verilog) đến mô phỏng (Simulation), tổng hợp (Synthesis) và kiểm tra trên phần cứng thực (FPGA Prototyping) [9], [16].

1.3 Phạm vi đề tài

Để đảm bảo tính khả thi trong khuôn khổ thời gian của đề án, nhóm thực hiện xác định phạm vi nghiên cứu như sau:

1.3.1 Phạm vi và Giới hạn đề tài

- **Vi xử lý:** Sử dụng lõi PicoRV32 (RISC-V 32-bit - RV32I) mã nguồn mở, tập trung vào việc tích hợp và xây dựng hệ thống bus (System Interconnect) thay vì thiết kế lại kiến trúc nhân CPU [8], [10].
- **Mô hình AI:** Tập trung hỗ trợ các mạng CNN cơ bản (như ALL CNN C, SqueezeNet) đã được lượng tử hóa (Quantization) xuống 8-bit integer để phù hợp với tài nguyên phần cứng, không đi sâu vào việc huấn luyện (training) các mô hình lớn [5], [6], [7].
- **Nền tảng phần cứng:** Hệ thống được thiết kế bằng ngôn ngữ Verilog và kiểm chứng trên Kit FPGA (AMD Virtex™ 7 FPGA VC707). Chưa bao gồm các bước thiết kế vật lý (Physical Design) để ra chip ASIC thực tế (Layout, GDSII) [9], [17].

1.3.2 Đối tượng và Công cụ nghiên cứu

- Ngôn ngữ thiết kế: Verilog, C/C++.
- Công cụ mô phỏng và tổng hợp: Vivado Design Suite.

1.4 Phân chia công việc

Đề án được thực hiện bởi hai thành viên với sự phân chia công việc cụ thể dựa trên kiến trúc hệ thống như sau:

Bảng 1.1: Bảng phân chia công việc của các thành viên

STT	Thành viên	Nội dung thực hiện
1	Lâm Nữ Uyên Nhi (Chịu trách nhiệm về Accelerator)	• Nghiên cứu lý thuyết về mạng CNN và các kỹ thuật tối ưu phần cứng [3], [18]. • Thiết kế kiến trúc khối CNN Accelerator (PE Array, Buffer Controller). • Hiện thực khối tính toán: Standard Convolution có nhiều cấu hình stride, padding, kích thước filter, ifmap khác nhau. • Viết Testbench kiểm tra chức năng khối Accelerator.
2	Vũ Đức Lâm (Chịu trách nhiệm về SoC & System)	• Nghiên cứu kiến trúc RISC-V và chuẩn bus AMBA AXI [8], [11]. • Thiết kế hệ thống SoC: Tích hợp CPU, Interconnect, Memory Controller. • Thiết kế các giao tiếp ngoại vi: UART, SPI, HYPERRAM, I2C, GPIO, TIMER, DVP (Camera/HDMI). • Phát triển Firmware/Driver để điều khiển hệ thống. • Tổng hợp hệ thống lên FPGA và đo đặc hiệu năng.

Chương 2

Cơ sở lý thuyết

Chương này cung cấp các kiến thức nền tảng về mô hình mạng nơ-ron tích chập (CNN) và các công trình nghiên cứu liên quan đến kiến trúc bộ gia tốc CNN.

2.1 Tổng quan về Mạng nơ-ron tích chập (CNN)

Phần này trình bày các cơ sở lý thuyết về trí tuệ nhân tạo, trọng tâm là các mạng nơ-ron tích chập (CNN) [1], [18]. Đồng thời, các phân tích về xu hướng tính toán biên (Edge AI) và đặc tả toán học của các phép tính cốt lõi cũng được thảo luận chi tiết nhằm làm rõ động lực thiết kế phần cứng của đề án.

2.1.1 Trí tuệ nhân tạo và Học sâu

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực khoa học kỹ thuật với mục tiêu kiến tạo các hệ thống máy móc thông minh, sở hữu khả năng thực hiện các tác vụ vốn đòi hỏi trí tuệ con người. Là một tập con quan trọng của AI, Học máy (Machine Learning - ML) cho phép máy tính tự học hỏi từ dữ liệu và cải thiện hiệu suất mà không cần lập trình cụ thể cho từng tác vụ. Thay vì dựa vào các quy tắc thủ công tính, các thuật toán ML sử dụng quá trình huấn luyện để xây dựng mô hình giải quyết vấn đề.

Trong đó, Học sâu (Deep Learning - DL) là bước tiến vượt bậc của ML, tập trung phát triển các Mạng nơ-ron sâu (Deep Neural Networks - DNNs) [1]. Các mạng hiện đại có thể sở hữu từ 5 đến hàng nghìn lớp, vượt xa quy mô của các mạng nơ-ron truyền thống. Sức mạnh vượt trội của DNN nằm ở khả năng phân cấp đặc trưng (Feature Hierarchy). Khi dữ liệu đi qua các lớp của mạng, thông tin được trích xuất theo mức độ trừu tượng tăng dần: từ các đặc trưng cấp thấp như cạnh, đường thẳng ở lớp đầu, đến hình dạng phức tạp ở lớp giữa, và cuối cùng là nhận diện vật thể hoàn chỉnh ở lớp cuối. Cấu trúc này đặc biệt hiệu quả trong các bài toán Thị giác máy tính (Computer Vision) như phân loại ảnh hay xe tự hành.

2.1.2 Xu hướng chuyển dịch tính toán xuống biên (Edge AI)

Vòng đời của một mô hình AI bao gồm hai giai đoạn chính là Huấn luyện (Training) và Suy luận (Inference). Trong khi quá trình huấn luyện đòi hỏi tài nguyên khổng lồ thường thực hiện trên Cloud, quá trình suy luận đang có xu hướng dịch chuyển mạnh mẽ xuống các thiết bị biên (Edge devices/IoT).

Việc đưa tác vụ Inference xuống biên, hay còn gọi là Edge AI, giải quyết được ba thách thức cốt lõi của mô hình tập trung. Thứ nhất là độ trễ (latency), yếu tố sống còn đối với các ứng dụng thời gian thực như xe tự lái, nơi độ trễ đường truyền Cloud có thể gây rủi ro an toàn. Thứ hai là tối ưu hóa băng thông mạng khi không cần truyền tải dữ liệu thô (như video giám sát) lên máy chủ. Cuối cùng là đảm bảo quyền riêng tư và bảo mật dữ liệu người dùng.

Tuy nhiên, các nền tảng nhúng thường bị giới hạn nghiêm ngặt về ngân sách năng lượng, tài nguyên tính toán và dung lượng bộ nhớ. Do đó, việc thiết kế các kiến trúc phần cứng chuyên dụng (AI Accelerator) để xử lý hiệu quả các thuật toán DNN dưới các ràng buộc này là yêu cầu cấp thiết [2], [3].

2.1.3 Cơ sở toán học của Mạng Nơ-ron Tích chập (CNN)

Mạng nơ-ron tích chập (CNN) là kiến trúc phổ biến nhất trong Deep Learning để xử lý dữ liệu hình ảnh [18]. Đồ án tập trung phân tích đặc tả toán học của Standard Convolution. Standard Convolution thực hiện trượt bộ lọc trên không gian đầu vào và tích lũy giá trị qua toàn bộ chiều sâu kênh (Channels). Giá trị đầu ra O tại kênh m , vị trí (h, w) được xác định bởi công thức:

$$O[m][h][w] = B[m] + \sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} I[c][h \cdot U + r - P][w \cdot U + s - P] \times W[m][c][r][s] \quad (2.1)$$

Trong đó, U là bước trượt (Stride), P là lượng đệm (Padding), W là trọng số và I là đầu vào. Việc xử lý biên (Padding) đóng vai trò quan trọng để duy trì kích thước không gian, yêu cầu phần cứng phải có logic tự động chèn giá trị 0 (Zero-padding) khi chỉ số truy cập nằm ngoài phạm vi hình ảnh thực tế.

2.1.4 Kỹ thuật Gập Batch Normalization (BN Folding)

Trong giai đoạn suy luận, để giảm thiểu độ phức tạp tính toán, đồ án áp dụng kỹ thuật BN Folding. Lớp Batch Normalization thường đi kèm sau Convolution có các tham số $(\mu, \sigma, \gamma, \beta)$ là hằng số cố định khi suy luận [19]. Ta có thể gộp các phép tính này vào trực tiếp trọng số (W) và bias (B) của lớp Convolution phía trước:

$$W' = W_{orig} \cdot \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}; \quad B' = (B_{orig} - \mu) \cdot \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.2)$$

Kỹ thuật này giúp loại bỏ hoàn toàn khối tính toán Batch Normalization trên phần cứng, giúp tiết kiệm tài nguyên và giảm độ trễ xử lý mà không làm ảnh hưởng đến độ chính xác của mô hình.

2.2 Công trình nghiên cứu liên quan kiến trúc bộ gia tốc CNN

2.2.1 Kiến trúc tham chiếu: Hệ thống Eyeriss

Để giải quyết bài toán tối ưu hóa năng lượng cho các mạng nơ-ron tích chập (CNN), kiến trúc Eyeriss (Chen et al., 2017) [3] tập trung vào việc cực tiểu hóa chi phí di chuyển dữ liệu thông qua thiết kế phần cứng chuyên biệt và phân cấp bộ nhớ hiệu quả. Hệ thống bao gồm chip tăng tốc kết nối với DRAM ngoại vi qua giao diện bất đồng bộ, cho phép tách biệt miền xung nhịp tính toán và giao tiếp. Trung tâm của kiến trúc là mảng 12×14 phần tử xử lý (PE) hoạt động độc lập, mỗi PE sở hữu bộ nhớ đệm cục bộ (Scratchpads - Spads) để lưu trữ trọng số, dữ liệu đầu vào và các tổng riêng. Thiết kế này tạo nên mô hình phân cấp bộ nhớ bốn mức, từ DRAM, Global Buffer, Inter-PE đến Spads, giúp khai thác tối đa tính cục bộ của dữ liệu. Đóng vai trò trung chuyển là bộ đệm toàn cục (Global Buffer) dung lượng 108KB, giúp giảm thiểu các truy cập bộ nhớ ngoài tốn kém. Hệ thống sử dụng mạng kết nối trên chip (NoC) tùy biến gồm Mạng đầu vào (GIN) hỗ trợ multicast và Mạng đầu ra (GON). Điểm đặc biệt của Eyeriss là luồng dữ liệu "Row Stationary" (RS), cho phép tái sử dụng dữ liệu hiệu quả ngay tại các bộ nhớ cục bộ trong từng PE, từ đó tối ưu hóa công suất tiêu thụ tổng thể.

2.2.2 Kiến trúc tham chiếu: Bộ tăng tốc Pixel-Level Fully Pipelined

Nhằm khắc phục nhược điểm về độ trễ và thông lượng của các kiến trúc truyền thống xử lý theo lớp (layer-by-layer), Li và cộng sự (2025) [20] đã đề xuất kiến trúc đường ống toàn phần ở cấp độ pixel (Pixel-Level Fully Pipelined). Thay vì yêu cầu bộ nhớ đệm lớn để lưu trữ bản đồ đặc trưng giữa các lớp, kiến trúc này triển khai toàn bộ mạng nơ-ron thành chuỗi nối tiếp, cho phép luồng pixel được xử lý liên tục từ đầu vào đến đầu ra. Quy trình xử lý dựa trên chiến lược

"pixel-by-pixel", trong đó mỗi lớp tính toán tích hợp ba thành phần chính: bộ chọn kênh (MUX), bộ đệm chỉnh lưu (Rectified FIFO) và đơn vị tính toán (CU). Cụ thể, MUX và Rectified FIFO chịu trách nhiệm trích xuất, đồng bộ và chuẩn hóa dữ liệu cửa sổ trượt từ luồng đầu vào để đảm bảo tính liên tục cho đường ống. Tại đơn vị tính toán (CU), hệ thống thực hiện các phép nhân chập song song và áp dụng kỹ thuật ghép kênh theo thời gian (Time-Division Multiplexing). Kỹ thuật này dựa trên các tham số "Initial Sparsity" (IS) và "Pooling Sparsity" (PS), cho phép tái sử dụng tài nguyên DSP cho nhiều tác vụ trong cùng một chu kỳ xung nhịp. Về mặt lưu trữ, toàn bộ trọng số và bias dạng 8-bit fixed-point được lưu trực tiếp trên các khối BRAM nội bộ đặt cạnh đơn vị xử lý. Thiết kế này loại bỏ hoàn toàn việc truy cập DRAM trong quá trình suy luận, giúp giảm độ trễ xuống dưới mức mili-giây và tối đa hóa hiệu quả năng lượng.

2.2.3 Kiến trúc tham chiếu: Tăng tốc CNN trên FPGA dựa trên OpenCL

Zhang và Li (2017)[21] đã đề xuất một giải pháp tăng tốc CNN sử dụng ngôn ngữ OpenCL nhằm cân bằng giữa hiệu năng phần cứng và tính linh hoạt trong lập trình. Hệ thống vận hành theo mô hình tính toán dị thể, bao gồm một CPU chủ (Host) điều khiển luồng chương trình và FPGA (Device) thực thi các tác vụ tính toán chuyên sâu. Để giải quyết nút thắt về băng thông bộ nhớ, nhóm tác giả xây dựng "Mô hình phân tích cân bằng" (Balance Analysis Model) giúp định lượng mối tương quan giữa năng lực tính toán và băng thông, từ đó xác định cấu hình tài nguyên tối ưu để thông lượng không bị giới hạn bởi tốc độ truy xuất Global Memory. Hiệu năng hệ thống được nâng cao nhờ thiết kế kernel OpenCL tối ưu. Cụ thể, các kernel được thiết kế dạng đường ống sâu (deep pipelining) để thực thi song song các chỉ lệnh tích chập. Đồng thời, hệ thống quản lý bộ nhớ phân cấp bằng cách tận dụng tối đa Local Memory/BRAM để lưu đệm các bản đồ đặc trưng và trọng số, giảm thiểu truy cập bộ nhớ ngoài (Off-chip Memory). Các kỹ thuật tối ưu hóa vòng lặp như trải phẳng (loop unrolling) và chia nhỏ dữ

liệu (loop tiling) cũng được áp dụng triệt để. Kết quả thực nghiệm trên Altera Arria 10 cho thấy kiến trúc đạt hiệu suất 866 GOPS và hiệu quả năng lượng vượt trội so với các thiết kế RTL truyền thống.

2.2.4 Kiến trúc tham chiếu: Hệ thống xử lý di thể trên nền tảng RISC-V cho IoT

Hướng đến các ứng dụng IoT với ràng buộc khắt khe về tài nguyên, Liu và cộng sự (2020) [2] đề xuất kiến trúc xử lý di thể kết hợp giữa lõi CPU RISC-V nhúng và khối tăng tốc phần cứng CNN chuyên biệt. Mô hình đồng thiết kế phần cứng/phần mềm này phân chia trách nhiệm rõ ràng: CPU RISC-V đóng vai trò bộ xử lý đa dụng, quản lý luồng chương trình và các tác vụ tiền/hậu xử lý ở tần số thấp (20 MHz) để tiết kiệm năng lượng nền. Trong khi đó, khối CNN Accelerator đảm nhận các phép toán chuyên sâu ở tần số cao hơn (100 MHz) nhằm đảm bảo thông lượng. Giao tiếp giữa hai thành phần được thực hiện qua cơ chế "lệnh vĩ macro" (macro instructions), cho phép CPU cấu hình và kích hoạt Accelerator xử lý trọn vẹn các lớp mạng phức tạp mà không cần can thiệp liên tục. Kiến trúc này minh chứng cho tính hiệu quả khi kết hợp sự linh hoạt của tập lệnh mở RISC-V với hiệu năng xử lý song song của các bộ tăng tốc miền cụ thể (domain-specific accelerators).

2.2.5 Kiến trúc tham chiếu: Bộ tăng tốc luồng cấu hình lại (RSA) cho IoT

Du và cộng sự (2017) [4] giới thiệu kiến trúc "Reconfigurable Streaming Architecture" (RSA) dành cho các thiết bị IoT, với đặc điểm cốt lõi là khả năng xử lý dữ liệu theo luồng liên tục (streaming). RSA loại bỏ hoàn toàn nhu cầu lưu trữ các bản đồ đặc trưng trung gian vào DRAM, giúp giảm đáng kể độ trễ và năng lượng tiêu thụ. Thay vì lưu trữ toàn bộ khung hình, hệ thống sử dụng các bộ đệm dòng (Line Buffers) dựa trên FIFO để lưu tạm thời các dòng pixel đầu

vào cần thiết cho cửa sổ trượt, giúp tối ưu hóa tài nguyên bộ nhớ on-chip. Các phép tính tích chập, pooling và kích hoạt được thực hiện bởi các Đơn vị tính toán cấu hình lại, kết nối qua mạng lưới chuyển mạch tùy biến. Thiết kế này cho phép định tuyến luồng dữ liệu động để hỗ trợ đa dạng kích thước kernel (như 3×3 , 1×1) mà không cần thay đổi phần cứng vật lý. Đồng thời, băng thông bộ nhớ ngoài được dành riêng cho việc nạp trọng số, hoặc trọng số được lưu trực tiếp trên SRAM nội bộ nhằm tối đa hóa hiệu quả năng lượng cho toàn hệ thống.

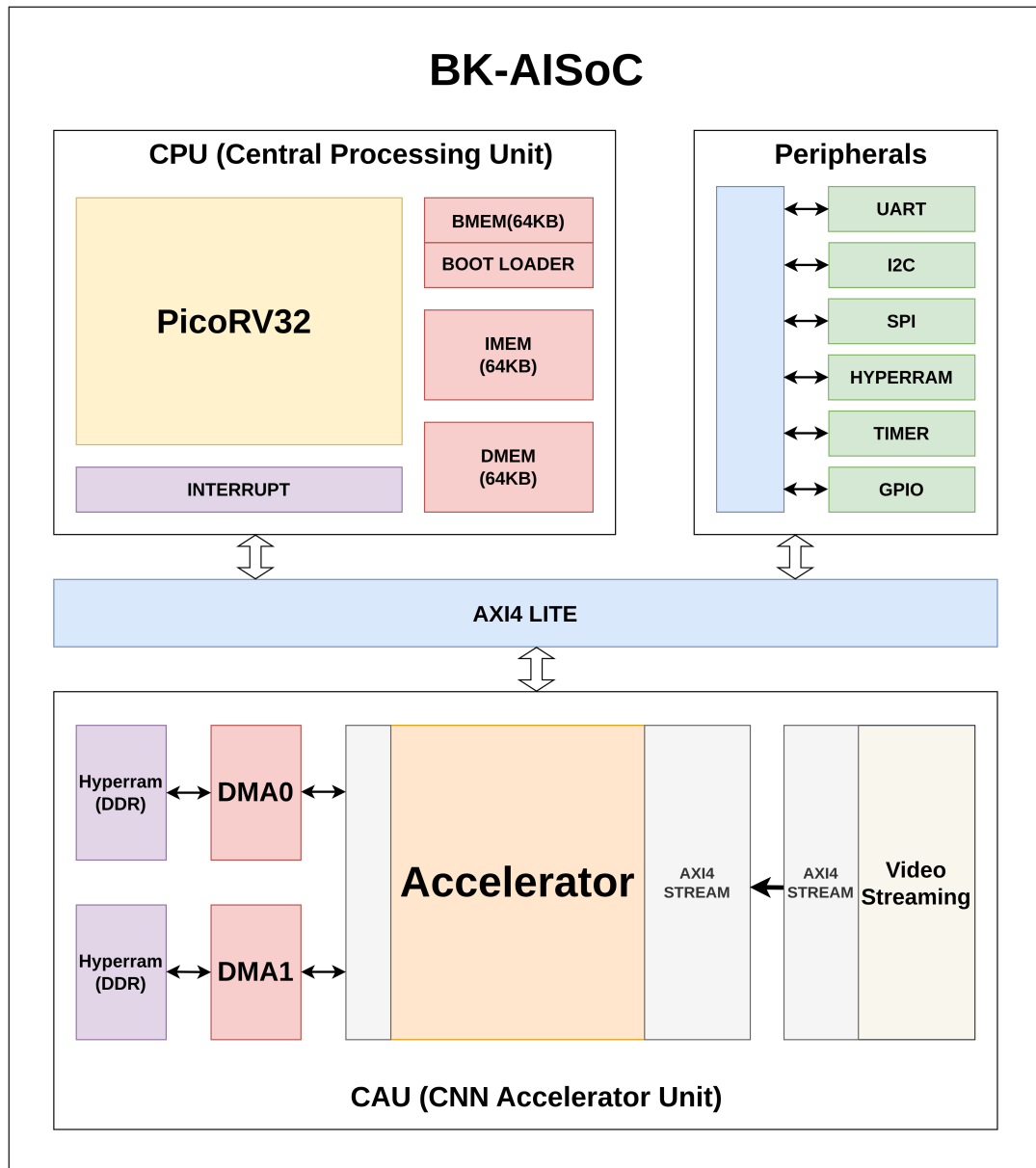
Chương 3

Kiến trúc đề xuất

3.1 Tổng quan kiến trúc đề xuất của BK-AISoC

BK-AISoC được đề xuất như một hệ thống SoC dị thể hướng đến các ứng dụng EdgeAI trên thiết bị IoT [2]. Thay vì để vi xử lý thực hiện toàn bộ luồng suy luận CNN bằng phần mềm, kiến trúc tách hệ thống thành hai miền chức năng chính: miền điều khiển do lõi RISC-V đảm nhiệm và miền xử lý dữ liệu lớn do các khối phần cứng chuyên dụng đảm nhiệm [8]. Cách tổ chức này giúp CPU giữ vai trò điều phối linh hoạt, trong khi các tác vụ có cường độ tính toán và băng thông cao như tích chập, truyền dữ liệu feature map, nạp trọng số và xử lý luồng ảnh được chuyển sang phần cứng chuyên dụng [3], [4].

Mục tiêu của kiến trúc đề xuất không chỉ là xây dựng một bộ gia tốc CNN độc lập, mà là tích hợp bộ gia tốc đó vào một SoC hoàn chỉnh có khả năng khởi động, nạp chương trình, giao tiếp ngoại vi, truy cập bộ nhớ ngoài và chạy thử nghiệm trên FPGA. Vì vậy, BK-AISoC được thiết kế theo hướng cân bằng giữa khả năng mở rộng hệ thống, hiệu năng xử lý CNN, mức sử dụng tài nguyên phần cứng và độ đơn giản của firmware điều khiển.



Hình 3.1: Kiến trúc tổng quan đề xuất của hệ thống BK-AISoC

Như minh họa trong Hình 3.1, BK-AISoC bao gồm lõi xử lý PicoRV32, bộ nhớ on-chip, hệ thống bus điều khiển, các ngoại vi cơ bản, bộ điều khiển truy cập bộ nhớ trực tiếp, bộ nhớ ngoài HyperRAM, khối Video Streaming và bộ gia tốc CNN. Các thành phần này cùng phục vụ một luồng xử lý thống nhất: dữ liệu đầu vào và trọng số được đưa vào hệ thống, CPU cấu hình các thanh ghi điều khiển, phần cứng chuyên dụng vận chuyển dữ liệu đến accelerator, sau đó kết quả được ghi trở lại bộ nhớ hoặc chuyển tiếp cho các khối xử lý phía sau.

3.1.1 Nguyên tắc phân tách control path và data path

Trong BK-AISoC, control path được dùng cho các thao tác có kích thước dữ liệu nhỏ nhưng yêu cầu tính linh hoạt cao, chẳng hạn ghi cấu hình layer, thiết lập địa chỉ buffer, chọn chế độ activation, khởi động DMA hoặc đọc trạng thái hoàn tất. Các thao tác này được thực hiện bởi CPU thông qua không gian địa chỉ memory-mapped [11]. Nhờ đó, firmware có thể điều khiển toàn bộ hệ thống bằng các thao tác đọc/ghi thanh ghi quen thuộc, đồng thời vẫn giữ khả năng thay đổi trình tự chạy cho nhiều mô hình CNN khác nhau.

Ngược lại, data path được thiết kế cho các luồng dữ liệu lớn như ảnh đầu vào, input feature map, trọng số, bias, partial sum và output feature map. Nếu toàn bộ dữ liệu này đi qua CPU, hiệu năng sẽ bị giới hạn bởi số lần load/store tuần tự và overhead phần mềm. Vì vậy, kiến trúc đề xuất sử dụng DMAC, HyperRAM, các FIFO trung gian và giao tiếp dạng streaming để vận chuyển dữ liệu trực tiếp giữa bộ nhớ ngoài, Video Streaming và CNN accelerator [3], [12]. CPU khi đó chỉ đóng vai trò cấu hình và giám sát, không tham gia vào từng giao dịch dữ liệu nhỏ.

3.1.2 Vai trò của các khối chính trong kiến trúc

Vai trò của các khối chính trong kiến trúc BK-AISoC được tóm tắt trong Bảng 3.1. Bảng này nhấn mạnh ranh giới giữa các khối điều khiển, khối lưu trữ, khối truyền dữ liệu và khối tính toán chuyên dụng.

Bảng 3.1: Vai trò của các khối chính trong kiến trúc BK-AISoC

Khối chức năng	Vai trò trong hệ thống
PicoRV32 RISC-V CPU	Điều phối luồng thực thi, cấu hình thanh ghi, khởi động các khối phần cứng và xử lý các tác vụ phần mềm còn lại.
On-chip memory	Lưu chương trình, dữ liệu tạm thời và các vùng bộ nhớ cần độ trễ thấp cho firmware.
AXI/control interconnect	Kết nối CPU với các vùng nhớ và peripheral theo mô hình memory-mapped I/O.
DMAC và HyperRAM	Vận chuyển dữ liệu khối lượng lớn giữa bộ nhớ ngoài và các khối xử lý, giảm số thao tác truyền dữ liệu do CPU thực hiện.
Video Streaming	Thu nhận, đệm và chuẩn bị dữ liệu ảnh đầu vào để phục vụ các pipeline xử lý hình ảnh hoặc AI.
CNN accelerator	Thực hiện các phép tính tích chập có cường độ MAC cao, bao gồm nạp dữ liệu, tính toán, cộng bias, scale và activation.

3.1.3 Luồng xử lý suy luận trên BK-AISoC

Ở mức hệ thống, một luồng suy luận CNN trên BK-AISoC có thể được chia thành bốn bước chính, tương tự hướng đồng thiết kế CPU và accelerator trong các hệ thống CNN nhúng [2]. Đầu tiên, firmware khởi tạo CPU, bộ nhớ, ngoại vi và các thanh ghi cấu hình của accelerator. Tiếp theo, dữ liệu đầu vào, trọng số và bias được đặt vào các vùng nhớ đã quy hoạch trong HyperRAM hoặc buffer nội bộ. Sau đó, CPU ghi thông tin layer vào các thanh ghi điều khiển hoặc instruction table, kích hoạt DMAC và CNN accelerator để thực thi từng lớp tích chập. Cuối cùng, kết quả đầu ra được ghi trở lại bộ nhớ ngoài hoặc được sử dụng cho các bước xử lý tiếp theo trong firmware.

Với cách tổ chức này, giá trị của BK-AISoC nằm ở sự phối hợp giữa kiến trúc

hệ thống và kiến trúc bộ gia tốc. Bộ gia tốc CNN quyết định năng lực tính toán cục bộ, còn SoC quyết định khả năng cấp dữ liệu, điều phối layer, giao tiếp với ngoại vi và chạy thử nghiệm end-to-end. Do đó, các phần tiếp theo của chương sẽ đi từ kiến trúc tổng thể của BK-AISoC xuống thiết kế chi tiết của bộ gia tốc CNN, bắt đầu từ cách phân rã phép tích chập chuẩn và tổ chức dữ liệu cho phần cứng.

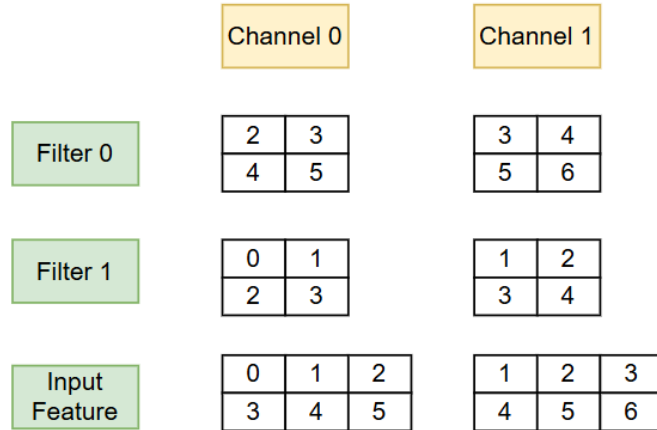
3.2 Phân tích và thiết kế kiến trúc tính toán cho lớp tích chập chuẩn

Trong các mô hình mạng nơ-ron tích chập, mỗi mô hình thường bao gồm nhiều lớp tính toán khác nhau, trong đó lớp tích chập chuẩn (*standard convolution*) là một trong những thành phần có chi phí tính toán lớn nhất [5], [18]. Vì vậy, trong phạm vi đồ án này, kiến trúc được xây dựng tập trung vào việc tăng tốc quá trình tính toán cho các lớp tích chập chuẩn. Với một lớp tích chập, đầu vào là feature map có kích thước $W_i \times H_i \times C_i$, bộ lọc có kích thước $w \times h \times C_i \times C_o$, và đầu ra là output feature map có kích thước $W_o \times H_o \times C_o$. Trong đó, W_i , H_i và C_i lần lượt là chiều rộng, chiều cao và số kênh của feature map đầu vào; w , h là chiều rộng và chiều cao của kernel; còn C_o là số lượng filter, đồng thời cũng là số kênh của feature map đầu ra.

Về mặt toán học, một phần tử đầu ra được tạo thành bằng cách lấy một cửa sổ con của feature map đầu vào, nhân từng phần tử với các hệ số tương ứng của filter, sau đó cộng dồn toàn bộ kết quả trên cả miền không gian của kernel và toàn bộ chiều sâu kênh đầu vào. Tuy nhiên, khi hiện thực trên phần cứng, vấn đề quan trọng không chỉ nằm ở công thức tích chập, mà còn nằm ở cách tổ chức dữ liệu filter, cách đưa dữ liệu ifmap vào các phần tử tính toán, cơ chế cộng dồn partial sum và cách sử dụng buffer sao cho phù hợp với nhiều kích thước layer khác nhau. Do đó, mục này trình bày quá trình phân tích và xây dựng kiến trúc tính toán tích chập, bắt đầu từ trường hợp một channel và sau đó mở rộng sang

nhiều filter, nhiều channel cũng như nhiều mức song song khác nhau.

Để minh họa nguyên lý tổ chức dữ liệu, xét một ví dụ đơn giản trong đó input feature có hai channel, mỗi channel có kích thước 3×2 , và có hai filter đầu ra, mỗi filter gồm hai channel với kích thước kernel 2×2 . Ví dụ này được trình bày trong Hình 3.2 và sẽ được sử dụng xuyên suốt phần giải thích cơ chế tách filter, tổ chức ifmap và cộng dồn kết quả trung gian.



Hình 3.2: Ví dụ minh họa input feature và các filter trên hai channel

3.2.1 Tổ chức lại dữ liệu filter theo hàng và theo kênh

Để đơn giản hóa quá trình ánh xạ phép tích chập lên phần cứng, kiến trúc không xử lý trực tiếp toàn bộ kernel hai chiều như một khối duy nhất. Thay vào đó, mỗi channel của một filter được tách thành nhiều filter con, trong đó mỗi filter con tương ứng với một hàng của filter ban đầu. Nếu filter có chiều cao là h , thì mỗi channel của filter ban đầu sẽ được tách thành h filter con. Cách tổ chức này giúp mỗi phần tử tính toán chỉ cần xử lý một đoạn dữ liệu một chiều của filter tại một thời điểm, từ đó làm giảm độ phức tạp của mạch tính toán cục bộ và thuận lợi hơn cho việc bố trí song song nhiều filter.

Trong ví dụ ở Hình 3.2, mỗi filter có kích thước 2×2 trên mỗi channel. Khi đó, mỗi channel của filter được tách thành hai hàng riêng biệt. Nếu kiến trúc đồng thời xét hai filter đầu ra, mỗi filter có hai hàng, thì sau bước tách dữ liệu ta thu được bốn filter con. Filter con thứ nhất tương ứng với hàng đầu tiên của

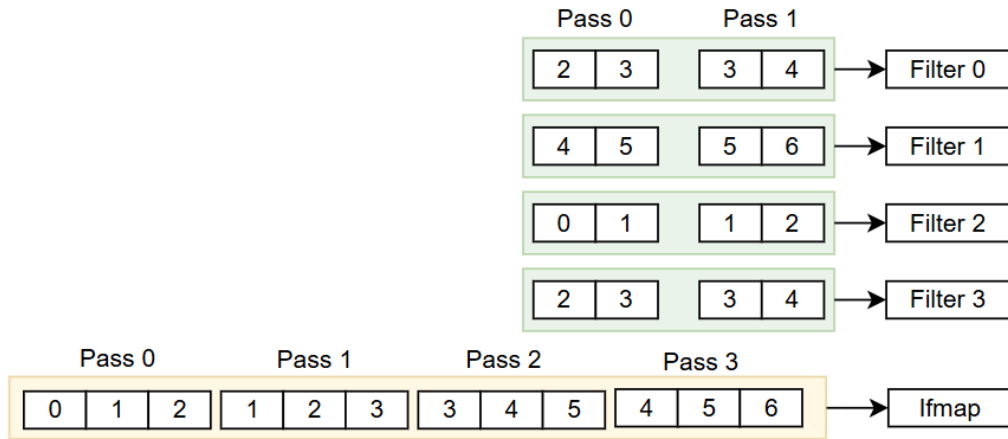
filter đầu ra thứ nhất, filter con thứ hai tương ứng với hàng thứ hai của filter đầu ra thứ nhất. Tương tự, hai filter con còn lại tương ứng với hai hàng của filter đầu ra thứ hai. Như vậy, thay vì xem hai filter ban đầu là hai khối hai chiều, kiến trúc chuyển chúng thành một tập các filter con một chiều để có thể đưa vào các phần tử tính toán theo từng pass.

Cách tách filter theo hàng còn giúp kiến trúc thích ứng tốt hơn với các layer có kích thước kernel khác nhau. Khi chiều cao kernel thay đổi, số lượng filter con sinh ra từ mỗi filter cũng thay đổi tương ứng, nhưng nguyên lý tính toán vẫn được giữ nguyên. Nhờ đó, kiến trúc không bị ràng buộc vào một kích thước kernel cố định, mà có thể hỗ trợ nhiều lớp tích chập khác nhau trong cùng một mô hình.

3.2.2 Tổ chức lại dữ liệu ifmap theo thứ tự quét của kiến trúc

Tương ứng với cách tổ chức lại filter, feature map đầu vào cũng cần được sắp xếp lại để phù hợp với quá trình tính toán theo hàng. Thay vì đưa toàn bộ của số hai chiều của ifmap vào kiến trúc trong cùng một chu kỳ tính toán, ifmap được cắt thành các đoạn hàng một chiều. Các đoạn này được đưa vào kiến trúc theo từng pass, sao cho tại mỗi pass, đoạn ifmap được chọn khớp với hàng filter tương ứng đang được xử lý.

Quá trình tách filter và sắp xếp lại ifmap theo từng pass được minh họa trong Hình 3.3. Trong hình này, các hàng của filter sau khi được tách sẽ được ánh xạ thành các filter con độc lập. Đồng thời, ifmap được duỗi theo từng đoạn hàng và đưa vào kiến trúc theo các pass liên tiếp. Cách tổ chức này giúp dữ liệu filter và ifmap có cùng dạng một chiều tại đầu vào của phần tử tính toán.



Hình 3.3: Tổ chức lại filter và ifmap theo các pass tính toán

Trong kiến trúc này, thứ tự đưa dữ liệu ifmap vào tính toán được tổ chức theo hướng ưu tiên thay đổi theo chiều sâu kênh trước, sau đó mới thay đổi theo chiều cao. Nói cách khác, kiến trúc lần lượt xử lý các đoạn dữ liệu thuộc các channel khác nhau của cùng một vùng không gian trước khi chuyển sang các hàng hoặc vị trí không gian tiếp theo. Cách tổ chức này phù hợp với bản chất của phép tích chập chuẩn, vì kết quả cuối cùng tại một vị trí đầu ra cần được cộng dồn qua toàn bộ chiều sâu C_i của feature map đầu vào.

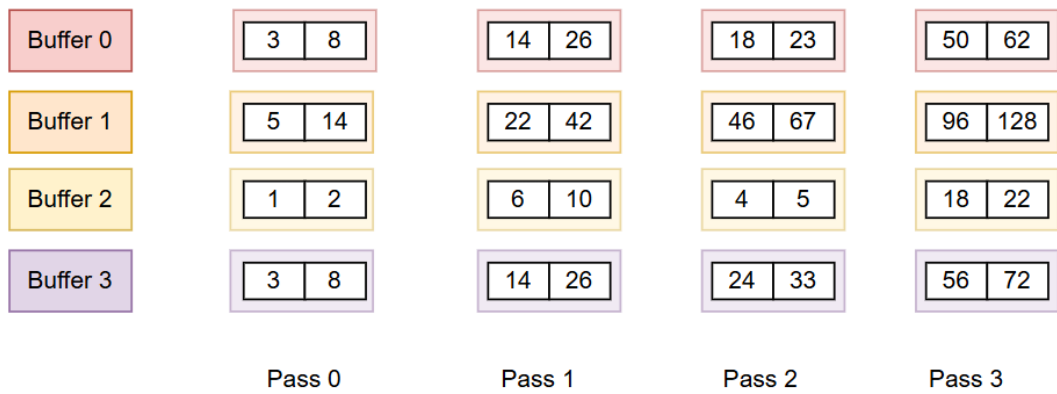
Việc sắp xếp lại ifmap theo pass cũng giúp đơn giản hóa cơ chế điều khiển. Ở mỗi pass, kiến trúc chỉ cần xác định đoạn ifmap hiện tại, nhóm filter con tương ứng và buffer cần ghi hoặc cộng dồn. Các phép nhân–cộng được thực hiện trên dữ liệu một chiều, trong khi kết quả đầy đủ của phép tích chập hai chiều và nhiều channel được hình thành dần thông qua quá trình cộng dồn partial sum qua nhiều pass liên tiếp.

3.2.3 Cơ chế tính toán tuần tự theo pass và cộng dồn trong buffer

Sau khi filter và ifmap được tổ chức lại, phép tích chập được thực thi theo từng pass. Tại mỗi pass, một nhóm filter con được nhân chập với đoạn ifmap tương ứng để tạo ra các kết quả trung gian. Các kết quả này chưa nhất thiết là

giá trị đầu ra cuối cùng, mà thường là các partial sum cần tiếp tục được cộng dồn với kết quả từ các pass khác. Vì vậy, kiến trúc sử dụng các buffer để lưu trữ partial sum và cập nhật chúng trong suốt quá trình tính toán.

Cơ chế cập nhật buffer qua các pass được minh họa trong Hình 3.4. Mỗi hàng trong hình tương ứng với một buffer, còn mỗi cột tương ứng với trạng thái kết quả sau một pass. Các giá trị trong buffer cho thấy kết quả trung gian không được tạo ra độc lập ở từng pass, mà được tích lũy dần để hình thành kết quả tích chập đầy đủ.



Hình 3.4: Cơ chế cộng dồn partial sum trong các buffer qua từng pass

Ở pass đầu tiên, các buffer nhận kết quả trực tiếp từ phép tính giữa filter con và đoạn ifmap tương ứng. Vì tại thời điểm này chưa có kết quả trung gian trước đó cho cùng vị trí đầu ra, giá trị mới có thể được ghi trực tiếp vào buffer. Sang các pass tiếp theo, nếu kết quả hiện tại thuộc cùng một phần tử đầu ra với kết quả đã lưu trước đó, kiến trúc sẽ thực hiện phép cộng dồn thay vì ghi đè. Nhờ đó, partial sum được tích lũy dần theo chiều cao kernel, chiều rộng kernel và chiều sâu kênh đầu vào.

Đối với trường hợp nhiều channel, buffer đóng vai trò đặc biệt quan trọng. Khi kiến trúc chuyển từ channel này sang channel tiếp theo, kết quả tích chập trên channel mới phải được cộng với partial sum đã thu được từ các channel trước đó. Quá trình này được lặp lại cho đến khi toàn bộ C_i channel đầu vào đã được xử lý. Khi đó, giá trị trong buffer mới tương ứng với kết quả đầy đủ của một phần tử output feature map đối với một filter đầu ra cụ thể.

Bên cạnh chiều sâu kênh, việc lựa chọn buffer để ghi mới hoặc cộng dồn còn phụ thuộc vào vị trí không gian của cửa sổ tích chập, giá trị stride và padding. Với stride lớn hơn một, các cửa sổ đầu vào được lấy cách nhau theo một khoảng nhất định, dẫn đến thứ tự sinh kết quả đầu ra cũng thay đổi. Với padding, một số phần tử trong cửa sổ tích chập có thể tương ứng với vùng đệm thay vì dữ liệu thật của ifmap. Do đó, bộ điều khiển của kiến trúc cần xác định chính xác tại mỗi pass dữ liệu hiện tại đang đóng góp vào phần tử output nào, từ đó chọn đúng buffer và đúng chế độ cập nhật.

Có thể hiểu quá trình tính toán theo pass như một cách phân rã phép tích chập đầy đủ thành nhiều phép tính nhỏ hơn. Mỗi pass chỉ xử lý một phần của kernel và một phần tương ứng của ifmap. Kết quả cuối cùng không được tạo ra trong một bước duy nhất, mà được hình thành thông qua quá trình cộng dồn có điều khiển trong buffer. Cách tiếp cận này giúp kiến trúc sử dụng lại cùng một tập phần tử tính toán cho nhiều phần khác nhau của kernel, đồng thời vẫn duy trì khả năng xử lý song song nhiều filter, tương đồng với nguyên tắc khai thác tái sử dụng dữ liệu trong các accelerator CNN chuyên dụng [3].

3.2.4 Mô hình hóa phần tử tính toán và điều kiện cộng dồn

Để mô tả hoạt động của kiến trúc ở mức tổng quát, có thể xem mỗi phép tính giữa một filter con và một đoạn ifmap là một phép tích chập một chiều. Gọi x_i là filter con thứ i và y là đoạn ifmap tương ứng tại pass hiện tại, phép tính cục bộ có thể được ký hiệu là

$$\text{conv}(x_i, y). \quad (3.1)$$

Giá trị này biểu diễn kết quả nhân-cộng giữa một hàng filter và một đoạn hàng của ifmap. Tùy thuộc vào vị trí của filter con, channel hiện tại và trạng thái của buffer, kết quả $\text{conv}(x_i, y)$ sẽ được ghi mới vào buffer hoặc cộng với giá trị đã có trước đó.

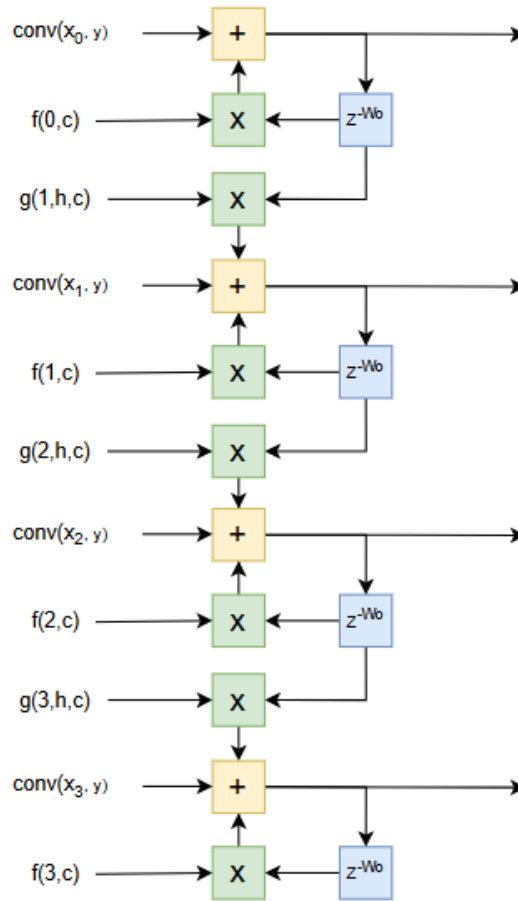
Trong kiến trúc, cần có một logic điều khiển để xác định khi nào partial sum phải được khởi tạo lại và khi nào nó phải được cộng dồn. Có thể mô hình hóa

logic này thông qua hàm nhị phân $f(i, c)$, trong đó i là chỉ số filter con và c là chỉ số channel hiện tại. Hàm này nhận giá trị bằng 0 khi kiến trúc đang xử lý channel đầu tiên, tương ứng với trường hợp chưa có partial sum theo chiều sâu cần cộng thêm. Ngược lại, khi c không phải là channel đầu tiên, kết quả mới cần được cộng với giá trị đã lưu từ các channel trước đó.

Bên cạnh cộng dồn theo chiều sâu kênh, kiến trúc cũng cần kiểm soát quá trình cộng dồn giữa các hàng khác nhau của cùng một filter. Điều này có thể được mô hình hóa bằng hàm nhị phân $g(i, h, c)$, trong đó h là chiều cao của kernel. Hàm này cho biết liệu filter con hiện tại có thuộc hàng tiếp theo của cùng một filter gốc hay không, từ đó xác định việc kết quả hiện tại có cần cộng với partial sum đã sinh ra bởi hàng trước đó hay không. Với cách mô hình hóa này, quá trình cập nhật buffer có thể được biểu diễn ở dạng tổng quát như sau:

$$B_{\text{new}} = \text{conv}(x_i, y) + f(i, c)B_{\text{channel}} + g(i, h, c)B_{\text{row}}, \quad (3.2)$$

trong đó B_{channel} biểu diễn partial sum đã tích lũy từ các channel trước, còn B_{row} biểu diễn partial sum đã tích lũy từ các hàng trước của cùng một vùng kernel.



Hình 3.5: Mô hình hóa datapath và logic cộng dồn partial sum

Biểu thức trong (3.2) không nhằm thay thế cho mô tả chi tiết ở mức RTL, mà đóng vai trò như một mô hình khái quát cho cơ chế cộng dồn trong kiến trúc. Thông qua các điều kiện điều khiển này, kiến trúc có thể phân biệt được ba trường hợp quan trọng: ghi mới partial sum, cộng dồn theo chiều cao kernel và cộng dồn theo chiều sâu channel. Đây là cơ sở để mở rộng kiến trúc từ trường hợp một filter, một channel sang trường hợp nhiều filter và nhiều channel được xử lý song song.

3.2.5 Khả năng mở rộng mức song song của kiến trúc

Từ cơ chế tính toán theo filter con và pass, kiến trúc có thể được mở rộng theo hai hướng chính: song song theo chiều sâu channel đầu vào và song song theo số lượng filter đầu ra. Song song theo channel giúp tăng số lượng dữ liệu

ifmap được xử lý trong cùng một khoảng thời gian, từ đó giảm số pass cần thiết để hoàn tất quá trình cộng dồn theo chiều sâu C_i . Tuy nhiên, việc tăng mức song song theo channel cũng làm tăng số lượng dữ liệu cần đọc đồng thời và số partial sum cần quản lý, do đó bị giới hạn bởi băng thông bộ nhớ, số lượng phần tử tính toán và dung lượng buffer.

Trong phạm vi hiện thực của đề án, số channel ifmap được xử lý song song được giới hạn ở mức tối đa là 4. Đây là một lựa chọn mang tính cân bằng giữa khả năng khai thác song song và chi phí tài nguyên phần cứng. Nếu số channel song song quá nhỏ, kiến trúc cần nhiều pass hơn để hoàn tất một layer, làm giảm thông lượng tính toán. Ngược lại, nếu số channel song song quá lớn, yêu cầu về bộ nhớ, mạch cộng dồn và số lượng đường dữ liệu sẽ tăng đáng kể, trong khi hiệu quả khai thác tài nguyên chưa chắc tăng tương ứng.

Đối với hướng song song theo filter đầu ra, mỗi phần tử tính toán có thể được xem như một PE (*Processing Element*). Giả sử kiến trúc sử dụng tổng cộng 24 PE, số filter có thể xử lý song song trong một layer sẽ phụ thuộc vào độ rộng của filter tại layer đó. Với filter có độ rộng càng lớn, mỗi filter cần nhiều PE hơn để xử lý một hàng dữ liệu, do đó số filter song song tối đa sẽ giảm. Ngược lại, với các filter có độ rộng nhỏ, kiến trúc có thể xử lý nhiều filter đầu ra hơn trong cùng một thời điểm.

Có thể biểu diễn số filter song song tối đa theo quan hệ gần đúng:

$$N_{\text{filterparallel}} = \left\lfloor \frac{N_{\text{PE}}}{w} \right\rfloor, \quad (3.3)$$

trong đó N_{PE} là tổng số phần tử tính toán khả dụng và w là độ rộng của kernel. Quan hệ trong (3.3) cho thấy mức song song của kiến trúc không cố định cho mọi layer, mà thay đổi tùy thuộc vào kích thước kernel của từng lớp tích chập. Đây là một đặc điểm quan trọng, vì trong các mô hình CNN thực tế, các layer khác nhau có thể sử dụng các kích thước kernel và số lượng filter khác nhau.

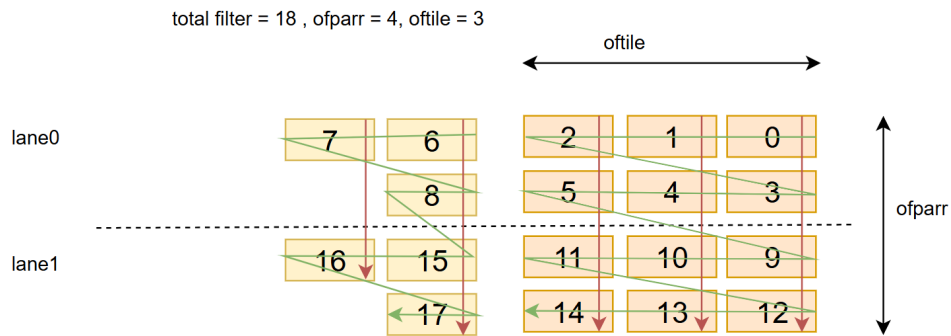
3.2.6 Tối ưu bộ nhớ bằng cơ chế tile filter

Ngoài bài toán tính toán, một vấn đề quan trọng khác trong thiết kế kiến trúc là sử dụng bộ nhớ hiệu quả. Trong các mô hình CNN, các layer đầu thường có feature map với kích thước không gian lớn, trong khi các layer sau có kích thước không gian nhỏ hơn nhưng số lượng channel hoặc số filter có thể tăng lên [5]. Nếu kiến trúc chỉ sử dụng một chiến lược cố định, chẳng hạn luôn tính một nhóm filter song song rồi ghi kết quả ra bộ nhớ trước khi chuyển sang nhóm filter tiếp theo, thì tài nguyên buffer có thể bị lãng phí ở các layer sâu, nơi kích thước không gian của ifmap và ofmap đã giảm đáng kể.

Để khắc phục vấn đề này, kiến trúc bổ sung thêm khái niệm tile filter bên cạnh tham số số filter được xử lý song song. Thay vì chỉ tính một nhóm filter song song tại một thời điểm, kiến trúc có thể tiếp tục xử lý thêm các tile filter khác tại cùng một vùng chiều cao của ifmap và lưu trữ kết quả trong buffer. Khi kích thước không gian của feature map giảm ở các layer sau, phần dung lượng buffer còn dư có thể được dùng để chứa kết quả của nhiều tile filter hơn. Nhờ đó, mức sử dụng bộ nhớ được cải thiện và số lần truy cập bộ nhớ ngoài có thể được giảm xuống.

Cơ chế tile filter đặc biệt hữu ích trong bối cảnh kích thước feature map thay đổi đáng kể giữa các layer. Ở các layer đầu, do kích thước ifmap lớn, buffer chủ yếu được dùng để lưu partial sum hoặc output cho một số lượng filter song song giới hạn. Ở các layer sau, khi kích thước ifmap nhỏ hơn, cùng một dung lượng buffer có thể chứa được nhiều nhóm kết quả hơn. Khi đó, việc tăng số tile filter giúp bù lại sự suy giảm kích thước không gian bằng cách tăng mức song song theo chiều filter, từ đó cải thiện hiệu quả khai thác tài nguyên phần cứng. Đây cũng là hướng tối ưu thường gặp trong các thiết kế accelerator FPGA, nơi việc chia nhỏ dữ liệu và tái sử dụng bộ nhớ cục bộ giúp giảm áp lực lên bộ nhớ ngoài [3], [21]. Ở đây nói thêm về cách kiến trúc sắp xếp các filter và bias khi được lưu vào offchip memory. Các mũi tên màu xanh đại diện cho thứ tự nạp filter, còn mũi tên đỏ đại diện cho thứ tự nạp bias. Các mũi tên tách rời nhau thể hiện

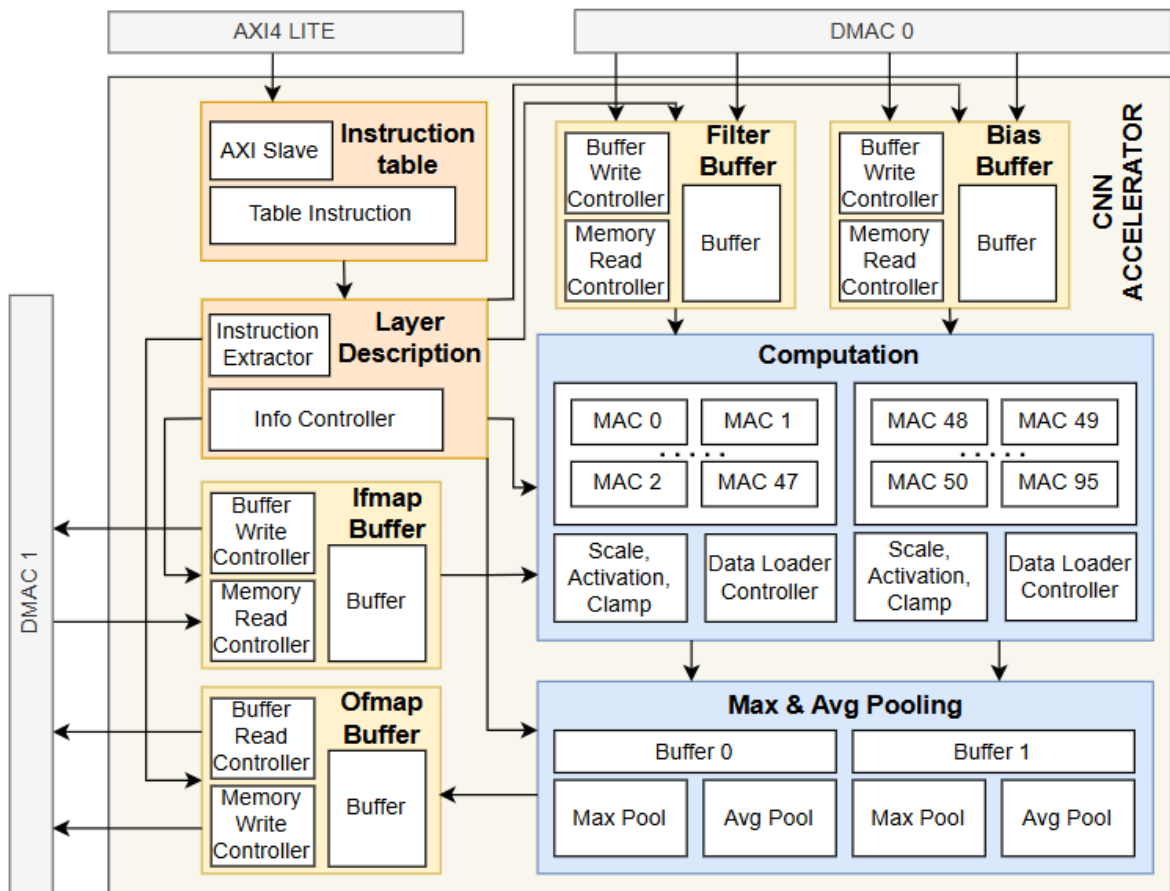
cách nạp dữ liệu theo từng pass. Cách tổ chức này giúp kiến trúc dễ dàng truy xuất dữ liệu filter và bias theo thứ tự cần thiết trong quá trình tính toán, đồng thời hỗ trợ cơ chế tile filter một cách linh hoạt. Lane 0 và Lane 1 tương ứng với 2 buffer dùng để lưu trữ giá trị ofmap, việc sắp xếp filter và bias ngay từ đầu giúp việc phân phối và lưu dữ liệu ofmap được đơn giản hơn.



Hình 3.6: Minh họa cách tổ chức dữ liệu filter và bias trong bộ nhớ ngoài

Tóm lại, kiến trúc tính toán tích chập trong đề án được xây dựng dựa trên việc phân rã phép tích chập hai chiều thành các phép tính một chiều theo hàng filter và đoạn ifmap tương ứng. Các kết quả trung gian được cộng dồn trong buffer thông qua cơ chế điều khiển theo pass, theo channel và theo hàng kernel. Cách tổ chức này cho phép kiến trúc hỗ trợ nhiều kích thước filter khác nhau, mở rộng mức song song theo cả channel và filter, đồng thời cải thiện hiệu quả sử dụng bộ nhớ thông qua cơ chế tile filter. Đây là nền tảng để hiện thực các lớp standard convolution trong mô hình CNN trên kiến trúc phần cứng được đề xuất.

3.3 Kiến trúc tổng quan của bộ gia tốc CNN



Hình 3.7: Kiến trúc tổng quan của bộ gia tốc CNN

Bộ gia tốc CNN được xây dựng theo kiến trúc module hóa, trong đó các khối chức năng được tách biệt rõ ràng nhằm tối ưu luồng dữ liệu giữa bộ nhớ đệm và khối tính toán. Cách nhấn mạnh phân cấp bộ nhớ và tái sử dụng dữ liệu này phù hợp với các nghiên cứu về accelerator CNN tiết kiệm năng lượng [3], [4]. Như minh họa ở Hình 3.7, hệ thống bao gồm bốn nhóm thành phần chính.

Thứ nhất, *Instruction Table* đóng vai trò là vùng đệm lưu trữ tập lệnh điều khiển do CPU cung cấp cho từng lớp của mạng CNN. Thứ hai, *Layer Description* đảm nhận chức năng giải mã tập lệnh và phân phối các thông số cấu hình đến toàn bộ phần cứng gia tốc. Thứ ba, hệ thống bộ đệm dữ liệu bao gồm *Buffer Filter*, *Buffer Ifmap*, *Buffer Bias* và *Buffer Ofmap*, có nhiệm vụ lưu trữ tạm thời

các tensor trọng số, đặc trưng đầu vào, độ chệch và đặc trưng đầu ra nhằm giảm số lần truy cập bộ nhớ ngoài. Cuối cùng, *Computation Unit* là lõi tính toán của hệ thống, thực hiện các phép nhân tích lũy giữa dữ liệu Ifmap và Filter để tạo ra kết quả tích chập.

Cách tổ chức này cho phép tách biệt đường điều khiển với đường dữ liệu, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và tối ưu từng khối chức năng một cách độc lập [2].

Các chức năng mà CNN accelerator hỗ trợ bao gồm:

- Standard Convolution
- Kích thước filter là hình vuông và chiều dài không quá 12
- Kích thước ifmap và ofmap là hình vuông và chiều rộng không quá 224.
- Số lượng channel đầu vào tối đa là 1024, số lượng filter đầu ra tối đa là 1024.
- Hỗ trợ stride không quá 7, hỗ trợ padding không quá 3.
- Hỗ trợ activation ReLU và Leaky ReLU (được thực thi theo pipeline).

3.4 Quy trình điều khiển và luồng dữ liệu

Quy trình vận hành của bộ gia tốc được thực hiện tuần tự nhằm bảo đảm tính nhất quán của cấu hình và tính liên tục của luồng dữ liệu. Trước hết, CPU nạp tập lệnh mô tả từng lớp mạng vào *Instruction Table*. Từ đây, *Layer Description* đọc tập lệnh của lớp hiện tại, giải mã các tham số như kích thước bộ lọc, stride, padding và các chế độ xử lý liên quan, sau đó phân phối các tín hiệu cấu hình đến các khối bộ đệm và khối tính toán.

Sau khi nhận được cấu hình, các khối *Buffer Filter*, *Buffer Ifmap* và *Buffer Bias* khởi tạo quá trình truy xuất dữ liệu từ bộ nhớ ngoài. Dữ liệu trọng số, đặc trưng đầu vào và độ chệch được nạp vào các vùng đệm tương ứng. Khi dữ liệu

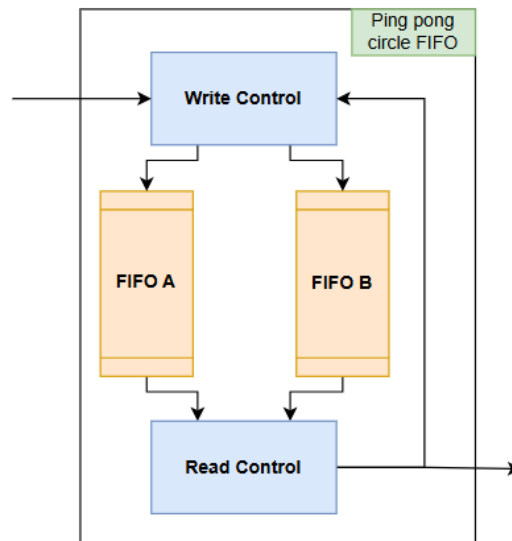
đã sẵn sàng, khối *Computation* tiến hành thực hiện các phép nhân tích lũy song song để tạo ra các giá trị trung gian của đặc trưng đầu ra. Tại đây, hệ thống thực hiện các bước hậu xử lý, bao gồm cộng độ chệch, hiệu chỉnh tỉ lệ và kích hoạt ReLU.

Kết quả tính toán sau đó được chuyển đến *Buffer Ofmap* với nhiệm vụ ghi kết quả cuối cùng trở lại bộ nhớ ngoài. Trình tự này giúp bảo đảm rằng mỗi lớp được xử lý trọn vẹn trước khi hệ thống chuyển sang lớp tiếp theo, qua đó tránh xung đột dữ liệu giữa các giai đoạn xử lý.

3.5 Các thành phần logic cơ sở

Để duy trì khả năng xử lý liên tục và tăng mức độ tái sử dụng dữ liệu, bộ gia tốc sử dụng hai cấu trúc lưu trữ cơ sở làm nền tảng cho các khối bộ đệm và một phần logic xử lý nội bộ, gồm *Ping-pong FIFO* và *Ping-pong Circle FIFO*.

3.5.1 Cấu trúc Ping-pong FIFO



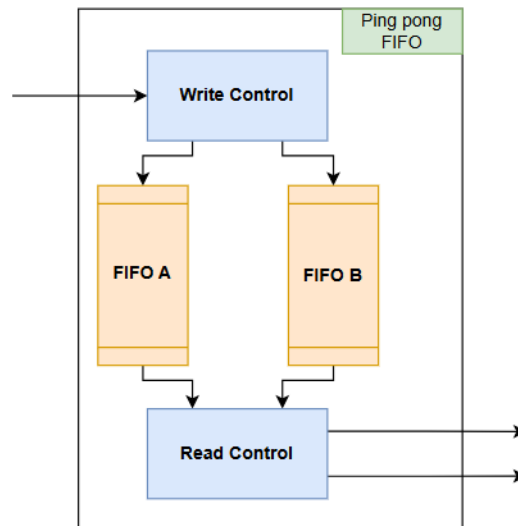
Hình 3.8: Cấu trúc Ping-pong FIFO

Ping-pong FIFO được hình thành từ hai hàng đợi FIFO độc lập kết hợp với logic điều khiển đọc/ghi luân phiên. Tại một thời điểm, dữ liệu mới được ghi vào

một FIFO, trong khi FIFO còn lại đồng thời cung cấp dữ liệu cho khối tiêu thụ. Khi cả hai quá trình hoàn tất, vai trò của hai FIFO được hoán đổi.

Cơ chế này cho phép che giấu độ trễ nạp dữ liệu từ bộ nhớ, vì hệ thống có thể vừa tiếp tục xử lý dữ liệu hiện tại, vừa chuẩn bị dữ liệu cho chu kỳ kế tiếp. Nhờ đó, các đơn vị tính toán không phải dừng chờ dữ liệu và hiệu suất hoạt động liên tục của hệ thống được cải thiện đáng kể. Đây là cùng mục tiêu với các kỹ thuật double buffering và local buffering thường được dùng để giảm truy cập bộ nhớ ngoài trong accelerator CNN [3], [21].

3.5.2 Cấu trúc Ping-pong Circle FIFO



Hình 3.9: Cấu trúc Ping-pong Circle FIFO

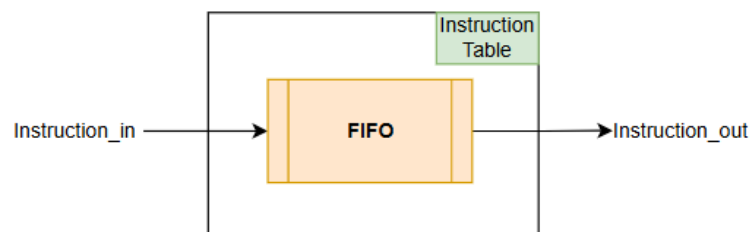
Ping-pong Circle FIFO là biến thể mở rộng của *Ping-pong FIFO*, được thiết kế cho các loại dữ liệu cần được tái sử dụng nhiều lần, điển hình là trọng số bộ lọc và dữ liệu đặc trưng đầu vào. Điểm khác biệt cốt lõi nằm ở cơ chế vòng lặp tại khối đọc: khi một phần tử dữ liệu được đọc ra để phục vụ tính toán, phần tử đó không bị loại bỏ khỏi hàng đợi mà được phản hồi trở lại đầu vào ghi, nhờ đó tiếp tục tồn tại trong FIFO để phục vụ cho các lượt xử lý kế tiếp.

Nhờ cơ chế này, hệ thống có thể tái sử dụng dữ liệu đã nạp mà không cần truy xuất lại từ bộ nhớ ngoài. Đồng thời, việc kết hợp với cơ chế ping-pong vẫn

cho phép một nhánh FIFO phục vụ tính toán trong khi nhánh còn lại chuẩn bị dữ liệu cho lượt tiếp theo. Sự kết hợp giữa tái sử dụng dữ liệu và chồng lấp nạp dữ liệu giúp giảm đáng kể độ trễ truy cập bộ nhớ [3].

3.6 Kiến trúc chi tiết các khối chức năng

3.6.1 Khối Instruction Table

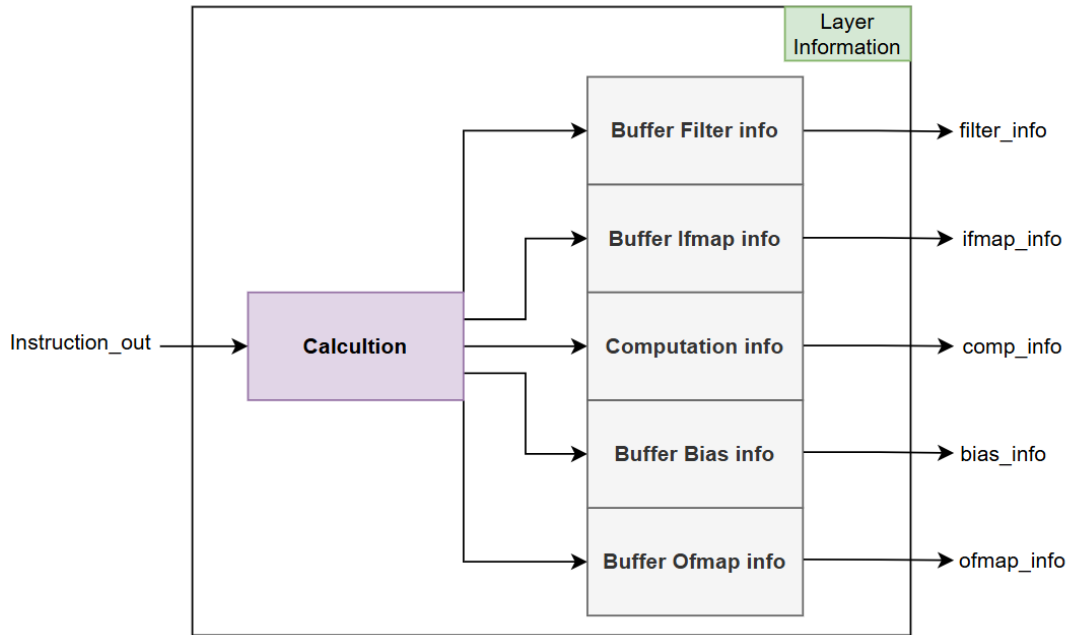


Hình 3.10: Khối Instruction Table

Instruction Table được xây dựng trên cấu trúc FIFO nhằm bảo đảm tính tuần tự và toàn vẹn của luồng dữ liệu điều khiển. Khối này tiếp nhận các tập lệnh từ CPU thông qua tín hiệu *Instruction_in* và lưu trữ chúng theo đúng thứ tự lớp của mạng CNN.

Việc sử dụng FIFO giúp hệ thống duy trì tiến trình thực thi ổn định: tập lệnh của lớp kế tiếp chỉ được xuất ra tại *Instruction_out* khi lớp hiện tại đã hoàn tất quá trình xử lý. Nhờ đó, các cấu hình điều khiển được áp dụng tuần tự, tránh chồng lấn hoặc xung đột giữa các lớp.

3.6.2 Khối Layer Information



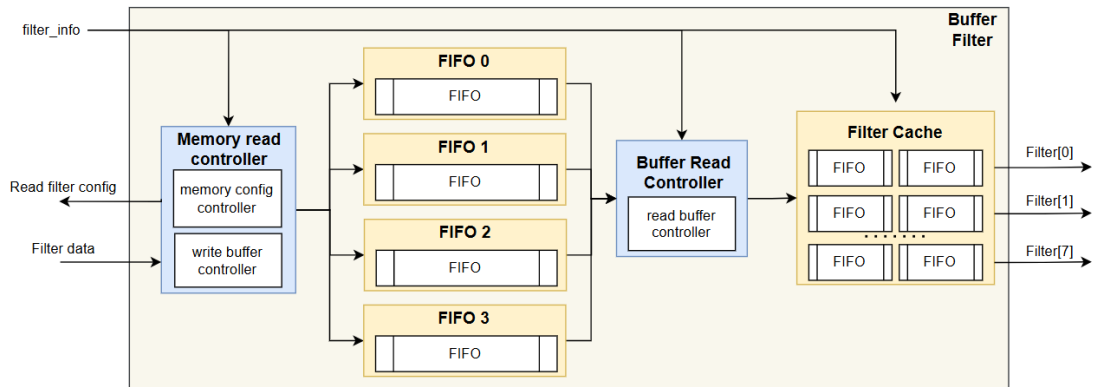
Hình 3.11: Khối Layer Information

Layer Information là trung tâm điều phối cấu hình của toàn bộ accelerator. Nhiệm vụ chính của khối là giải mã tập lệnh đầu ra từ *Instruction Table* và ánh xạ chúng thành các tham số kỹ thuật cụ thể cho từng khối chức năng.

Về mặt kiến trúc, khối gồm hai thành phần chính. Thành phần thứ nhất là *Lookup Table*, có nhiệm vụ giải mã tập lệnh thành các tham số như kích thước dữ liệu, chế độ tính toán và cấu hình truy cập bộ nhớ. Thành phần thứ hai là hệ thống thanh ghi cấu hình, dùng để lưu tạm các tham số sau giải mã và phân phối chúng đến các khối tương ứng thông qua các nhóm tín hiệu *filter_info*, *ifmap_info*, *comp_info*, *bias_info* và *ofmap_info*.

Việc tách các nhóm tham số theo từng khối chức năng giúp hệ thống duy trì trạng thái cấu hình ổn định trong suốt thời gian xử lý một lớp, đồng thời tạo điều kiện chuyển lớp linh hoạt mà không làm gián đoạn toàn bộ datapath.

3.6.3 Khối Buffer Filter



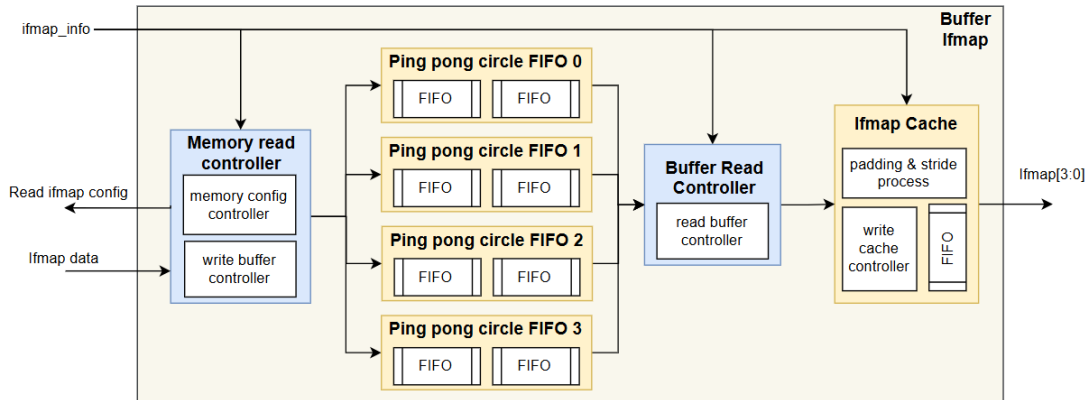
Hình 3.12: Kiến trúc khối Buffer Filter

Buffer Filter chịu trách nhiệm lưu trữ và cung cấp trọng số cho khối tính toán. Để bảo đảm băng thông cấp dữ liệu đủ lớn và duy trì tính liên tục của luồng xử lý, khối được tổ chức theo cấu trúc phân cấp gồm bộ điều khiển truy cập bộ nhớ và hệ thống hàng đợi song song.

Khối *Write Mem Control* tiếp nhận thông tin cấu hình từ `filter_info`, thiết lập địa chỉ gốc và tham số truy cập bộ nhớ ngoài, sau đó điều phối dữ liệu trọng số vào các hàng đệm nội bộ. Hệ thống hàng đệm của *Buffer Filter* bao gồm 8 khối *Cache*, cho phép cung cấp đồng thời 8 giá trị trọng số cho 4 channel của 2 filter cho khối *Computation*.

Điểm nhấn của khối nằm ở cơ chế tải trọng số lần tính hàng đầu tiên của toàn bộ kênh, những hàng tiếp theo sẽ không cần tải lại trọng số. Cung cấp cơ chế vòng lặp cho phép các trọng số đã được đọc tiếp tục được giữ lại trong hàng đệm để sử dụng nhiều lần trong quá trình trượt cửa sổ trên *Ifmap*, nhờ đó giảm số lần truy xuất bộ nhớ ngoài.

3.6.4 Khối Buffer Ifmap



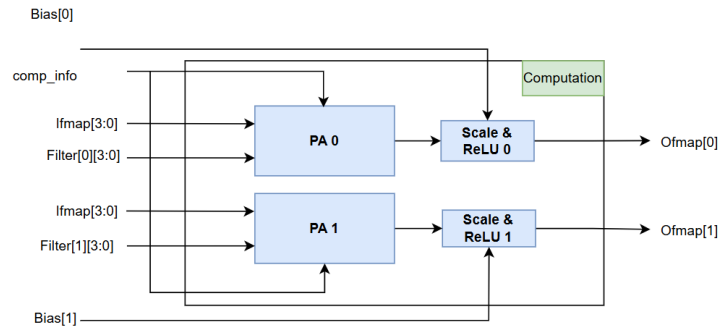
Hình 3.13: Kiến trúc khối Buffer Ifmap

Buffer Ifmap có vai trò lưu trữ và tổ chức dữ liệu đặc trưng đầu vào trước khi chuyển tới khối tính toán. Về cấu trúc, khối này khác với *Buffer Filter*, sử dụng *Ping Pong Circle FIFO* để lưu trữ tối đa 8 kênh dữ liệu ifmap để sử dụng thay vì phải lưu trữ hết toàn bộ kênh dữ liệu.

Khối *Write Mem Control* tiếp nhận cấu hình từ *ifmap_info*, tính toán địa chỉ truy xuất và điều phối dữ liệu *Ifmap data* vào các hàng đệm. Hệ thống hàng đệm gồm 4 khối *Ping-pong Circle FIFO*, tương ứng với khả năng xử lý song song 4 kênh dữ liệu đầu vào. Phía sau các hàng đệm là khối *Ifmap Cache*, chịu trách nhiệm tổ chức cửa sổ tính toán dựa trên kích thước hàng bộ lọc và tham số stride.

Về chiến lược tái sử dụng dữ liệu, *Buffer Ifmap* tận dụng cơ chế *Circle FIFO* để giữ lại các hàng dữ liệu đầu vào sau khi đọc, cho phép sử dụng lại chúng cho các cụm bộ lọc khác nhau mà không cần nạp lại từ bộ nhớ ngoài. Đồng thời, cơ chế *Ping-pong* cho phép chồng lấp giữa nạp dữ liệu và tính toán, qua đó giảm thời gian chờ và tăng hiệu suất sử dụng phần cứng.

3.6.5 Khối Computation

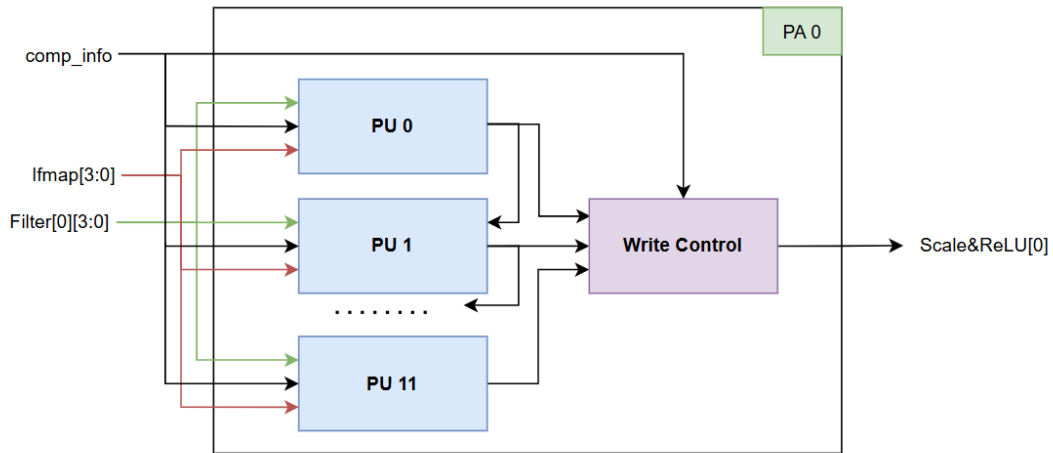


Hình 3.14: Kiến trúc tổng quan khối Computation

Computation là lõi thực thi các phép nhân tích lũy của tầng tích chập. Để tăng thông lượng và đồng thời giảm nguy cơ tắc nghẽn ở đầu ra, khối này được thiết kế theo cấu trúc phân cấp và phân luồng song song.

Ở mức cao nhất, *Computation* được chia thành hai nhánh xử lý độc lập, tương ứng với hai *Processing Array* là *PA 0* và *PA 1*. Toàn bộ 4 kênh dữ liệu đầu vào *Ifmap[3:0]* được phân phối đồng thời tới cả hai nhánh, trong khi tập trọng số được chia đôi: *PA 0* tiếp nhận lần lượt 4 giá trị tương ứng 4 channel cho tới hết kích thước 1 channel của 1 filter nhân với số filter song song được phân chia *Filter[0][3:0]* và *PA 1* tiếp nhận *Filter[1][3:0]*. Cấu trúc này cho phép xử lý song song hai nhóm bộ lọc, đồng thời tách riêng luồng kết quả đầu ra thành *Ofmap[0]* và *Ofmap[1]*, phù hợp với tổ chức hai FIFO của *Buffer Ofmap*.

3.6.5.1 Mảng xử lý Processing Array (PA)

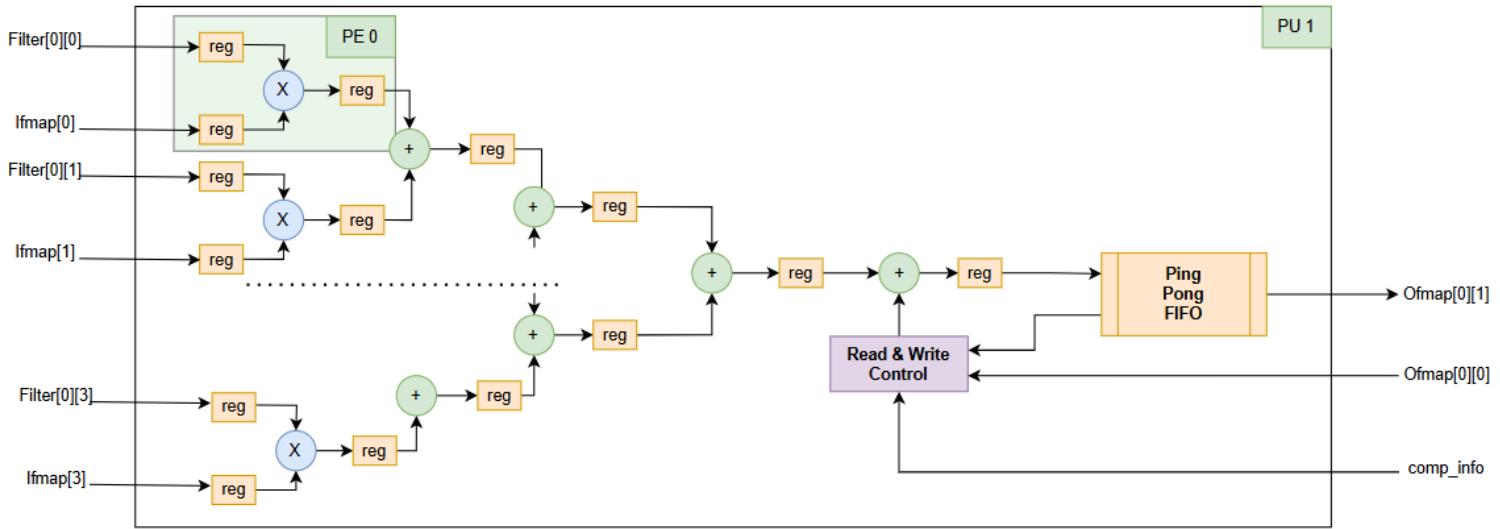


Hình 3.15: Kiến trúc Mảng xử lý Processing Array (PA)

Mỗi *Processing Array* được xây dựng từ 12 khối *Processing Unit* hoạt động song song. Mỗi PU lần lượt nhận 4 channel của 1 filter tương ứng và đảm nhiệm phép tích chập giữa 4 kênh Ifmap và 4 kênh trọng số tương ứng của một bộ lọc, nhờ đó toàn bộ PA có thể xử lý đồng thời nhiều bộ lọc trong cùng một chu kỳ xử lý.

Bên cạnh các PU, mỗi PA còn tích hợp khối *Write Control*, có nhiệm vụ thu thập các giá trị đặc trưng đầu ra từ các PU, sắp xếp chúng theo cơ chế xen kẽ hàng và điều phối quá trình ghi sang *Buffer Ofmap*. Khối này đồng thời xử lý các trường hợp biên và tham số stride, quyết định hàng dữ liệu nào được giữ lại hoặc loại bỏ trong quá trình tạo đặc trưng đầu ra hợp lệ.

3.6.5.2 Đơn vị xử lý Processing Unit (PU)



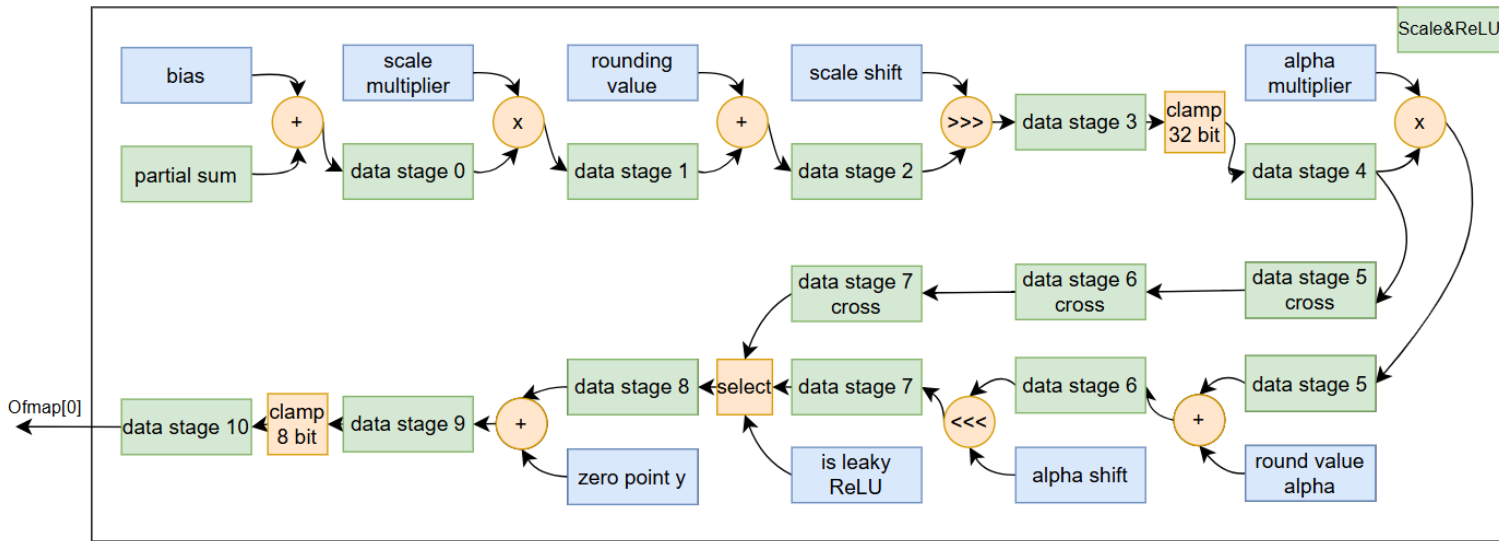
Hình 3.16: Kiến trúc Đơn vị xử lý (PU)

Processing Unit là đơn vị tính toán cơ bản của toàn bộ accelerator. Kiến trúc bên trong PU được tối ưu cho bài toán cộng dồn nhiều tích số trên 4 kênh dữ liệu đầu vào. Các phép nhân và cộng được tổ chức theo cấu trúc pipeline nhiều tầng nhằm rút ngắn đường truyền tối hạn và nâng cao tần số hoạt động. Như vậy, về cơ bản kiến trúc có tất cả 96 phần tử tính toán (PE) hoạt động song song trong toàn bộ *Computation*, với 1 PE phụ trách 1 phép nhân.

Bên trong PU, khối *Read & Write Control* giữ vai trò điều phối logic cộng dồn, hỗ trợ cả hai dạng tích lũy: cộng dồn theo chiều kênh (*temporal accumulation*) và cộng dồn theo chiều không gian (*spatial accumulation*). Ngoài ra, PU còn tích hợp cơ chế *Ping-pong FIFO* để tách biệt kết quả của hàng Ifmap trước với hàng hiện tại. Nếu kết quả đã hoàn thiện, dữ liệu sẽ được chuyển tới *Write Control* của PA; ngược lại, dữ liệu tiếp tục được phản hồi nội bộ để thực hiện các bước cộng dồn tiếp theo.

3.6.5.3 Kiến trúc khối Scale & ReLU

Khối *Scale & ReLU* được đặt sau datapath tính toán của PU, có nhiệm vụ xử lý kết quả *partial sum* trước khi ghi ra bộ nhớ *ofmap*. Các chức năng chính của khối bao gồm cộng bias, thực hiện phép scale, áp dụng hàm kích hoạt và giới hạn miền giá trị đầu ra bằng phép *clamp*. Kiến trúc tổng quát của khối được minh họa trong Hình 3.17.

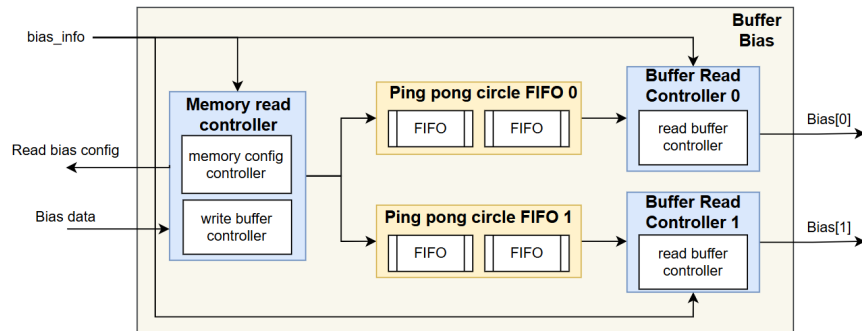


Hình 3.17: Kiến trúc pipeline của khối Scale & ReLU

Khối hiện được hiện thực dưới dạng pipeline gồm 11 stage. Ở các stage đầu, dữ liệu *partial sum* được cộng với *bias*, sau đó nhân với hệ số *scale multiplier*, cộng thêm *rounding value* và dịch phải bởi *scale shift*. Chuỗi thao tác này giúp đưa kết quả tích lũy có độ rộng lớn về miền giá trị phù hợp hơn với định dạng đầu ra. Sau bước scale, dữ liệu được đưa qua khối *clamp 32 bit* để giới hạn miền giá trị trung gian. Tiếp theo, khối hỗ trợ hai chế độ activation là ReLU và Leaky ReLU. Với ReLU, các giá trị âm được thay bằng *zero point y*. Với Leaky ReLU, nhánh dữ liệu âm được xử lý thêm bằng *alpha multiplier* và *alpha shift* trước khi đưa tới mạch chọn đầu ra. Nhờ đó, cùng một datapath có thể hỗ trợ linh hoạt hai dạng activation khác nhau. Ở cuối pipeline, kết quả được đưa qua khối *clamp 8 bit* để bảo hòa về miền biểu diễn 8 bit trước khi ghi ra *ofmap*. Đây là bước cần thiết để bảo đảm dữ liệu đầu ra phù hợp với định

dạng lượng tử hóa của accelerator [7], [22].

3.6.6 Khối Buffer Bias

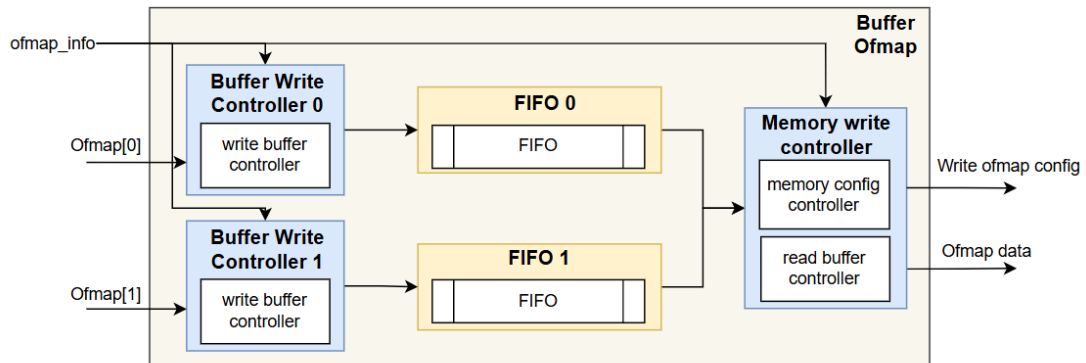


Hình 3.18: Kiến trúc khối Buffer Bias

Buffer Bias có nhiệm vụ lưu trữ và cung cấp các giá trị độ lệch để phục vụ bước hậu xử lý sau tích chập. Về cấu trúc, khối bao gồm bộ điều khiển đọc bộ nhớ và một đơn vị *Ping-pong Circle FIFO* dùng để lưu trữ toàn bộ các giá trị bias của cụm bộ lọc đang xử lý.

Khối *Write Mem Control* tiếp nhận thông tin cấu hình từ *bias_info* và thực hiện nạp dữ liệu *Bias data* từ bộ nhớ ngoài. Cơ chế vòng lặp của *Circle FIFO* cho phép truy xuất lại các giá trị bias khi cần lặp lại các bước tính toán trên cùng một tập bộ lọc mà không cần truy cập lại bộ nhớ ngoài.

3.6.7 Khối Buffer Ofmap



Hình 3.19: Kiến trúc khối Buffer Ofmap

Buffer Ofmap là chặng cuối trong chuỗi xử lý dữ liệu của accelerator. Khối này tiếp nhận các kết quả đầu ra từ *Computation*, thực hiện hậu xử lý và điều phối quá trình ghi trở lại bộ nhớ ngoài.

Kiến trúc của *Buffer Ofmap* gồm hai FIFO đầu vào, tương ứng trực tiếp với hai nhánh xử lý song song *PA 0* và *PA 1*. Khối *Write Control* thiết lập cấu hình ghi, quản lý địa chỉ bộ nhớ và bảo đảm dữ liệu đặc trưng đầu ra được lưu đúng vị trí.

Nhờ cấu trúc này, *Buffer Ofmap* không chỉ đóng vai trò là bộ đệm đầu ra, mà còn là nơi hoàn thiện đặc trưng đầu ra trước khi chuyển sang lớp kế tiếp hoặc ghi về bộ nhớ ngoài.

3.7 Mô hình hóa thời gian và bộ nhớ của bộ gia tốc

3.7.1 Giả thiết mô hình

Để phục vụ cho việc đánh giá hiệu năng ở mức kiến trúc, mô hình phân tích trong phạm vi này chỉ tập trung vào ba thành phần chính của quá trình thực thi tầng tích chập, bao gồm: nạp dữ liệu đầu vào (*ifmap load*), nạp trọng số (*weight load*), thực hiện tính toán nhân tích lũy (*compute*) và ghi dữ liệu đầu ra (*ofmap*

store). Các chi phí liên quan đến điều phối cấu hình, giải mã lệnh, cộng bias, scale và hàm kích hoạt ReLU không được đưa vào mô hình chu kỳ, nhằm làm nổi bật ảnh hưởng của tổ chức dữ liệu và cấu trúc xử lý song song đến độ trễ thực thi [3], [21].

Mô hình được xây dựng dựa trên chiến lược *Filter Decomposition*, trong đó phép tích chập hai chiều được phân rã thành các chuỗi tích chập một chiều theo từng hàng của bộ lọc. Với cách tổ chức này, một lượt xử lý (*pass*) tương ứng với một phân đoạn hàng của Ifmap trên một nhóm kênh đầu vào xác định. Trọng số của cụm bộ lọc đang xét được nạp ở hàng xử lý đầu tiên, sau đó được giữ lại trong hệ thống đệm vòng để tái sử dụng cho các hàng kế tiếp. Do đó, các hàng tiếp theo không còn chịu chi phí nạp lại trọng số từ bộ nhớ ngoài. Ngược lại, dữ liệu Ifmap vẫn được tính chi phí nạp cho từng hàng xử lý.

3.7.2 Ký hiệu và tham số mô hình

Xét một tầng tích chập được đặc trưng bởi bộ tham số

$$L = (W_i, H_i, C_i, w, h, W_o, H_o, C_o), \quad (3.4)$$

trong đó W_i, H_i, C_i lần lượt là chiều rộng, chiều cao và số kênh của đặc trưng đầu vào; w, h là kích thước kernel; còn W_o, H_o, C_o là chiều rộng, chiều cao và số kênh của đặc trưng đầu ra.

Kiến trúc phần cứng hỗ trợ xử lý song song tối đa $K = 4$ kênh Ifmap tại một thời điểm. Dựa trên đó, mô hình sử dụng ba tham số thiết kế:

- N_{ip} : số kênh Ifmap được xử lý trong một tile theo chiều sâu, với $1 \leq N_{ip} \leq K$;
- N_{fp} : số bộ lọc được xử lý song song trong một pass;
- N_{pass} : số pass được gộp trong một cụm kênh đầu ra.

Ký hiệu τ_{mem} được sử dụng để biểu diễn số chu kỳ cần thiết để truyền một phần tử dữ liệu giữa bộ nhớ và phần cứng tính toán.

3.7.3 Tổ chức dữ liệu theo tile và theo cụm bộ lọc

Số tile theo chiều sâu kênh đầu vào được xác định bởi:

$$T_{ifmap} = \left\lceil \frac{C_i}{N_{ip}} \right\rceil. \quad (3.5)$$

Mỗi cụm đầu ra xử lý đồng thời

$$G = N_{pass}N_{fp} \quad (3.6)$$

bộ lọc. Do đó, số cụm bộ lọc cần xử lý cho toàn bộ tầng là:

$$T_{of} = \left\lceil \frac{C_o}{G} \right\rceil. \quad (3.7)$$

Với cụm đầu ra thứ q , số bộ lọc thực tế trong cụm được viết:

$$F_q = \min(G, C_o - qG), \quad (3.8)$$

và số pass thực tế tương ứng là:

$$P_q = \left\lceil \frac{F_q}{N_{fp}} \right\rceil. \quad (3.9)$$

Tương tự, với tile Ifmap thứ t , số kênh đầu vào thực tế trong tile là:

$$I_t = \min(N_{ip}, C_i - tN_{ip}). \quad (3.10)$$

3.7.4 Mô hình thời gian của một pass

Do kiến trúc thực hiện tích chập theo từng hàng của bộ lọc, thời gian tính toán của một pass được mô hình hóa bởi:

$$C_{q,t,p}^{comp} = W_o \cdot w. \quad (3.11)$$

Trong đó, $C_{q,t,p}^{comp}$ là số chu kỳ cần thiết để thực hiện phép tích chập một chiều theo bề rộng đầu ra cho pass thứ p , tại tile Ifmap thứ t và cụm bộ lọc thứ q .

Mặt khác, để thực hiện đầy đủ phép tính của một pass, trọng số tương ứng với toàn bộ kernel $w \times h$ của nhóm bộ lọc đang xét phải được nạp vào hệ thống đệm. Khi đó, thời gian nạp trọng số của pass được viết:

$$C_{q,t,p}^{loadW} = F_{q,p} I_t w h \tau_{mem}, \quad (3.12)$$

trong đó $F_{q,p}$ là số bộ lọc thực tế trong pass thứ p của cụm q .

Ở hàng xử lý đầu tiên, thời gian của toàn bộ phần pass trong một tile được xác định bởi:

$$C_{q,t}^{pass,first} = \sum_{p=0}^{P_q-1} \max \left(C_{q,t,p}^{comp}, C_{q,t,p}^{loadW} \right). \quad (3.13)$$

Ở các hàng xử lý tiếp theo, trọng số đã được giữ lại trong bộ đệm vòng và được tái sử dụng, do đó thời gian pass ở trạng thái ổn định chỉ còn phụ thuộc vào tính toán:

$$C_{q,t}^{pass,steady} = \sum_{p=0}^{P_q-1} C_{q,t,p}^{comp}. \quad (3.14)$$

3.7.5 Mô hình nạp Ifmap và ghi Ofmap

Với mỗi tile theo chiều sâu, thời gian nạp dữ liệu đầu vào được mô hình hóa là:

$$C_t^{loadI} = I_t \cdot W_i \cdot \tau_{mem}. \quad (3.15)$$

Đối với dữ liệu đầu ra, chi phí ghi được phân bổ trung bình trên toàn bộ các tile của tầng:

$$C_{tile}^{store} = \left\lceil \frac{H_o \cdot C_o \cdot W_o \cdot \tau_{mem}}{T_{ifmap} \cdot W_i \cdot T_{of}} \right\rceil. \quad (3.16)$$

Cách mô hình hóa này phản ánh rằng quá trình ghi Ofmap được trải đều trong suốt thời gian xử lý lớp, thay vì quy về một thời điểm duy nhất ở cuối lớp.

3.7.6 Mô hình thời gian của một tile và của một lớp

Từ các thành phần trên, thời gian của tile ở hàng đầu tiên được xác định là:

$$C_{q,t}^{tile,first} = \max \left(C_{q,t}^{pass,first}, C_t^{loadI} + C_{tile}^{store} \right). \quad (3.17)$$

Đối với các hàng còn lại, thời gian của một tile được viết:

$$C_{q,t}^{tile,steady} = \max \left(C_{q,t}^{pass,steady}, C_t^{loadI} + C_{tile}^{store} \right). \quad (3.18)$$

Tổng thời gian theo chiều sâu kênh của cụm bộ lọc thứ q tại hàng đầu tiên là:

$$C_q^{depth,first} = \sum_{t=0}^{T_{ifmap}-1} C_{q,t}^{tile,first}, \quad (3.19)$$

trong khi thời gian tương ứng ở trạng thái ổn định là:

$$C_q^{depth,steady} = \sum_{t=0}^{T_{ifmap}-1} C_{q,t}^{tile,steady}. \quad (3.20)$$

Từ đó, tổng số chu kỳ xử lý của một tầng tích chập được mô hình hóa bởi:

$$C_{layer} = \sum_{q=0}^{T_{of}-1} \left[C_q^{depth,first} + (H_i - 1) C_q^{depth,steady} \right]. \quad (3.21)$$

Biểu thức trên cho thấy hàng đầu tiên của mỗi cụm bộ lọc phải chịu đầy đủ chi phí nạp trọng số, nạp Ifmap, tính toán và ghi kết quả; trong khi các hàng còn

lại chỉ còn chi phí nạp Ifmap, tính toán và ghi Ofmap nhờ cơ chế tái sử dụng trọng số trong bộ đệm vòng.

3.7.7 Mô hình bộ nhớ vật lý

Các biểu thức trong phần trên cho phép xác định lượng bộ nhớ được sử dụng tại từng lớp tích chập. Tuy nhiên, đối với bài toán thiết kế phần cứng, đại lượng cần quan tâm không chỉ là bộ nhớ sử dụng cục bộ của một lớp, mà là dung lượng SRAM vật lý thực sự phải được provision để accelerator có thể hỗ trợ toàn bộ tập mô hình khảo sát. Nói cách khác, nếu các lớp khác nhau cực đại hóa các khối nhớ khác nhau, thì kiến trúc vẫn phải được thiết kế theo trường hợp xấu nhất của từng khối nhớ riêng biệt.

Với một lớp tích chập ℓ , lượng bộ nhớ sử dụng được tách thành bốn thành phần:

$$M_{\text{partial}}^{(\ell)}, \quad M_{\text{ofmap}}^{(\ell)}, \quad M_{\text{ifmap}}^{(\ell)}, \quad M_{\text{weight}}^{(\ell)}. \quad (3.22)$$

Trong đó, bộ nhớ lưu kết quả tích lũy trung gian được xác định bởi

$$M_{\text{partial}}^{(\ell)} = 2 \cdot (4 \cdot N_{\text{pass}} \cdot W_o) \cdot N_{\text{fp}}^{\text{max}}, \quad (3.23)$$

với hệ số 2 phản ánh cơ chế ping-pong của bộ đệm trung gian, hệ số 4 là số byte cho một phần tử tích lũy, và $N_{\text{fp}}^{\text{max}}$ là số nhánh đệm trung gian cực đại của kiến trúc.

Bộ nhớ đệm đầu ra được mô hình hóa bởi

$$M_{\text{ofmap}}^{(\ell)} = N_{\text{fp}} \cdot N_{\text{pass}} \cdot W_o. \quad (3.24)$$

Bộ nhớ đệm đầu vào được xác định là

$$M_{\text{ifmap}}^{(\ell)} = 2 \cdot K_{\text{max}} \cdot W_i, \quad (3.25)$$

trong đó hệ số 2 tiếp tục phản ánh tổ chức ping-pong của bộ đệm Ifmap.

Cuối cùng, bộ nhớ lưu trữ trọng số cho cụm bộ lọc đang xử lý được viết dưới dạng

$$M_{\text{weight}}^{(\ell)} = N_{fp} \cdot N_{pass} \cdot w \cdot h \cdot C_i. \quad (3.26)$$

Tổng bộ nhớ đang được sử dụng tại lớp ℓ là

$$M_{\text{layer}}^{(\ell)} = M_{\text{partial}}^{(\ell)} + M_{\text{ofmap}}^{(\ell)} + M_{\text{ifmap}}^{(\ell)} + M_{\text{weight}}^{(\ell)}. \quad (3.27)$$

Tuy nhiên, $M_{\text{layer}}^{(\ell)}$ chỉ phản ánh lượng bộ nhớ cần dùng tại một thời điểm của một lớp cụ thể. Để xác định dung lượng SRAM vật lý cần hiện thực cho toàn bộ accelerator, cần lấy cực đại của từng khối nhớ trên toàn bộ tập lớp kiểm thử $\mathcal{L}_{\text{test}}$. Khi đó, dung lượng vật lý cần provision cho từng loại bộ nhớ được xác định bởi

$$M_{\text{partial}}^{\text{phys}} = \max_{\ell \in \mathcal{L}_{\text{test}}} M_{\text{partial}}^{(\ell)}, \quad (3.28)$$

$$M_{\text{ofmap}}^{\text{phys}} = \max_{\ell \in \mathcal{L}_{\text{test}}} M_{\text{ofmap}}^{(\ell)}, \quad (3.29)$$

$$M_{\text{ifmap}}^{\text{phys}} = \max_{\ell \in \mathcal{L}_{\text{test}}} M_{\text{ifmap}}^{(\ell)}, \quad (3.30)$$

$$M_{\text{weight}}^{\text{phys}} = \max_{\ell \in \mathcal{L}_{\text{test}}} M_{\text{weight}}^{(\ell)}. \quad (3.31)$$

Từ đó, tổng dung lượng SRAM vật lý tối thiểu cần provision cho kiến trúc accelerator được xác định là

$$M_{\text{phys}} = M_{\text{partial}}^{\text{phys}} + M_{\text{ofmap}}^{\text{phys}} + M_{\text{ifmap}}^{\text{phys}} + M_{\text{weight}}^{\text{phys}}. \quad (3.32)$$

Biểu thức trên cho phép chuyển bài toán từ mức “bộ nhớ sử dụng của một lớp” sang mức “bộ nhớ vật lý phải hiện thực trên chip”, phù hợp hơn với mục tiêu thiết kế phần cứng.

3.7.8 Hàm mục tiêu tối ưu

Sau khi đã xây dựng mô hình thời gian và mô hình bộ nhớ vật lý, bài toán tối ưu được phát biểu theo hướng đồng thời xem xét cả hiệu năng thực thi và chi phí bộ nhớ phần cứng. Gọi C_ℓ là số chu kỳ thực thi của lớp tích chập thứ ℓ , khi đó tổng số chu kỳ của toàn bộ tập lớp được khảo sát được xác định bởi

$$C_{\text{net}} = \sum_{\ell \in \mathcal{L}_{\text{test}}} C_\ell. \quad (3.33)$$

Trên cơ sở đó, hàm mục tiêu tối ưu được lựa chọn là

$$ADP_{\text{phys}} = C_{\text{net}} \cdot M_{\text{phys}}. \quad (3.34)$$

Ở đây, C_{net} phản ánh độ trễ thực thi tổng thể của accelerator trên toàn bộ tập mô hình, trong khi M_{phys} phản ánh chi phí SRAM vật lý cần provision để hiện thực kiến trúc. Do đó, hàm mục tiêu ADP_{phys} cho phép lựa chọn cấu hình phần cứng cân bằng giữa hai yêu cầu: đạt hiệu năng thực thi cao và giảm thiểu chi phí tài nguyên bộ nhớ.

3.7.9 Đánh giá cấu hình bộ nhớ vật lý

Để xác định dung lượng SRAM vật lý cần provision cho từng khối nhớ trong kiến trúc accelerator, quá trình mô hình hóa được thực hiện trên hai mô hình CNN có đặc trưng kiến trúc khác nhau, bao gồm *VGG16* với đầu vào $224 \times 224 \times 3$ và *ALL CNN C* với đầu vào $32 \times 32 \times 3$ [5], [6]. Việc khảo sát đồng thời hai mô hình này nhằm bảo đảm rằng kích thước bộ nhớ được lựa chọn không chỉ phù hợp với một cấu trúc mạng cụ thể, mà còn phản ánh được nhiều dạng yêu cầu bộ nhớ khác nhau trong quá trình thực thi các lớp tích chập.

VGG16 là mô hình có độ sâu lớn, gồm nhiều lớp tích chập 3×3 được xếp liên tiếp [5]. Đặc trưng của mô hình này là kích thước không gian của feature map giảm dần qua từng nhóm lớp, trong khi số lượng channel tăng lên đáng kể

ở các lớp sâu. Do đó, VGG16 tạo ra các trường hợp kiểm tra quan trọng đối với cả bộ nhớ lưu partial sum và bộ nhớ trọng số, đặc biệt ở các lớp có số kênh đầu vào và số filter đầu ra lớn. Ngược lại, ALL CNN C là mô hình gọn hơn, được thiết kế cho ảnh đầu vào kích thước nhỏ, nhưng vẫn bao gồm nhiều lớp tích chập liên tiếp với sự thay đổi về kích thước không gian, số channel và cả kernel 1×1 ở các lớp cuối [6]. Vì vậy, ALL CNN C giúp kiểm tra khả năng thích ứng của kiến trúc đối với các lớp có kích thước feature map nhỏ hơn, số filter khác nhau và đặc biệt là các lớp pointwise convolution.

3.7.10 Xác định kích thước vật lý

Để xác định kích thước vật lý cần provision cho từng khối nhớ của accelerator, quá trình mô hình hóa được thực hiện trên hai mô hình CNN khác nhau, gồm VGG16, ALL CNN C với đầu vào lần lượt là $224 \times 224 \times 3$ và $32 \times 32 \times 3$ [5], [6].

Một mô hình đơn lẻ khó có thể đồng thời tạo ra trường hợp cực đại cho tất cả các khối nhớ trong accelerator. Chẳng hạn, một lớp có kích thước không gian đầu ra lớn có thể làm tăng yêu cầu đối với bộ nhớ lưu partial sum hoặc ofmap, trong khi một lớp khác với số kênh đầu vào và số filter đầu ra lớn lại có thể làm tăng yêu cầu đối với bộ nhớ trọng số. Vì vậy, việc mô hình hóa trên cả VGG16 và ALL CNN C cho phép đánh giá toàn diện hơn dung lượng vật lý cần provision cho từng khối SRAM, thay vì chỉ dựa trên một mạng CNN đơn lẻ.

Kết quả tìm kiếm không gian thiết kế cho thấy cấu hình tối ưu toàn cục đạt được tại:

$$K_{\max}^* = 4. \quad (3.35)$$

Dung lượng SRAM vật lý cần provision cho từng khối nhớ của accelerator được xác định bằng cách lấy giá trị cực đại trên toàn bộ các lớp của cả hai mô hình. Kết quả cụ thể như sau:

$$M_{\text{partial}}^{\text{phys}} = 43,008 \text{ bytes}, \quad (3.36)$$

$$M_{\text{ofmap}}^{\text{phys}} = 1,792 \text{ bytes}, \quad (3.37)$$

$$M_{\text{ifmap}}^{\text{phys}} = 1,792 \text{ bytes}, \quad (3.38)$$

$$M_{\text{weight}}^{\text{phys}} = 37,600 \text{ bytes}. \quad (3.39)$$

Do đó, tổng dung lượng SRAM vật lý tối thiểu cần provision cho toàn bộ kiến trúc accelerator là:

$$M_{\text{phys}} = M_{\text{partial}}^{\text{phys}} + M_{\text{ofmap}}^{\text{phys}} + M_{\text{ifmap}}^{\text{phys}} + M_{\text{weight}}^{\text{phys}} = 84,192 \text{ bytes}. \quad (3.40)$$

Có thể thấy hai thành phần chi phối chính trong chi phí bộ nhớ của kiến trúc là *Partial SRAM* và *Weight SRAM*. Trong đó, *Partial SRAM* đạt giá trị lớn nhất ở các lớp có kích thước feature map đầu ra lớn, điển hình là các lớp đầu của VGG16. Ngược lại, *Weight SRAM* tăng mạnh ở các lớp sâu, nơi số lượng channel đầu vào và số filter đầu ra lớn hơn. Hai khối *Ofmap SRAM* và *Ifmap SRAM* có dung lượng nhỏ hơn đáng kể, do kiến trúc chỉ lưu trữ dữ liệu theo từng tile thay vì lưu toàn bộ feature map tại một thời điểm.

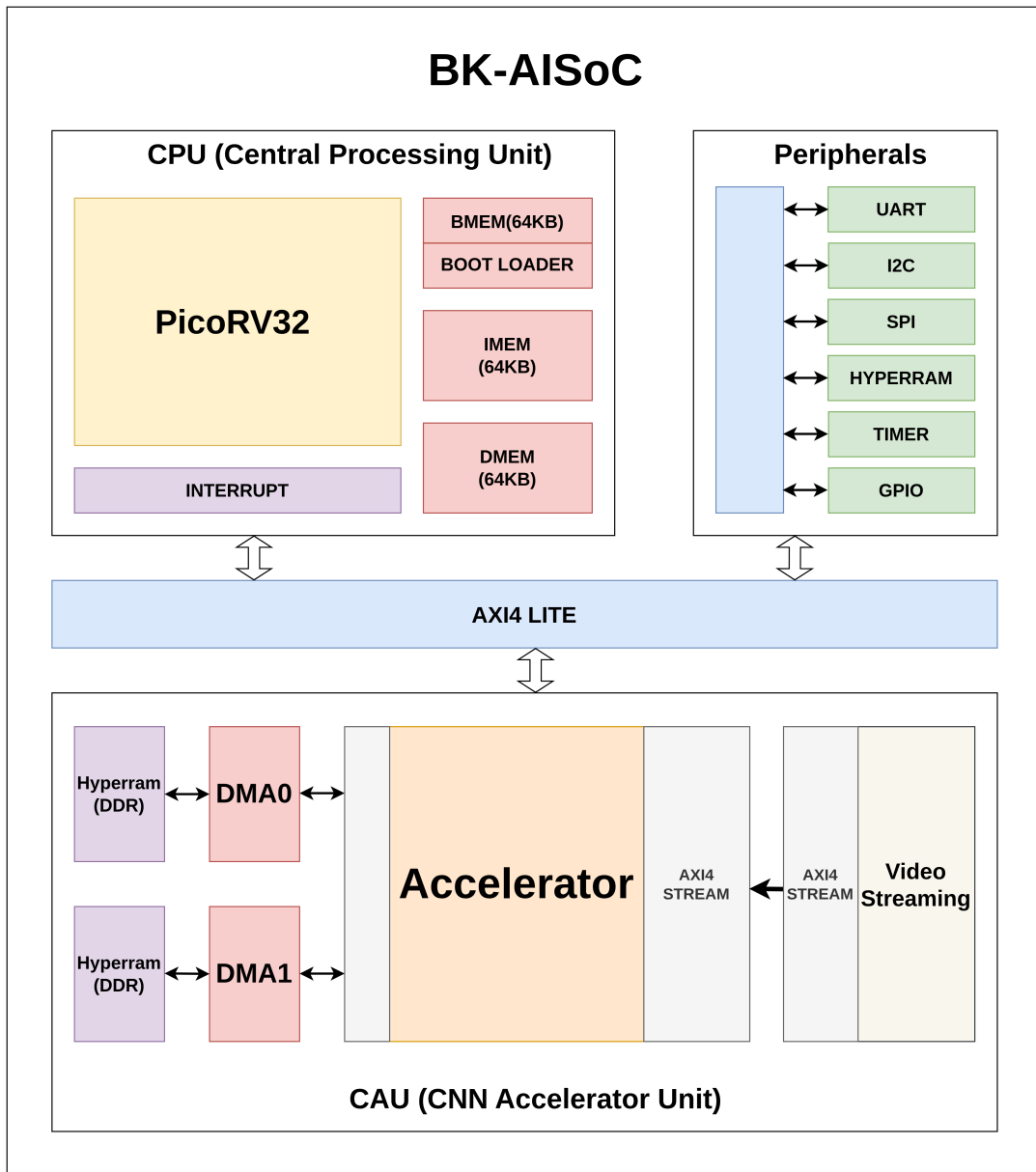
Kết quả trên cho thấy quá trình khảo sát trên hai mô hình CNN có đặc trưng khác nhau đã cung cấp cơ sở định lượng để lựa chọn kích thước vật lý cho từng khối SRAM trong accelerator. Cấu hình $K_{\text{max}}^* = 4$ cho phép đạt được sự cân bằng giữa tài nguyên bộ nhớ, mức song song tính toán và số chu kỳ thực thi. Đây là cơ sở để chuyển từ mô hình đánh giá kiến trúc sang bước thiết kế phần cứng chi tiết, trong đó các khối SRAM được provision theo giá trị cực đại đã xác định từ quá trình mô hình hóa.

Chương 4

Hiện thực kiến trúc

4.1 Tổng quan kiến trúc BK-AISoC

BK-AISoC được thiết kế theo hướng heterogeneous SoC, kết hợp giữa một soft RISC-V core cho các tác vụ điều khiển và các hardware block chuyên dụng cho AI data path [2], [8]. Cách tổ chức này phù hợp với hệ thống nhúng trên FPGA: CPU giữ vai trò điều phối tổng thể, trong khi các khối như CNN accelerator, DMAC và Video Streaming đảm nhiệm những data path có yêu cầu bandwidth hoặc timing cao [3], [9].



Hình 4.1: Sơ đồ khối kiến trúc tổng quan của hệ thống BK-AISoC

Như minh họa trong Hình 4.1, kiến trúc phần cứng của SoC gồm năm thành phần chính: PicoRV32 processor core, on-chip memory, bus interconnect, các peripheral controller và các high-speed data path. Các thành phần này được tích hợp trong cùng một 32-bit address space, giúp CPU truy cập các control/status register của phần cứng thông qua cơ chế memory-mapped I/O [10], [11].

So với kiến trúc vi điều khiển thuần túy, điểm khác biệt của BK-AISoC nằm ở việc tách rõ control path và data path. Control path dùng cho các giao dịch cấu hình register, đọc trạng thái và xử lý Interrupt. Data path được tổ chức riêng

cho các luồng có dung lượng lớn như image data, feature map, weights và kết quả trung gian của accelerator. Cách phân tách này giúp giảm tải cho CPU và làm cho kiến trúc có khả năng mở rộng khi tích hợp thêm các accelerator trong tương lai.

4.2 Processor core và on-chip memory

Khối xử lý trung tâm của hệ thống sử dụng **PicoRV32**, một soft RISC-V core 32-bit tối ưu tài nguyên cho FPGA [10]. Trong SoC này, PicoRV32 đóng vai trò primary master trên control bus, phát sinh các giao dịch read/write đến memory và peripheral controller. Việc chọn một processor core nhỏ gọn giúp tiết kiệm logic resource, đồng thời vẫn đủ khả năng thực hiện các tác vụ điều phối hệ thống.

On-chip memory được chia thành các vùng chức năng riêng, bao gồm boot memory, instruction memory và data memory. Cách tách memory region giúp đơn giản hóa address mapping và giảm xung đột giữa instruction access và data access. Ở mức phần cứng, mỗi memory region được gắn với một address range xác định, từ đó Address decoder có thể dẫn giao dịch của CPU tới đúng target block.

4.3 Bus và address space

Interconnect của BK-AISoC được tổ chức theo hướng phân tầng. Các khối điều khiển có bandwidth thấp được kết nối qua memory-mapped control bus, trong khi các luồng dữ liệu lớn được truyền qua streaming interface hoặc data path chuyên biệt. Ý tưởng này được kế thừa từ thiết kế cũ, nhưng được điều chỉnh lại cho cấu hình phần cứng hiện tại của BK-AISoC.

Trong tầng control bus, CPU phát address đến interconnect, sau đó Address decoder xác định peripheral hoặc memory slave tương ứng. Cách tổ chức này có ba lợi ích chính. Thứ nhất, mỗi hardware block có một address range riêng nên

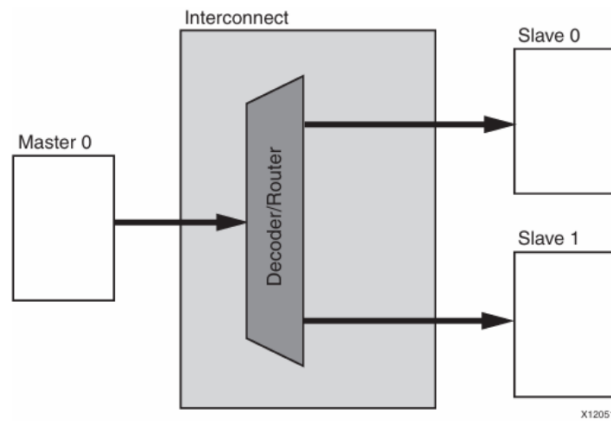
tránh được xung đột địa chỉ. Thứ hai, register interface của các peripheral được chuẩn hóa quanh các control/status register. Thứ ba, kiến trúc có thể mở rộng bằng cách bổ sung slave mới vào address map mà không làm thay đổi mô hình truy cập của CPU.

32-bit address space của CPU cho phép hệ thống quy hoạch on-chip memory, peripheral region và các vùng mở rộng khác trong cùng một flat address space. Trong cấu hình hiện tại, các peripheral chính được đặt trong một address region riêng để thuận tiện cho address decode và bus integration. Ở Chương 3, nội dung tập trung vào nguyên lý phần cứng của cơ chế memory-mapped I/O và cách các khối được tích hợp vào interconnect.

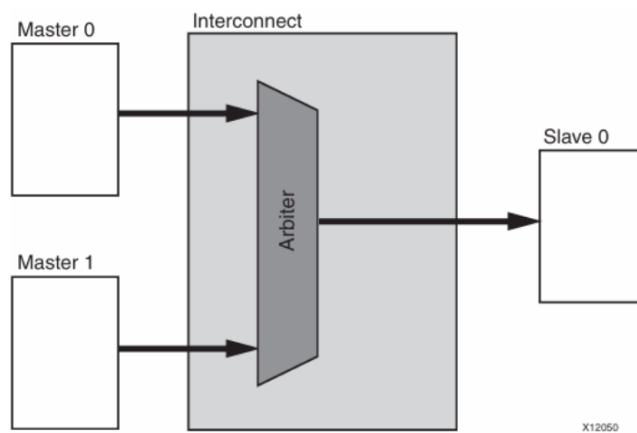
4.4 AXI4-Lite Interconnect

Trong BK-AISoC, **AXI4-Lite Interconnect** đóng vai trò là hạ tầng kết nối chính giữa PicoRV32 và các khối memory-mapped. Giao thức AXI4-Lite phù hợp với control path vì mỗi transaction chỉ cần truyền một data word hoặc đọc một status word, không yêu cầu burst dài như các streaming data path [11], [23]. Việc chuẩn hóa giao tiếp quanh AXI4-Lite giúp các peripheral có thể được tích hợp theo cùng một mẫu: Address decoder, AXI4-Lite slave interface, control register và status register.

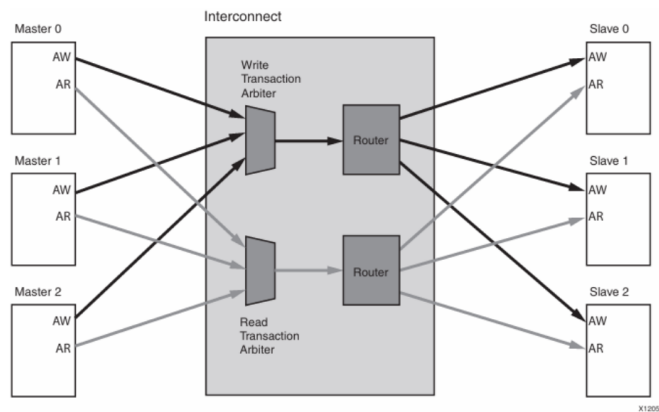
Thiết kế interconnect được tổ chức theo ba dạng kết nối cơ bản như Hình 4.2. Mô hình $1 \times N$ dùng khi một master truy cập nhiều slave, phù hợp với trường hợp CPU điều khiển toàn bộ peripheral. Mô hình $N \times 1$ dùng khi nhiều master cần truy cập một tài nguyên dùng chung, khi đó bộ Arbiter quyết định master nào được cấp quyền tại một thời điểm. Mô hình $N \times M$ được xây dựng bằng cách kết hợp hai dạng trên, tạo thành cấu trúc tổng quát hơn cho hệ thống có nhiều master và nhiều slave.



(a) $1 \times N$



(b) $N \times 1$

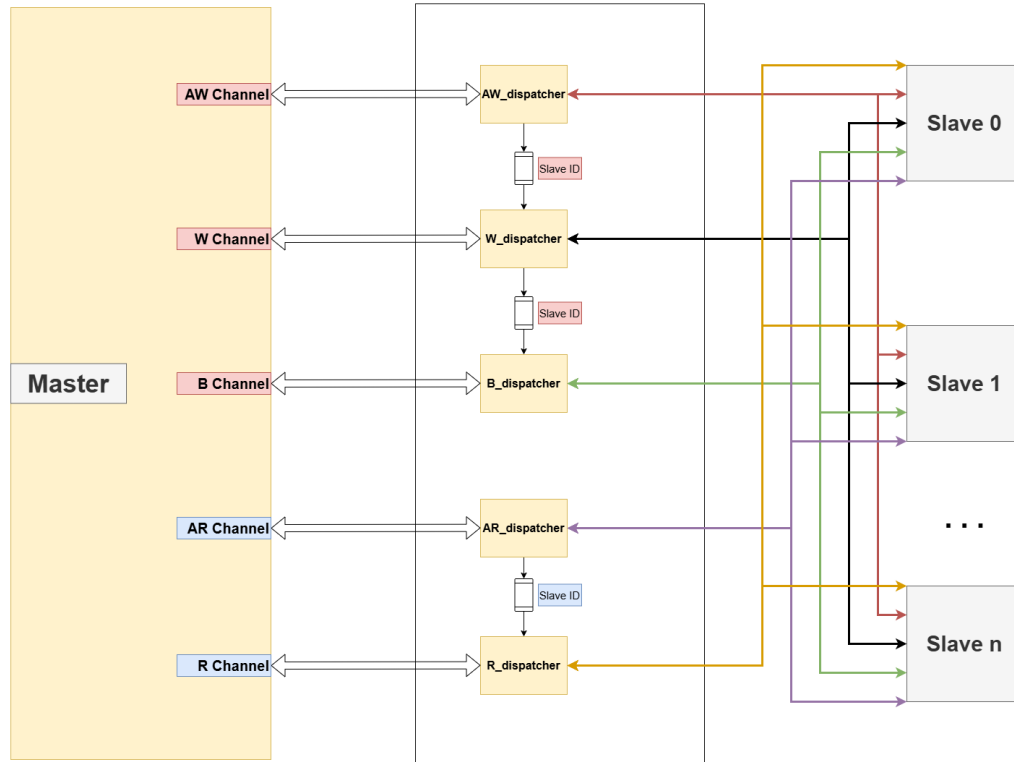


(c) $N \times M$

Hình 4.2: Các mô hình kết nối cơ bản của AXI4-Lite Interconnect

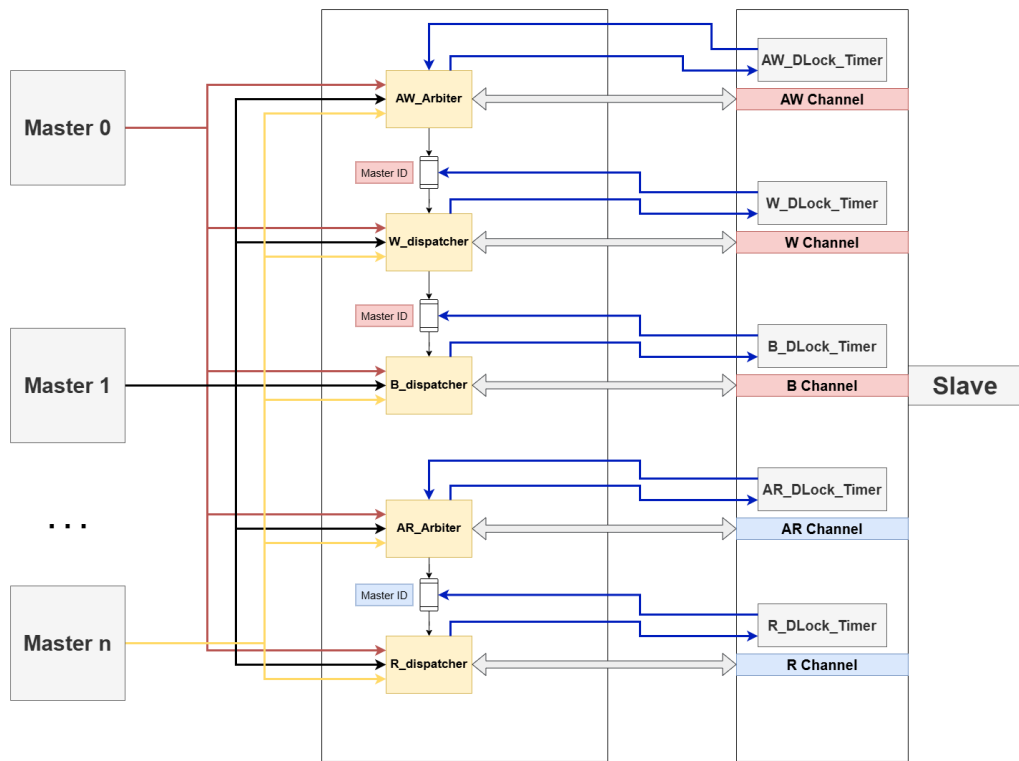
Với mô hình $1 \times N$, một master duy nhất phát sinh transaction đến nhiều slave khác nhau. Cấu trúc chi tiết của mô hình này được thể hiện trong Hình 4.3. Khi interconnect nhận address từ master ở write address channel hoặc read address channel, sau đó so sánh với address map để chọn slave tương ứng. Ở chiều ghi,

thông tin chọn slave từ write address channel được giữ lại để định tuyến đúng write data và write response. Ở chiều đọc, read data trả về từ slave được đưa ngược về master thông qua read channel.



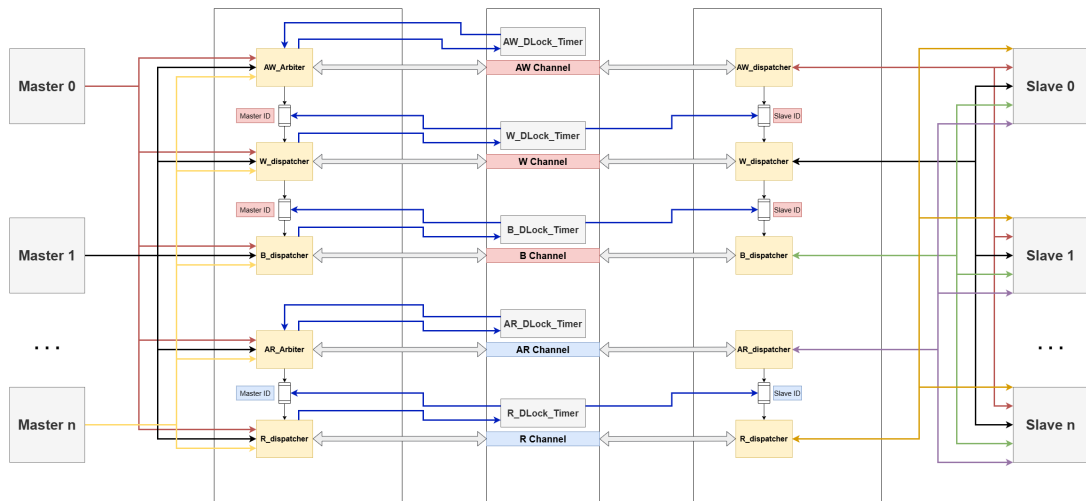
Hình 4.3: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $1 \times N$

Mô hình $N \times 1$ được dùng khi nhiều master có khả năng truy cập cùng một slave. Trong trường hợp này, vấn đề chính không còn là address decode mà là arbitration. Như Hình 4.4, các request từ nhiều master được đưa vào khối Arbiter. Tại một thời điểm, Arbiter chỉ cấp grant cho một master, sau đó các Multiplexer sẽ nối các kênh AXI4-Lite của master được chọn tới slave dùng chung. Cách làm này bảo đảm các transaction không bị chồng lấn và response được trả về đúng master phát sinh request.



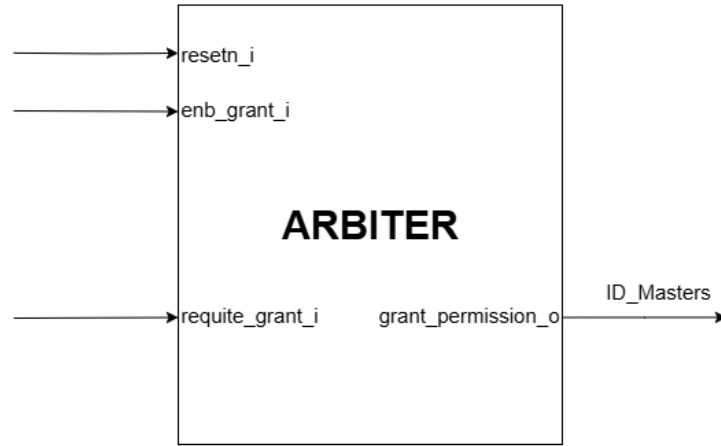
Hình 4.4: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $N \times 1$

Khi hệ thống có nhiều master và nhiều slave, interconnect được tổ chức theo mô hình $N \times M$. Cấu trúc này kết hợp hai bước: arbitration giữa các master trước, sau đó address decode để chọn slave. Hình 4.5 cho thấy tầng $N \times 1$ chịu trách nhiệm chọn master hợp lệ, còn tầng $1 \times N$ chịu trách nhiệm định tuyến transaction đến target slave. Cách ghép tầng này giúp kiến trúc bus có thể mở rộng theo số lượng master và slave mà không cần thay đổi nguyên lý giao tiếp của từng peripheral.

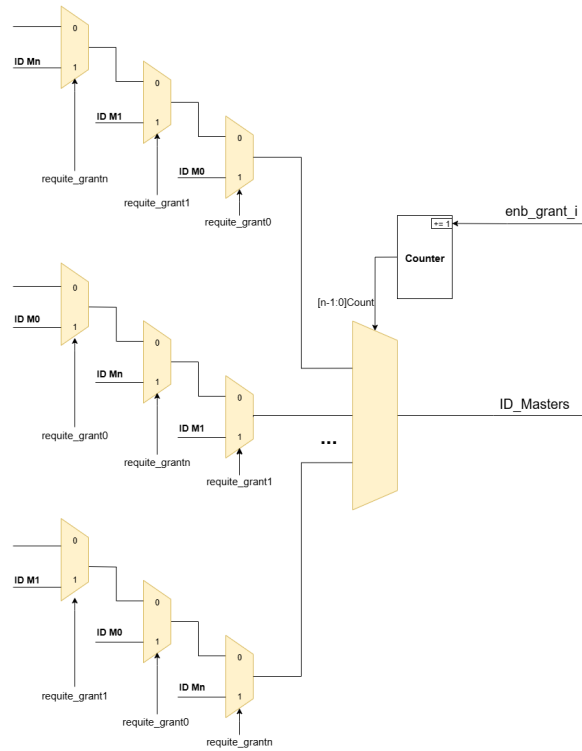


Hình 4.5: Cấu trúc chi tiết của AXI4-Lite Interconnect mô hình $N \times M$

Trong arbitration stage, thuật toán Round-Robin được sử dụng để tránh việc một master chiếm bus quá lâu. Hình 4.6 minh họa cấu trúc và control flow của Arbiter. Priority counter luân phiên qua các master đang phát request; khi một request hợp lệ được chọn, tín hiệu grant được phát ra để khóa đường đi của transaction hiện tại. Sau khi transaction kết thúc, priority được cập nhật cho lượt arbitration tiếp theo. Cơ chế này phù hợp với SoC có nhiều nguồn phát sinh transaction, vì nó giữ được fairness giữa CPU, DMA hoặc các khối xử lý có khả năng truy cập bus.



(a) Sơ đồ khối Arbiter



(b) Luồng điều khiển Round-Robin

Hình 4.6: Cơ chế Arbiter Round-Robin trong AXI4-Lite Interconnect

Nhờ các mô hình kết nối trên, AXI4-Lite Interconnect vừa đáp ứng được cấu hình hiện tại của BK-AISoC, vừa giữ khả năng mở rộng khi cần bổ sung thêm master hoặc peripheral mới. Đối với các peripheral điều khiển như UART, SPI, I2C, Timer và GPIO, mô hình $1 \times N$ là cấu trúc sử dụng chính; các mô hình có nhiều master là cơ sở cho những cấu hình mở rộng có DMA hoặc accelerator

phát sinh truy cập độc lập.

4.5 Peripheral controller

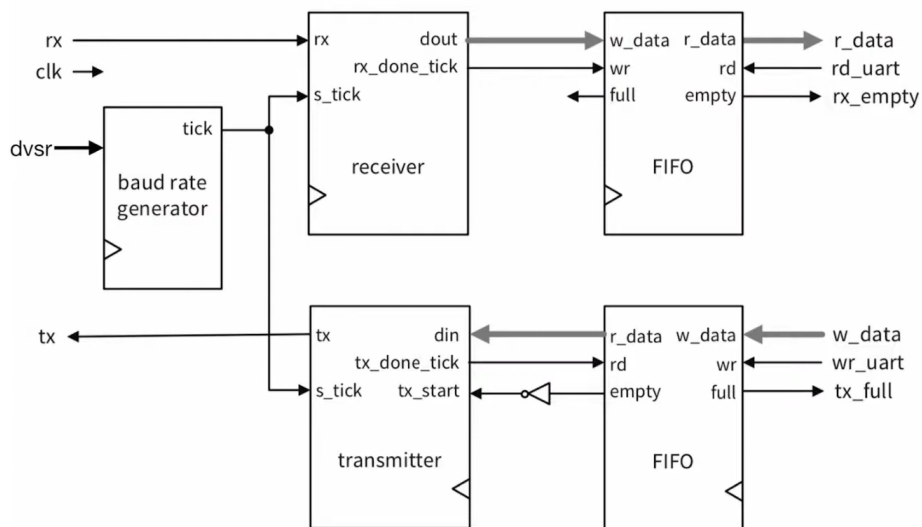
Cấu hình phần cứng hiện tại của SoC tích hợp một instance cho từng serial interface chính: **UART**, **SPI** và **I2C**. Bên cạnh đó, hệ thống còn có GPIO, Timer, Timer Interrupt, Interrupt controller, Video Streaming, HyperRAM và CNN accelerator. Các khối này được đóng gói thành các peripheral controller có control register, status register và các hardware signal tương ứng.

Ở mức hardware design, UART, SPI và I2C đại diện cho ba kiểu serial interface khác nhau: asynchronous, synchronous và two-wire. GPIO cung cấp các digital I/O signal tổng quát. Timer là cycle counter độc lập, còn Timer Interrupt là nguồn Interrupt định thời riêng. Interrupt controller tập trung các Interrupt source từ các hardware block để đưa về CPU theo một interface thống nhất.

Việc chỉ hiện thực một instance cho mỗi chuẩn giao tiếp trong cấu hình hiện tại giúp giảm tài nguyên phần cứng và đơn giản hóa quá trình tích hợp. Tuy nhiên, kiến trúc bus vẫn cho phép nhân bản các controller cùng loại nếu các phiên bản phần cứng sau này cần bổ sung thêm cổng giao tiếp.

4.5.1 UART

UART là peripheral controller cho asynchronous serial interface [13]. Về mặt phần cứng, khối UART gồm Baud generator, transmitter, receiver và các data FIFO. Baud generator chia system clock thành sampling clock phù hợp với baud rate đã cấu hình. Receiver sử dụng cơ chế oversampling để lấy mẫu bit ở vùng ổn định của tín hiệu, trong khi transmitter chuyển dữ liệu song song thành serial frame gồm start bit, data bits và stop bit.



Hình 4.7: Hardware architecture của khối UART

Khi tích hợp vào SoC, UART được bọc bởi AXI4-Lite slave interface để ánh xạ các internal resource thành memory-mapped register. Nhờ lớp interface này, serial TX/RX logic được tách khỏi system bus logic. Register map của khối UART được minh họa trong Hình 4.8, gồm data register, control register và status register.

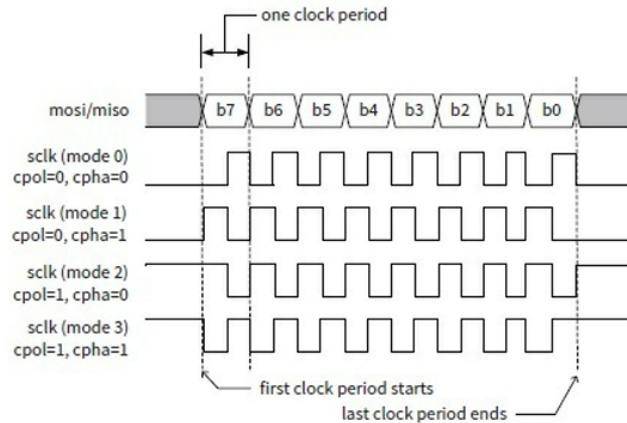
0		tx_full 9	rx_empty 8	r_data 7 ... 0	r
1		10	dvsr ...	0	w
2			w_data 7 ... 0		w
3		dummy data write			w

Hình 4.8: Register map của khối UART qua AXI4-Lite interface

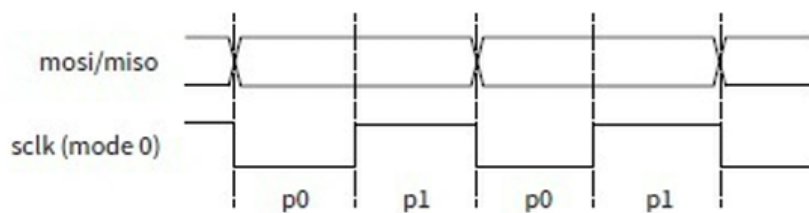
4.5.2 SPI

SPI là controller cho synchronous serial interface [14]. Khác với UART, SPI sử dụng clock do master phát ra, nhờ đó quá trình truyền và nhận dữ liệu được đồng bộ theo từng clock edge. Phần cứng SPI gồm clock divider, shift register,

bit counter, slave-select logic và control FSM. Hai tham số CPOL và CPHA quyết định clock polarity và clock phase, cho phép controller tương thích với nhiều SPI mode khác nhau.



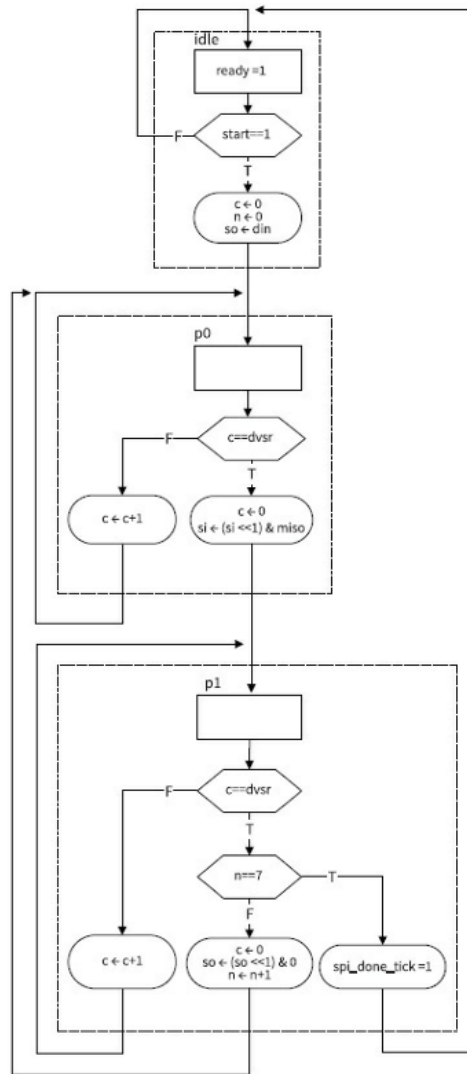
(a) Các mode SPI



(b) Dịch dữ liệu theo xung SPI

Hình 4.9: SPI timing và các mode truyền dữ liệu

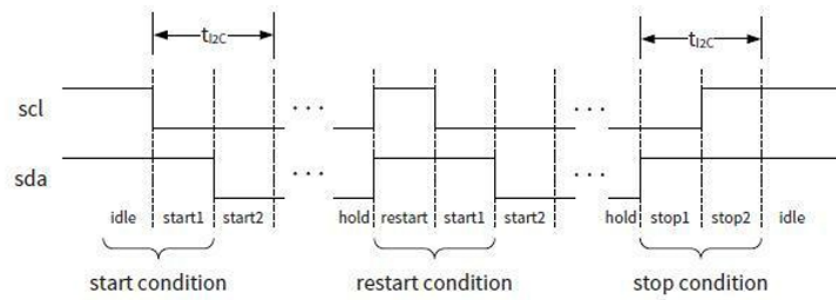
Control FSM của SPI điều phối các pha chính của một transaction: Idle, kích hoạt slave, shift data và kết thúc transaction. Trong pha truyền, shift register đẩy dữ liệu ra MOSI đồng thời lấy mẫu dữ liệu từ MISO. Sau khi đủ số bit của một byte hoặc một word được truyền, done status được cập nhật để lớp điều khiển phía trên có thể biết transaction đã kết thúc.



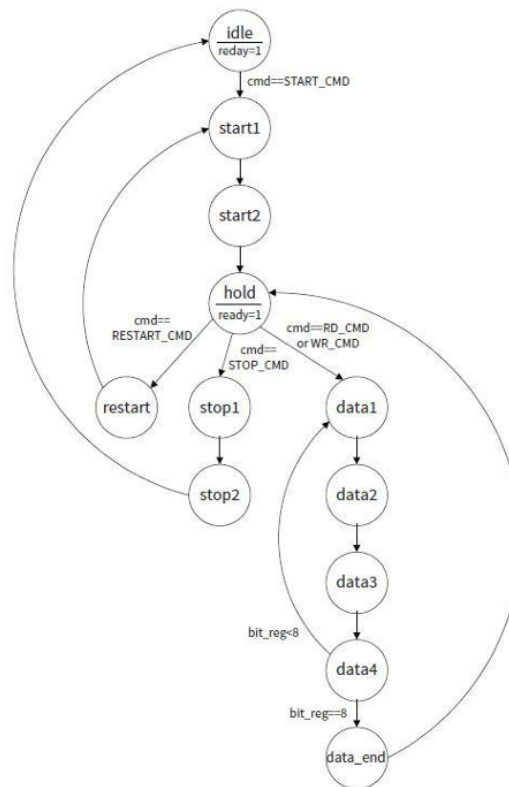
Hình 4.10: Control FSM của khối SPI

4.5.3 I2C

I2C là two-wire controller, gồm clock line SCL và data line SDA [15]. Do đặc trưng open-drain của giao thức I2C, phần cứng controller không chỉ phát mức logic mà còn phải điều khiển trạng thái high-impedance để nhường bus cho thiết bị phía ngoài phản hồi ACK hoặc truyền dữ liệu. Vì vậy, I2C được thiết kế quanh control FSM để điều khiển chặt chẽ các điều kiện Start, Stop, Write, Read và ACK/NACK.



(a) Các pha trong chu kỳ I2C



(b) FSM điều khiển I2C

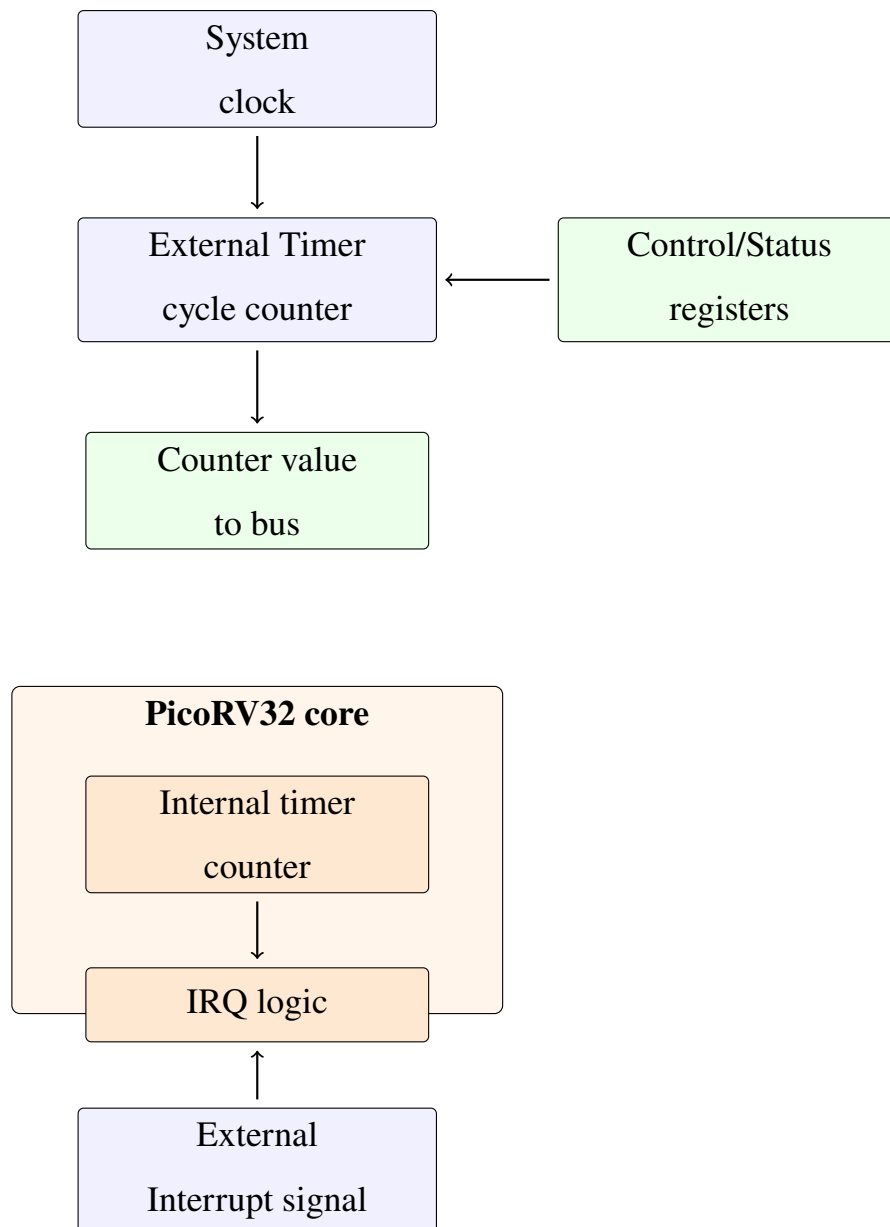
Hình 4.11: Timing và control FSM của khối I2C

Để tạo xung SCL từ system clock, I2C sử dụng clock divider và chia nhỏ một transfer cycle thành các timing phase. Cách chia này giúp phần cứng kiểm soát setup time và hold time của SDA quanh cạnh SCL, đồng thời bảo đảm điều kiện Start và Stop được tạo đúng trình tự. Khi tích hợp vào bus, các command và dữ liệu truyền nhận của I2C được ánh xạ thành memory-mapped register.

4.5.4 Timer, Timer Interrupt và GPIO

Timer là cycle counter độc lập được đặt bên ngoài processor core. Khối này nhận system clock, tăng counter value theo từng chu kỳ hoặc theo prescaler và ánh xạ giá trị đếm thành memory-mapped register để firmware đọc khi cần đo chu kỳ hoặc profiling. Do đó, Timer trong thiết kế này được dùng chủ yếu cho mục đích quan sát thời gian thực thi của phần cứng và phần mềm.

Timer Interrupt không phải là một peripheral timer riêng đặt trong peripheral subsystem của SoC. Chức năng này thuộc cơ chế Interrupt có sẵn của PicoRV32, trong đó core có logic/bộ đếm nội bộ để tạo sự kiện Timer Interrupt. Các logic ngoại vi bên ngoài PicoRV32 chỉ đưa tín hiệu **External Interrupt** vào các chân IRQ của core khi cần báo sự kiện. Firmware sau đó xử lý các sự kiện này trong Interrupt handler hoặc ghi nhận flag để xử lý tiếp trong main loop. Cách tổ chức này giúp hệ thống tận dụng sẵn cơ chế IRQ của PicoRV32, đồng thời vẫn giữ Timer ngoại vi như một bộ đếm độc lập phục vụ đo lường.



Hình 4.12: Phân tách Timer ngoại vi và Timer Interrupt trong PicoRV32

Hình 4.12 thể hiện sự phân tách giữa Timer ngoại vi và cơ chế Timer Interrupt của PicoRV32. Phần trên là Timer memory-mapped, nhận system clock và đưa counter value ra bus để CPU đọc như một register thông thường. Phần dưới là cơ chế Interrupt bên trong PicoRV32: internal timer counter có thể tạo sự kiện cho IRQ logic, trong khi các tín hiệu Interrupt từ ngoại vi chỉ được nối vào IRQ input để core nhận biết và chuyển quyền xử lý cho firmware.

Khối **GPIO** cung cấp các digital input/output signal tổng quát. Mỗi GPIO pin có thể được điều khiển thông qua direction register, output data register

và input status register. Về mặt tích hợp, GPIO có cấu trúc đơn giản hơn các serial controller, nhưng vẫn tuân theo cùng mô hình AXI4-Lite slave và memory-mapped register như các peripheral còn lại.

4.6 External memory và data path

Hệ thống sử dụng external memory interface **HyperRAM**. Vùng nhớ này được xem như memory resource độc lập, cho phép các hardware block có bandwidth lớn truy cập dữ liệu ngoài SoC thay vì phụ thuộc vào on-chip memory dung lượng nhỏ. Cách bố trí này đặc biệt quan trọng đối với các AI processing block, nơi input data, model parameters và intermediate result có thể được truy cập với tần suất cao.

Bên cạnh control bus, các khối DMAC, Video Streaming và CNN accelerator tạo thành high-speed data path của SoC. Các data path này không chỉ phục vụ việc truyền dữ liệu giữa CPU và peripheral, mà còn cho phép các hardware block trao đổi trực tiếp với nhau hoặc với external memory. Đây là cơ sở để hệ thống có thể xử lý image data và neural network data mà không phụ thuộc hoàn toàn vào việc CPU thực hiện từng load/store transaction [3], [12].

4.7 CNN accelerator trong hệ thống

Khối **CNN accelerator** là phần cứng tính toán chuyên dụng trong BK-AISoC. Khối này được thiết kế để thực hiện các phép toán có tính lặp cao trong convolutional neural network, trong đó các processing element có thể được tổ chức song song để tăng throughput [3], [4]. Ở cấp hệ thống, accelerator được kết nối với CPU thông qua control register và kết nối với external memory thông qua các data path riêng.

Việc đặt accelerator như một hardware block độc lập giúp tách control plane khỏi compute plane. CPU chỉ cần cấu hình chế độ hoạt động và theo dõi status, trong khi dữ liệu lớn được vận chuyển qua DMAC và các external memory

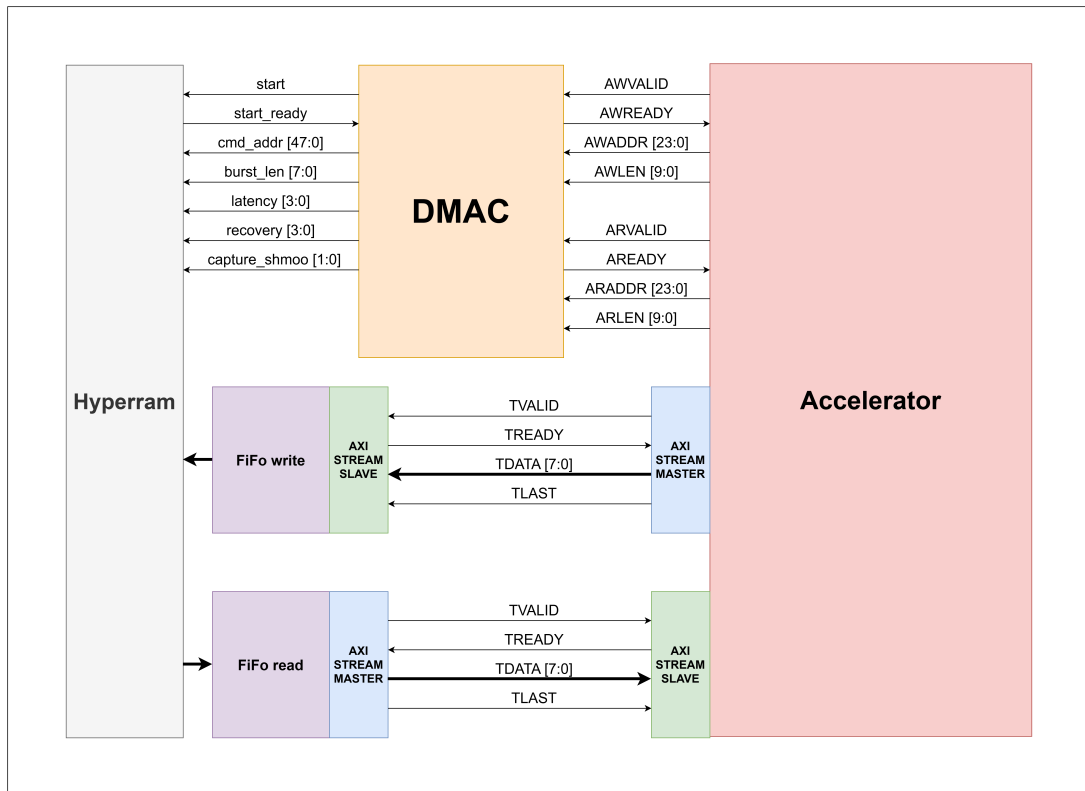
channel. Nhờ đó, kiến trúc tổng thể giữ được vai trò rõ ràng giữa CPU, memory và accelerator.

4.8 DMAC cho CNN accelerator

Trong kiến trúc BK-AISoC, **DMAC** (Direct Memory Access Controller) được tích hợp để đảm nhiệm việc truyền dữ liệu giữa external memory và CNN accelerator. Nếu toàn bộ dữ liệu đều được di chuyển thông qua CPU, hệ thống sẽ phải tiêu tốn nhiều chu kỳ load/store và dễ tạo ra bandwidth bottleneck. Vì vậy, DMAC được đặt trên high-speed data path để accelerator có thể nhận và ghi dữ liệu với mức can thiệp tối thiểu từ CPU [2], [3].

4.8.1 Kiến trúc tổng thể

Sơ đồ tổng thể của khối DMAC được trình bày trong Hình 4.13. Về mặt kiến trúc, DMAC nằm giữa CNN accelerator và HyperRAM controller. Khối này tiếp nhận data transfer request, tạo memory access command và điều phối luồng dữ liệu đọc/ghi thông qua các FIFO trung gian.



Hình 4.13: Kiến trúc tổng thể của DMAC kết nối CNN accelerator và HyperRAM

Trong Hình 4.13, CNN accelerator không trực tiếp điều khiển toàn bộ giao thức truy cập HyperRAM mà chỉ phát các thông tin cần thiết cho một data transfer, bao gồm địa chỉ bắt đầu, chiều dài giao dịch và hướng truyền dữ liệu. DMAC nhận các thông tin này, chuyển thành memory access command phù hợp với HyperRAM controller và quản lý quá trình đưa dữ liệu qua các FIFO. Vì vậy, accelerator có thể tập trung vào phần tính toán, còn chi tiết truy cập external memory được gom vào DMAC.

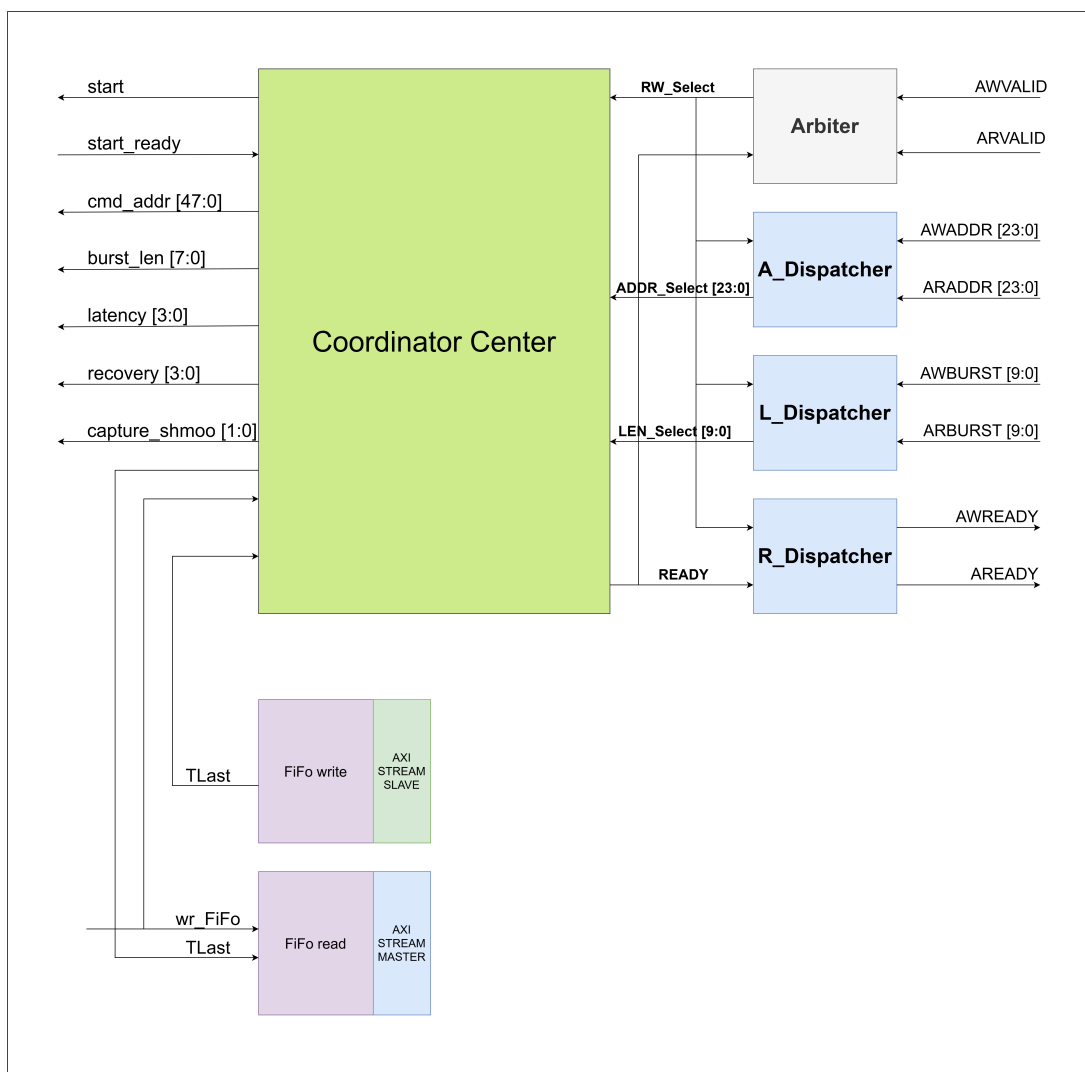
Control path của DMAC chịu trách nhiệm nhận address, transaction length và transaction type từ phía accelerator. Các thông tin này được chuyển thành control signal tương ứng cho HyperRAM controller. Data path được tổ chức theo dạng streaming, trong đó dữ liệu đọc từ HyperRAM được đưa qua read FIFO trước khi cấp cho accelerator, còn dữ liệu ghi từ accelerator được đệm qua write FIFO trước khi đưa ngược ra external memory [12].

Việc tách control path và data path giúp khối DMAC hoạt động ổn định hơn

khi xử lý các burst transfer dài. Các FIFO đóng vai trò hấp thụ sai lệch tốc độ giữa external memory và accelerator, đồng thời giảm nguy cơ mất dữ liệu khi một phía tạm thời chưa sẵn sàng.

4.8.2 Phân xử truy cập đọc/ghi

Cấu trúc chi tiết bên trong DMAC được minh họa trong Hình 4.14. Thành phần trung tâm của khối là control logic, có nhiệm vụ quản lý transfer state, chọn read channel hoặc write channel và phát lệnh đến external memory controller.



Hình 4.14: Kiến trúc chi tiết bên trong khối DMAC

Hình 4.14 mô tả rõ hơn cách DMAC tách các đường read và write. Ở đường read, control logic phát lệnh đọc đến HyperRAM controller, dữ liệu trả về được

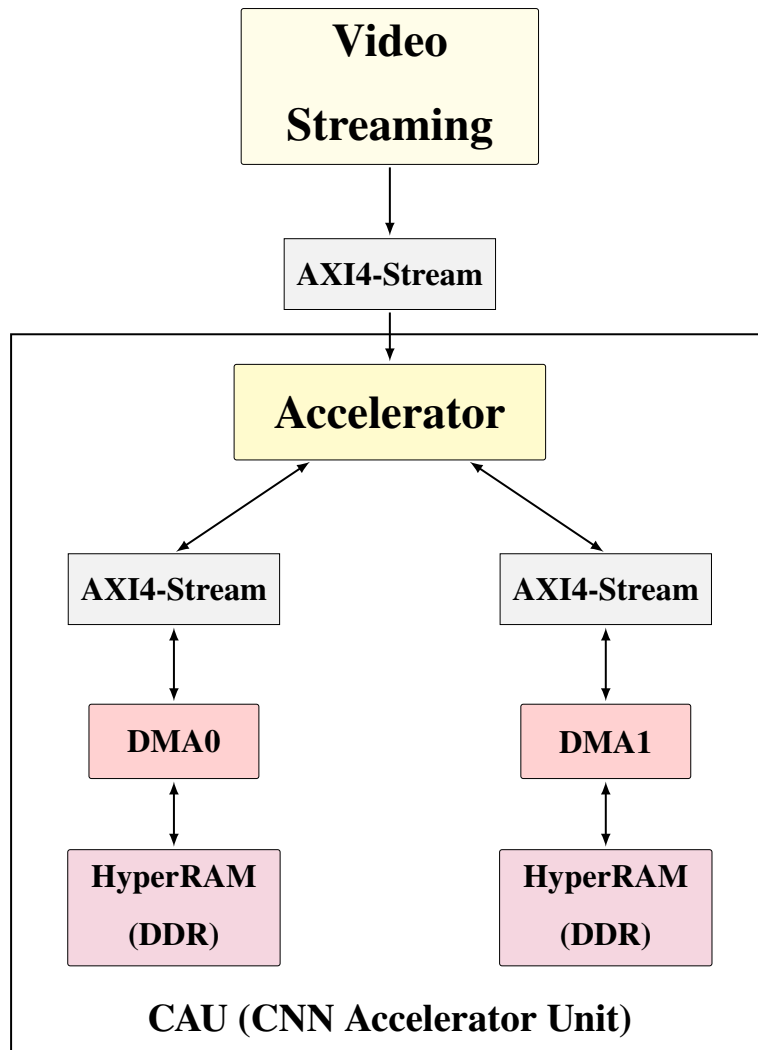
giữ trong read FIFO rồi cấp cho accelerator theo cơ chế handshake. Ở đường write, dữ liệu từ accelerator được đệm vào write FIFO trước khi DMAC phát lệnh ghi ra external memory. Việc dùng FIFO ở cả hai chiều giúp hai phía không cần hoạt động cùng tốc độ tuyệt đối, đồng thời tạo vùng đệm khi HyperRAM hoặc accelerator tạm thời chưa sẵn sàng.

Khi cả hai read request và write request cùng xuất hiện, DMAC cần có arbitration để tránh xung đột truy cập memory. Arbiter bên trong khối lựa chọn một transaction hợp lệ tại mỗi thời điểm, sau đó chuyển address, burst length và control signal tới control logic. Cơ chế này bảo đảm chỉ một luồng được cấp quyền truy cập memory trong một transaction, trong khi luồng còn lại được giữ ở trạng thái chờ.

Đối với luồng dữ liệu của CNN accelerator, nhu cầu đọc dữ liệu thường lớn hơn nhu cầu ghi kết quả. Vì vậy, arbitration policy có thể ưu tiên read channel trong một số giai đoạn để giữ cho accelerator không bị thiếu input data. Tuy nhiên, write channel vẫn phải được cấp quyền định kỳ để tránh làm đầy write FIFO và gây back-pressure về phía accelerator.

4.8.3 Trình tự hoạt động của phần cứng

Quá trình hoạt động của DMAC có thể được chia thành ba giai đoạn chính. Ở giai đoạn đầu, accelerator phát sinh data transfer request kèm address và transaction length. Ở giai đoạn thứ hai, DMAC arbitration chọn giữa read/write request và phát memory access command tới HyperRAM controller. Ở giai đoạn cuối, dữ liệu được truyền qua các FIFO theo handshake của streaming interface.



Hình 4.15: Vị trí của DMA0/DMA1 trong CAU và kết nối với Video Streaming

Hình 4.15 bổ sung góc nhìn ở mức hệ thống của CAU. Hai kênh DMA được đặt giữa HyperRAM và accelerator, đồng thời kết nối với accelerator qua các AXI4-Stream interface để vận chuyển weights, feature map hoặc kết quả trung gian mà không cần CPU thực hiện từng thao tác load/store [12]. Ở phía còn lại, khối Video Streaming có thể đưa dữ liệu ảnh vào accelerator thông qua AXI4-Stream, tạo thành đường dữ liệu trực tiếp từ camera/video pipeline sang khối xử lý CNN.

Với cách tổ chức này, DMAC trở thành một hardware block trung gian chuyên dụng giữa external memory và accelerator. CPU chỉ giữ vai trò điều phối cấp hệ thống, còn việc vận chuyển dữ liệu khối lượng lớn được thực hiện ở hardware. Đây là một thành phần quan trọng giúp BK-AISoC đáp ứng yêu cầu bandwidth

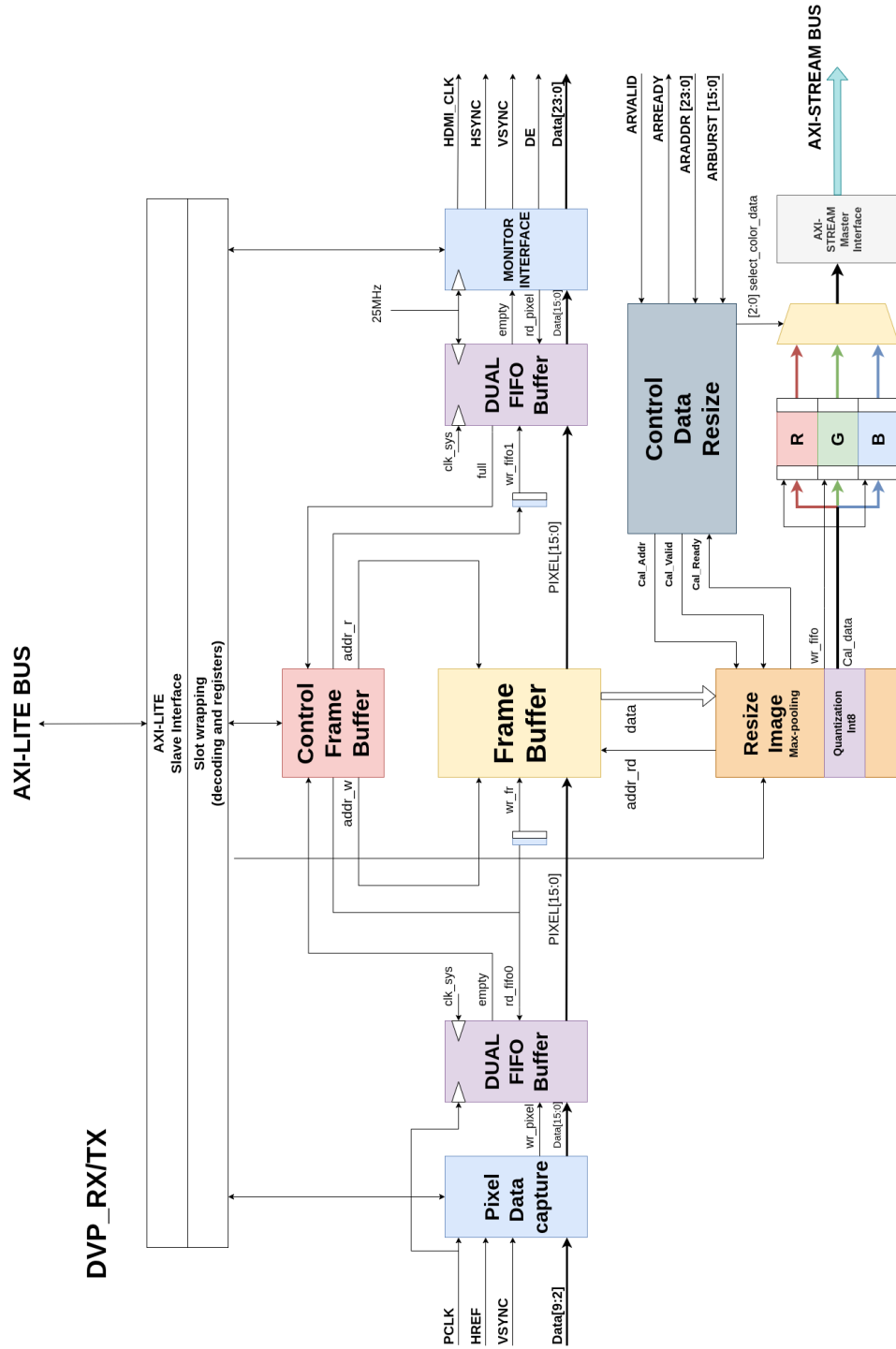
của các tác vụ AI.

4.9 Video Streaming

Video Streaming được tích hợp vào BK-AISoC để tạo image data path ở mức hardware. Khối này xử lý ba hướng dữ liệu chính: thu nhận dữ liệu từ image sensor, lưu trữ tạm thời trong Frame Buffer và cung cấp processed stream cho các khối phía sau. Đây là một phần của high-speed data path, tách biệt với control bus dùng cho register configuration, phù hợp với hướng xử lý dữ liệu hình ảnh theo luồng trong các accelerator thị giác nhúng [4].

4.9.1 Kiến trúc tổng thể

Kiến trúc của khối Video Streaming được minh họa trong Hình 4.16. Về mặt phần cứng, khối này gồm các thành phần chính: DVP receiver, clock-domain crossing FIFO, Frame Buffer, frame read/write controller, display output path và streaming path về phía accelerator. Trong phiên bản hiện tại, cụm Resize Image được bổ sung thêm các bước max-pooling, padding, INT8 quantization và tách dữ liệu theo từng kênh màu trước khi đưa về các FIFO độc lập.



Hình 4.16: Kiến trúc của khối Video Streaming có tích hợp Resize Image

Khối Video Streaming được kết nối với hệ thống thông qua memory-mapped control interface. Interface này chỉ đảm nhiệm cấu hình và đọc status, trong khi pixel data được truyền trên các data path riêng. Cách tách này giúp control bus không phải gánh toàn bộ pixel traffic của frame [11], [12].

4.9.2 Thu nhận dữ liệu ảnh

Image data đầu vào được thu từ camera sensor thông qua giao diện DVP song song. Ở tầng capture, phần cứng lấy mẫu các timing signal như PCLK, HREF và VSYNC để xác định valid pixel trong từng line và từng frame. Dữ liệu nhiều byte từ camera được ghép lại thành pixel hoàn chỉnh trước khi đưa vào internal buffer.

Do clock từ camera thuộc clock domain khác với system clock, khối Video Streaming sử dụng dual-clock FIFO để thực hiện Clock Domain Crossing (CDC). FIFO này giúp dữ liệu đi từ PCLK domain sang system domain mà không làm mất tính toàn vẹn tín hiệu, đồng thời giảm nguy cơ metastability khi hai clock domain không đồng bộ.

4.9.3 Frame Buffer và điều phối khung hình

Frame Buffer là vùng đệm trung gian lưu trữ pixel sau khi capture. Frame Buffer controller tạo write address theo camera data stream và tạo read address theo yêu cầu của các data path phía sau. Nhờ đó, luồng ghi và luồng đọc có thể hoạt động ở các tốc độ khác nhau nhưng vẫn cùng chia sẻ một frame data region.

Việc điều phối Frame Buffer cần bảo đảm hai yêu cầu. Thứ nhất, dữ liệu đang được ghi không được làm sai lệch dữ liệu đang được đọc. Thứ hai, các read path phục vụ display hoặc accelerator phải nhận dữ liệu theo thứ tự ổn định. Vì vậy, controller được tổ chức quanh line/column counter, frame sync signal và các read/write state.

4.9.4 Display output path

Display output path đọc dữ liệu từ Frame Buffer và tạo các timing signal cần thiết cho giao diện hiển thị bên ngoài. Khối này tạo các tín hiệu như HSYNC, VSYNC và Data Enable, đồng thời chuyển đổi pixel data sang định dạng phù hợp với video output. Một clock-domain crossing FIFO được sử dụng để đưa dữ

liệu từ system clock domain sang display clock domain.

Ở mức hardware, display path giúp kiểm chứng trực tiếp image capture flow và Frame Buffer. Nếu DVP receiver, Frame Buffer và Timing Generator hoạt động đúng, hình ảnh có thể được tái tạo ổn định ở display output.

4.9.5 Streaming path về phía accelerator

Bên cạnh display path, khối Video Streaming còn tạo một data path hướng về CNN accelerator. Đường này đọc dữ liệu từ Frame Buffer, sau đó đưa qua khối Resize Image để chuyển đổi kích thước ảnh về dạng phù hợp hơn cho xử lý AI. Việc preprocessing ở hardware giúp giảm lượng dữ liệu cần đưa vào accelerator và tránh để CPU thực hiện thao tác biến đổi pixel theo từng điểm ảnh.

Trong cụm Resize Image, ảnh đầu vào kích thước 640×480 được chuyển về kích thước 224×224 bằng cơ chế max-pooling kết hợp padding. Max-pooling chọn giá trị lớn nhất trong từng vùng lấy mẫu, nhờ đó vừa giảm kích thước ảnh vừa giữ lại các đặc trưng cường độ nổi bật. Cơ chế padding được dùng để xử lý vùng biên và bảo đảm output frame có kích thước cố định, phù hợp với kích thước đầu vào của mô hình CNN.

Sau bước resize, dữ liệu pixel được lượng tử hóa về định dạng **INT8**. Việc dùng INT8 giúp giảm bandwidth trên data path và giảm dung lượng lưu trữ trung gian so với biểu diễn có độ rộng lớn hơn [7]. Dữ liệu sau quantization được tách thành ba kênh màu riêng biệt, mỗi kênh đi vào một FIFO độc lập. Cách tổ chức ba FIFO độc lập giúp tách luồng dữ liệu theo kênh, giảm phụ thuộc giữa các kênh màu và tạo interface rõ ràng hơn cho khối accelerator ở phía sau.

Như vậy, Video Streaming không chỉ là camera interface block, mà còn là một phần trong AI data path của SoC. Khối này đảm nhiệm việc đưa image data từ peripheral domain vào processing domain, đồng thời chuẩn bị dữ liệu ở mức hardware trước khi chuyển sang accelerator.

4.10 Kết luận chương

Phần trên đã trình bày kiến trúc phần cứng của BK-AISoC ở mức hệ thống, bao gồm PicoRV32 processor core, on-chip memory, control bus, peripheral controller, external memory, DMAC, Video Streaming và CNN accelerator. Các phần tiếp theo trình bày lớp firmware dùng để khởi động, điều khiển và kiểm chứng hệ thống, sau đó mô tả quá trình triển khai toàn bộ thiết kế trên FPGA.

4.11 Thiết kế Firmware BK-AISoC

4.11.1 Mục tiêu và vai trò của Firmware

Firmware của BK-AISoC được phát triển theo mô hình **bare-metal**, chạy trực tiếp trên vi xử lý RISC-V PicoRV32 mà không thông qua hệ điều hành [8], [10]. Trong kiến trúc tổng thể của hệ thống, firmware đóng vai trò trung gian giữa hardware của SoC, các peripheral trên bo mạch và khối CNN accelerator. Cụ thể, firmware đảm nhiệm các nhiệm vụ: khởi tạo hệ thống sau reset, cung cấp *hardware abstraction layer* (HAL) cho các peripheral, hỗ trợ nạp chương trình qua UART bootloader, nạp dữ liệu weights xuống bộ nhớ ngoài, cấu hình khối Camera và Video Streaming, và cung cấp các hàm điều khiển để CPU có thể cấu hình, khởi chạy và giám sát CNN accelerator.

Trong phạm vi luận văn, firmware được hiện thực như một dự án độc lập có tên là BK-AISoC-Firmware. Dự án này bao gồm application code, startup code, linker script, bootloader, các HAL driver, unit test và các host tool chạy trên host computer. Việc tách firmware thành một dự án riêng giúp quá trình kiểm thử và hiệu chỉnh hardware diễn ra linh hoạt hơn: khi thay đổi application hoặc test scenario, chương trình có thể được build và nạp lại qua UART mà không cần tổng hợp lại toàn bộ thiết kế FPGA.

4.11.2 Tổ chức mã nguồn Firmware

Cấu trúc thư mục mã nguồn chính của firmware được tổ chức theo các lớp chức năng rõ ràng như Listing 4.1. Listing này chỉ trình bày thư mục mã nguồn bên trong dự án, nơi chứa phần mã chạy trên SoC và các công cụ hỗ trợ trực tiếp cho quá trình build, nạp firmware và kiểm thử. Cách tổ chức này giúp tách riêng application code, driver, bootloader và các host tool hỗ trợ trên host computer.

Listing 4.1: Source tree của Firmware

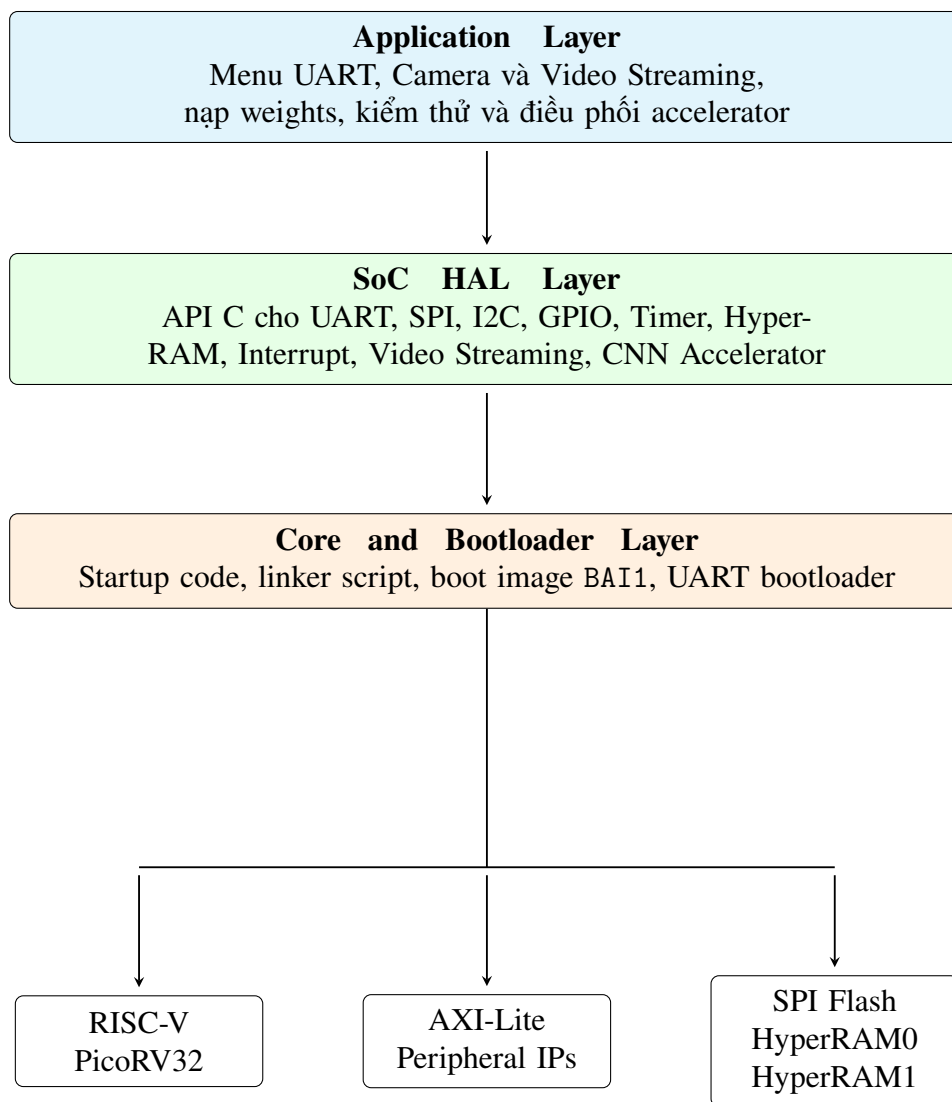
```
BK-AISoC_Firmware/  
|-- App/  
|   |-- Inc/  
|   '-- Src/  
|-- Core/  
|   |-- Linker/  
|   '-- Startup/  
|-- Drivers/  
|   |-- Bootloader/  
|   '-- SoC_HAL/  
|       |-- Inc/  
|       '-- Src/  
|-- scripts/  
'-- testing/
```

Vai trò của các thành phần mã nguồn được tóm tắt trong Bảng 4.1. Nhân mạnh ranh giới chức năng giữa application, phần khởi động, driver phần cứng và các công cụ hỗ trợ trên host computer.

Kiến trúc phần mềm có thể được xem như ba lớp. Lớp thấp nhất là **Core** và **Bootloader**, chịu trách nhiệm đưa chương trình vào đúng memory region và chuyển quyền điều khiển cho application. Lớp giữa là **SoC HAL**, che giấu thao tác đọc và ghi register, đồng thời cung cấp API C cho từng peripheral. Lớp trên cùng là **Application**, nơi các driver được kết hợp thành các luồng kiểm thử và AI inference hoàn chỉnh.

Bảng 4.1: Vai trò của các thành phần mã nguồn trong firmware

Thành phần	Mô tả chức năng
App/	Chứa application logic chạy sau khi hệ thống khởi động, bao gồm menu UART, điều khiển Camera và Video Streaming, nạp weights, kiểm thử HyperRAM và các luồng chạy thử CNN accelerator. Hai thư mục con Inc/ và Src/ lần lượt chứa header file và phần hiện thực C của application.
Core/	Chứa các thành phần nền tảng để chương trình có thể được đặt đúng vào memory map của SoC và bắt đầu thực thi. Trong đó, Linker/ chứa linker script, còn Startup/ chứa startup code trước khi gọi vào main().
Drivers/	Chứa các driver mức thấp, tách thao tác truy cập phần cứng khỏi application logic. Bootloader/ hiện thực giao thức nạp firmware qua UART, kiểm tra CRC, ghi SPI Flash và chuyển quyền điều khiển sang application. SoC_HAL/ cung cấp API C cho UART, SPI, I2C, GPIO, Timer, HyperRAM, Video Streaming, Interrupt và CNN accelerator.
scripts/	Chứa các host tool phục vụ build, flash firmware, nạp weights và tự động hóa thao tác từ host computer.
testing/	Chứa các file hoặc cấu hình hỗ trợ kiểm thử firmware trong mô phỏng và trên bo mạch thật.



Hình 4.17: Kiến trúc phân lớp của firmware BK-AISoC

4.11.3 Boot Flow và Memory Map

4.11.3.1 Memory map của application

Application firmware sử dụng instruction memory và data memory riêng. Instruction region của application bắt đầu tại địa chỉ 0x01100000 với kích thước 64 KB. Data region dùng DMEM bắt đầu tại 0x00000000. Tổng dung lượng DMEM vật lý là 64 KB, tuy nhiên firmware chia cố định 56 KB đầu cho application và giữ 8 KB cuối, từ 0x0000E000 đến 0x0000FFFF, cho bootloader. Cách chia này tránh trường hợp application ghi đè stack, biến cục bộ hoặc trạng thái còn cần thiết của bootloader trong quá trình nạp và chuyển quyền điều khiển.

Trong linker script, các section `.vectors`, `.text` và `.rodata` được đặt vào vùng FLASH tại 0x01100000. Các section `.data`, `.bss`, `.heap` và `.stack` được đặt vào vùng RAM của application. Stack được cấp phát mặc định 2048 bytes và được align 16 byte để phù hợp với calling convention của RISC-V. Linker script cũng có các lệnh ASSERT để phát hiện lỗi tràn RAM ngay ở bước build, đặc biệt là lỗi application lấn sang vùng 8 KB dành cho bootloader.

Bảng 4.2: Phân vùng bộ nhớ chính của firmware

Phân vùng	Địa chỉ hoặc offset	Mục đích sử dụng
Không gian địa chỉ hệ thống		
App IMEM	0x01100000 - 0x0110FFFF	Mã lệnh, vector, hằng số
App DMEM	0x00000000 - 0x0000DFFF	<code>.data</code> , <code>.bss</code> , heap, stack
Bootloader DMEM	0x0000E000 - 0x0000FFFF	RAM riêng của bootloader
Bố cục lưu trữ trong SPI Flash		
Metadata region	Flash offset 0x00000000	Magic, size và CRC của firmware
Application image	Flash offset 0x00001000	Boot image của application

Trong Bảng 4.2, các giá trị của *Metadata region* và *Application image* là offset nội bộ trong chip SPI Flash. Đây không phải là địa chỉ trong không gian địa chỉ hệ thống của CPU; bootloader truy cập các vùng này thông qua SPI

controller để đọc metadata và boot image trước khi nạp nội dung vào IMEM và DMEM.

4.11.3.2 Segmented boot image BAI1

Trong quá trình thiết kế bootloader, định dạng boot image của firmware được xây dựng dựa trên ý tưởng từ boot flow của các chipset MediaTek. Trong các hệ thống này, image lưu trong non-volatile memory không chỉ là một dãy byte liên tục; bootloader cần đọc phần header hoặc metadata để nhận diện image, kiểm tra tính hợp lệ và xác định từng thành phần cần được nạp vào bộ nhớ trước khi chuyển quyền điều khiển. Luận văn này áp dụng lại nguyên lý đó cho BK-AISoC, nhưng không sử dụng nguyên xi định dạng của MediaTek do memory map, kích thước bộ nhớ và yêu cầu nạp IMEM và DMEM của hệ thống là khác nhau.

Trong firmware BK-AISoC, BAI1 được dùng như một *magic number* bốn byte của boot image. Trong mã nguồn bootloader, giá trị này được định nghĩa là 0x31494142, tương ứng với chuỗi ASCII BAI1 theo thứ tự little-endian. Khi đọc image từ SPI Flash, bootloader kiểm tra giá trị này để phân biệt image hợp lệ với dữ liệu sai định dạng, từ đó tránh việc copy nhầm dữ liệu vào instruction memory hoặc data memory.

Về mặt kỹ thuật, BAI1 được tổ chức theo dạng **segmented image**. Thay vì xem firmware sau build là một dãy byte liên tục, boot image được chia thành nhiều segment. Mỗi segment mô tả một thao tác nạp cụ thể, bao gồm loại segment, vị trí dữ liệu trong image, địa chỉ đích trong SoC và kích thước dữ liệu. Cách tổ chức này tương tự về mặt ý tưởng với các executable format có program header, chẳng hạn ELF, nhưng được rút gọn để bootloader bare-metal có thể phân tích nhanh và dễ kiểm soát.

Firmware hiện tại vì vậy không sử dụng *flat binary*. Sau khi build, file Build/main.bin được tạo dưới dạng **segmented boot image** với magic BAI1. Định dạng này mô tả rõ dữ liệu nào cần được nạp vào instruction memory, dữ liệu nào cần được nạp vào data memory và vùng nào cần được zero trước khi

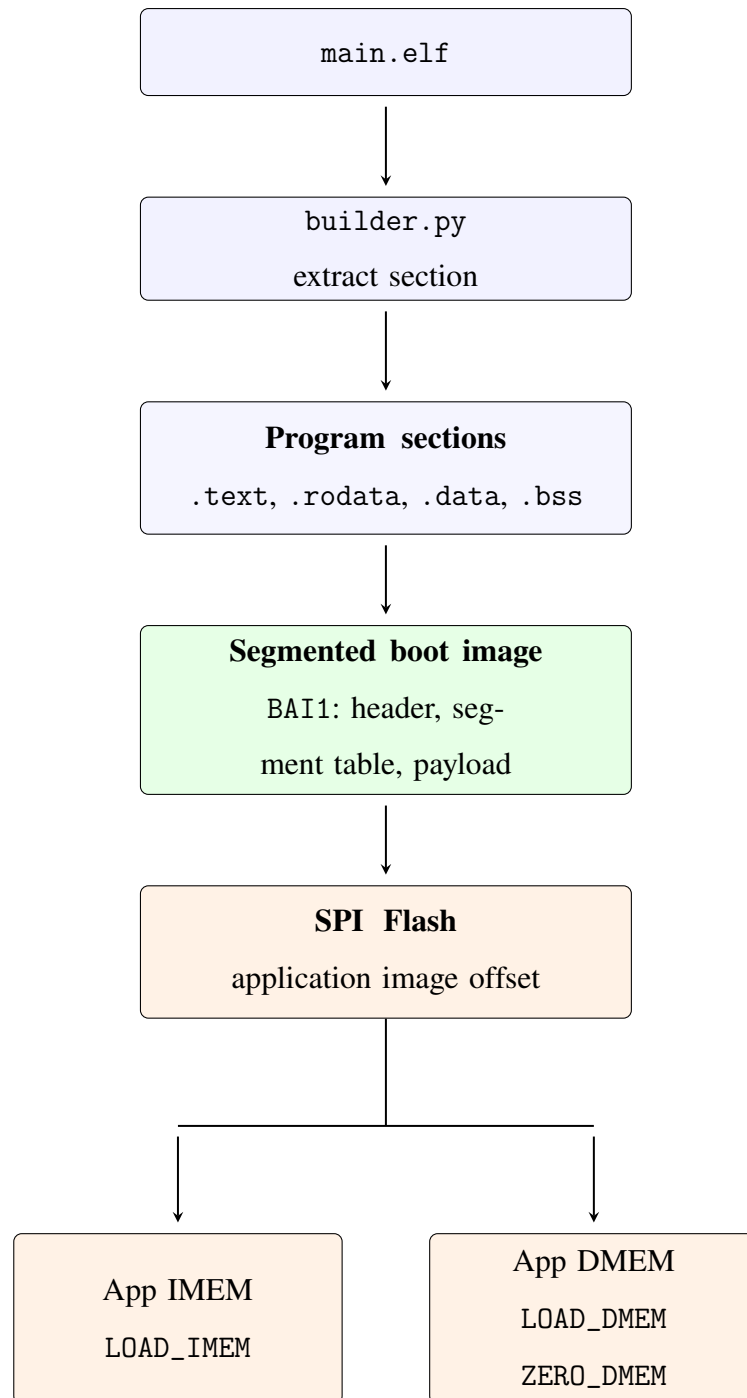
application bắt đầu chạy.

Về cấu trúc, một boot image BAI1 gồm ba thành phần chính. Thành phần thứ nhất là **header**, chứa magic BAI1, format version, entry point và segment count. Thành phần thứ hai là **segment table**, trong đó mỗi entry mô tả segment type, image offset, destination address và size. Thành phần cuối cùng là **payload**, chứa dữ liệu thực tế của các section chương trình. Mỗi segment mô tả một thao tác mà bootloader cần thực hiện trước khi chuyển quyền điều khiển cho application:

- **LOAD_IMEM**: copy các section chứa vector, instruction code và hằng số vào vùng IMEM của application.
- **LOAD_DMEM**: copy dữ liệu khởi tạo của `.data` vào vùng DMEM.
- **ZERO_DMEM**: ghi 0 cho vùng `.bss`.

Sự cần thiết của định dạng BAI1 xuất phát từ memory architecture của SoC. Application không chỉ có instruction code nằm trong IMEM mà còn có initialized data trong DMEM và vùng `.bss` cần được zero. Nếu chỉ sử dụng một flat binary, bootloader khó xác định boundary giữa instruction code, initialized data và vùng cần zero. Với BAI1, thông tin này được mô tả tường minh trong segment table, nhờ đó bootloader có thể kiểm tra destination address, copy đúng memory region và từ chối image không hợp lệ.

Với cơ chế này, bootloader chịu trách nhiệm chuẩn bị cả IMEM và DMEM trước khi chuyển quyền điều khiển. Startup code của application vì vậy không cần tự copy `.data` từ IMEM sang DMEM như phương án cũ. Điều này giúp tránh lưu trùng initialized data trong IMEM, đồng thời làm giảm rủi ro ghi nhầm sang vùng RAM 8 KB dành cho bootloader.



Hình 4.18: Định dạng segmented boot image của application

4.11.4 Bootloader và công cụ nạp firmware

4.11.4.1 Giao thức flash application qua UART

Bootloader được nhúng trong SoC để nhận application mới từ host computer thông qua UART. Trên host computer, tool `host_flasher.py` đọc file `Build/`

main.bin, tính size và CRC, sau đó truyền xuống bootloader theo các command điều khiển.

Ở thời điểm reset, bootloader không mặc định yêu cầu nạp lại firmware trong mọi lần khởi động. Thay vào đó, bootloader mở một cửa sổ chờ ngắn để host computer có thể gửi lệnh flash mới qua UART. Trong hiện thực hiện tại, khoảng chờ này được đặt là 2 s. Nếu trong khoảng thời gian này không có yêu cầu nạp firmware hợp lệ, bootloader sẽ tự động đọc image hiện có trong SPI Flash, kiểm tra metadata và chuyển quyền điều khiển sang application. Cách làm này giúp quá trình phát triển thuận tiện hơn: hệ thống vẫn có thể tự boot vào application khi không kết nối tool nạp, nhưng vẫn cho phép host can thiệp khi cần cập nhật firmware.

Giá trị timeout 2 s hiện được chọn theo hướng đơn giản và chưa quá khắt khe, phù hợp với giai đoạn kiểm thử trên FPGA. Trong các phiên bản sau, cơ chế vào program mode có thể được cải tiến bằng cách kết hợp các tín hiệu điều khiển của USB-UART, chẳng hạn DTR hoặc RTS. Khi đó, host tool có thể chủ động đưa SoC vào trạng thái nạp firmware thay vì phụ thuộc hoàn toàn vào việc chờ timeout sau reset.

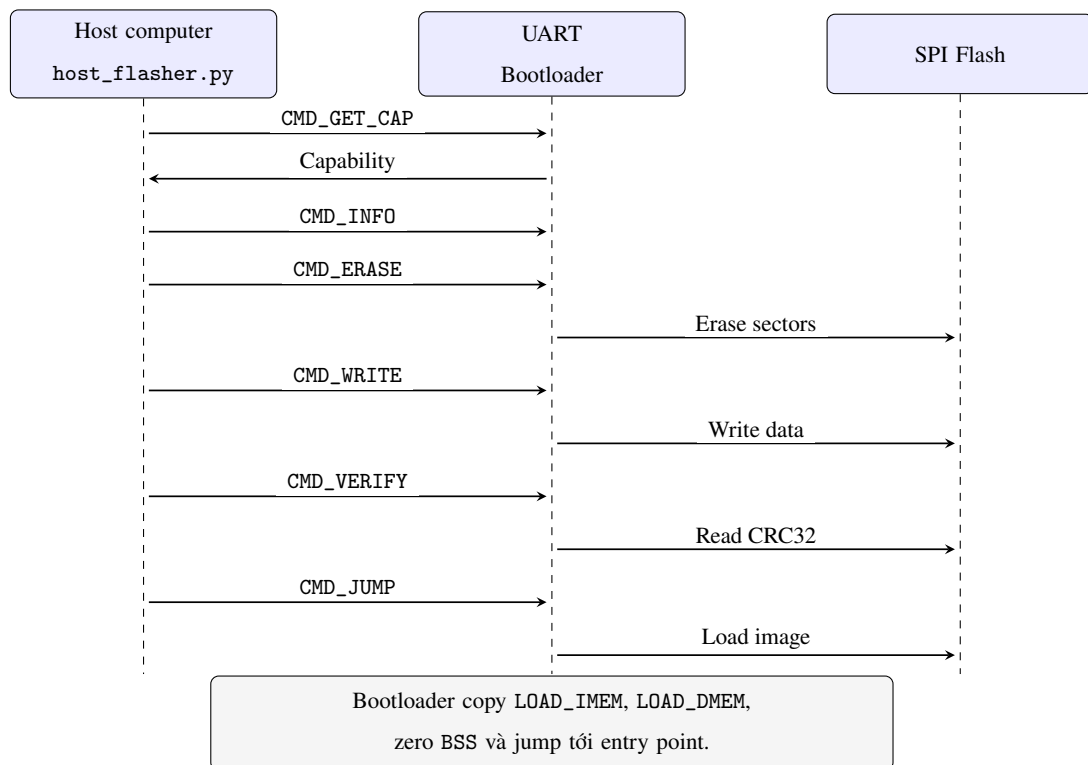
Khi host computer yêu cầu nạp firmware mới, giao thức chính gồm các bước:

1. CMD_GET_CAP: handshake với bootloader và đọc capability của target.
2. CMD_INFO: gửi metadata của firmware, gồm size và CRC32.
3. CMD_ERASE: yêu cầu xóa vùng SPI Flash dành cho application.
4. CMD_WRITE: gửi từng block dữ liệu xuống flash.
5. CMD_VERIFY: yêu cầu target kiểm tra CRC32 toàn image đã ghi.
6. CMD_JUMP: yêu cầu bootloader đọc boot image, copy từng segment và nhảy vào application.

Mỗi frame UART dùng byte bắt đầu 0xA5, byte kết thúc 0x55, trường sequence, command, length, payload và CRC16. Kích thước payload tối đa là 1024

bytes. Riêng lệnh CMD_WRITE chứa thêm offset 4 byte trong payload, do đó kích thước dữ liệu thực tế của mỗi packet thường được chọn nhỏ hơn giới hạn này. Trong giai đoạn kiểm thử hardware, kích thước mỗi packet dữ liệu thường được đặt là 64 bytes để tăng độ ổn định trên đường UART.

Bootloader có cơ chế phục hồi khi UART bị nhiễu hoặc frame truyền dở. Nếu frame parser đang nhận dữ liệu nhưng không hoàn tất trong 100 ms, trạng thái nhận sẽ được reset. Host tool cũng hỗ trợ retry khi timeout hoặc nhận NACK, đồng thời thống kê số lần lỗi truyền trong quá trình nạp. Sau khi ghi xong, bước CMD_VERIFY dùng CRC32 toàn file để bảo đảm image trong SPI Flash khớp với image trên host.



Hình 4.19: Luồng flash application qua UART bootloader

4.11.4.2 Quá trình chuyển quyền điều khiển

Khi nhận CMD_JUMP, bootloader không nhảy trực tiếp vào dữ liệu trong SPI Flash. Thay vào đó, bootloader đọc metadata ở sector đầu của flash, kiểm tra magic, size và CRC đã commit. Sau đó bootloader đọc header BAI1, kiểm tra

segment table và đảm bảo destination address của từng segment nằm trong vùng hợp lệ. Các thao tác nạp chính gồm:

- Segment LOAD_IMEM được copy vào vùng 0x01100000 - 0x0110FFFF.
- Segment LOAD_DMEM được copy vào vùng 0x00000000 - 0x0000DFFF.
- Segment ZERO_DMEM được ghi 0 trước khi application chạy.

Khi toàn bộ bộ nhớ application đã được chuẩn bị, bootloader nhảy tới entry point 0x01100000.

Quy trình này giúp bootloader đóng vai trò như một program loader hoàn chỉnh thay vì chỉ là một UART receiver đơn giản. Lợi ích quan trọng nhất là application có thể chứa cả initialized data trong DMEM mà không cần cơ chế tự copy phức tạp ở giai đoạn startup. Đồng thời, bootloader có thể từ chối các image sai định dạng hoặc cố ghi vượt vùng nhớ được phép, giúp hạn chế lỗi treo hệ thống trong quá trình phát triển.

4.11.5 SoC Hardware Abstraction Layer

4.11.5.1 Mục tiêu thiết kế HAL

Trong firmware BK-AISoC, *hardware abstraction layer* (HAL) đóng vai trò là lớp giao tiếp giữa application, bootloader và các peripheral được tích hợp trong SoC. Do hệ thống chạy theo mô hình bare-metal, firmware không có kernel driver hoặc device tree như trong hệ điều hành. Vì vậy, HAL được thiết kế theo hướng gọn nhẹ: cung cấp các API C đủ rõ ràng để application có thể khởi tạo, cấu hình, truyền nhận dữ liệu và kiểm tra trạng thái phần cứng mà không phải thao tác trực tiếp với từng register.

Mục tiêu của HAL không phải là che giấu hoàn toàn đặc tính của hardware, vì trong quá trình kiểm thử FPGA người phát triển vẫn cần quan sát địa chỉ, trạng thái và timing của từng peripheral. Thay vào đó, HAL chuẩn hóa cách truy cập phần cứng, cô lập các chi tiết như base address, register offset, bit điều khiển

và trình tự khởi tạo vào một lớp trung gian. Nhờ vậy, các lớp firmware phía trên chỉ cần sử dụng một tập API thống nhất khi điều khiển peripheral, thay vì lặp lại thao tác truy cập register ở nhiều vị trí khác nhau.

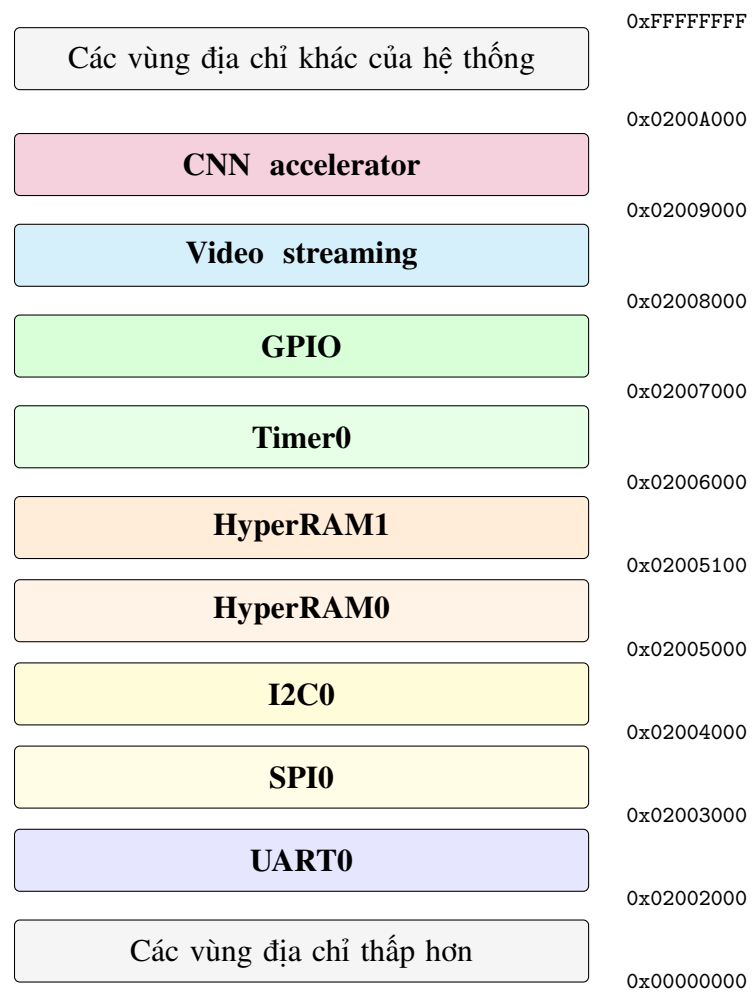
4.11.5.2 Mô hình truy cập register

Các peripheral trong BK-AISoC được ánh xạ vào không gian địa chỉ của CPU theo cơ chế **memory-mapped I/O** [11]. Do CPU sử dụng địa chỉ 32 bit, không gian địa chỉ lý thuyết có kích thước 2^{32} byte, tương ứng **4 GB**, trải từ 0x00000000 đến 0xFFFFFFFF. Firmware truy cập register bằng các con trỏ volatile, bảo đảm compiler không tối ưu bỏ các thao tác đọc và ghi có tác dụng phụ ở phía hardware. Trên cơ sở đó, mỗi khối HAL nhận base address tương ứng, sau đó thực hiện thao tác đọc và ghi thông qua offset của từng register.

Cách tổ chức này giúp firmware tách được hai phần: phần mô tả vị trí peripheral trong hệ thống và phần logic điều khiển peripheral. Với cấu hình firmware hiện tại, application đang truy cập trực tiếp các instance nối tiếp chính gồm **UART0**, **SPI0** và **I2C0**. Tuy nhiên, các API được tổ chức theo hướng *instance-based*; khi cần dùng thêm UART1, SPI1 hoặc I2C1, firmware chủ yếu cần bổ sung base address và cấu hình instance, trong khi phần logic truyền nhận có thể tái sử dụng.

4.11.5.3 Tổ chức chức năng của HAL

Xét theo vai trò trong firmware, HAL có thể được tổ chức thành bốn phần chức năng chính. Phần thứ nhất bao gồm các giao tiếp điều khiển và cấu hình, như UART, SPI và I2C. Ví dụ, trong cấu hình firmware hiện tại, UART0 đóng vai trò console, kênh nhận command từ người dùng và kênh giao tiếp với bootloader; SPI0 đảm nhiệm truy cập SPI Flash để lưu boot image và weights; I2C0 đảm nhiệm cấu hình cảm biến camera OV5640. Các giao tiếp này có bảng thông và vai trò khác nhau, nhưng đều được application truy cập thông qua các API



Hình 4.20: Sơ đồ không gian địa chỉ 32-bit được HAL sử dụng

truyền nhận ở mức byte hoặc transaction.

Phần thứ hai bao gồm các khối đảm nhiệm lưu trữ và di chuyển dữ liệu. SPI Flash giữ firmware image và weights ở dạng non-volatile, trong khi HyperRAM đóng vai trò bộ nhớ ngoài dung lượng lớn cho dữ liệu trung gian của mô hình CNN. Ví dụ, trong luồng chạy accelerator hiện tại, HyperRAM0 lưu weights và biases, còn HyperRAM1 lưu IFMAP và OFMAP. HAL che giấu các thao tác timing, burst access và kiểm tra trạng thái FIFO, để application chỉ cần quan tâm đến địa chỉ logic, kích thước dữ liệu và thứ tự thực hiện.

Phần thứ ba là các khối đồng bộ và giám sát trạng thái, gồm Timer0, GPIO và Interrupt. Trong đó, Timer0 là khối timer memory-mapped riêng, được firmware sử dụng để đọc cycle counter, đo thời gian thực thi và profiling các đoạn chương trình. Phần interrupt được tổ chức riêng với hai nguồn chính: *timer interrupt* và *event interrupt* từ ngoại vi CNN accelerator. Timer interrupt tạo sự kiện ngắt theo cấu hình thời gian, còn accelerator event interrupt cho phép accelerator báo về CPU khi có sự kiện cần xử lý, ví dụ trạng thái hoàn tất hoặc cập nhật trạng thái của accelerator. GPIO đảm nhiệm các tín hiệu điều khiển số đơn giản trong quá trình kiểm thử. Interrupt được tổ chức theo hướng handler ngắn: khi hardware phát sinh sự kiện, firmware chỉ ghi nhận trạng thái hoặc đặt flag, còn phần xử lý dài hơn được thực hiện ở main loop. Cách làm này giúp giảm interrupt latency và tránh làm phức tạp các luồng truy cập bộ nhớ ngoài.

Phần cuối cùng là các khối gắn trực tiếp với chức năng AI và xử lý video của hệ thống, bao gồm Video Streaming và CNN accelerator. Ở lớp này, HAL chuyển các tham số mức application thành giá trị register: địa chỉ buffer, kích thước feature map, trạng thái enable, start, done và các tham số cấu hình layer. Nhờ có lớp HAL, application có thể điều phối camera, HyperRAM và accelerator theo một luồng kiểm thử thống nhất mà không cần lặp lại thao tác đọc và ghi register ở từng bước.

Nhìn chung, HAL trong luận văn này được xem như một lớp thích nghi giữa phần cứng đang phát triển và application kiểm thử. Khi register map hoặc số

lượng peripheral thay đổi, phần cần hiệu chỉnh chủ yếu nằm ở lớp HAL; các luồng application phía trên vẫn giữ được cấu trúc điều khiển tổng quát.

4.11.6 Application mẫu kiểm chứng hệ thống

Phần application trong firmware được hiện thực như một *reference application* nhằm chứng minh BK-AISoC có thể vận hành đồng thời phần cứng và phần mềm trong một luồng hoàn chỉnh. Phần này trình bày các hướng công việc mà application đảm nhiệm trong quá trình bring-up, kiểm thử và chạy thử mô hình AI trên hệ thống.

Application mẫu không được xem là sản phẩm cuối, mà là chương trình kiểm chứng nền tảng. Nó cho phép CPU RISC-V sử dụng lớp HAL để điều phối peripheral, bộ nhớ ngoài, Video Streaming và CNN accelerator. Từ đó, luận văn có thể chứng minh rằng hệ thống không chỉ có từng IP hoạt động riêng lẻ, mà còn có khả năng phối hợp thành một pipeline phần mềm và phần cứng hoàn chỉnh.

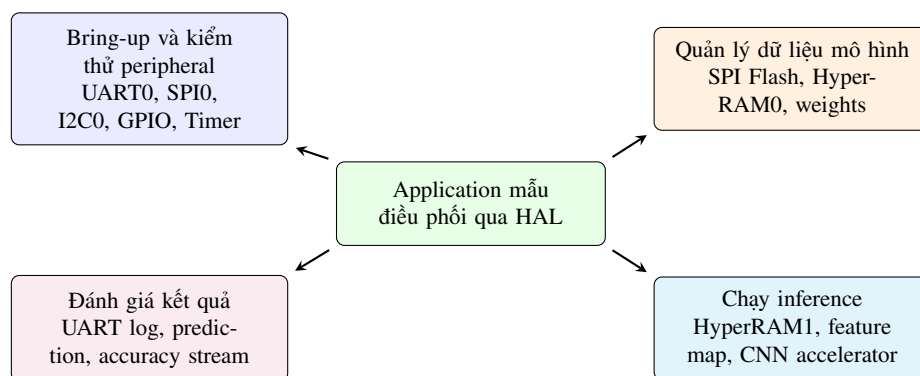
4.11.6.1 Các hướng chức năng của application

Hướng thứ nhất là **bring-up và kiểm thử peripheral**. Application cung cấp giao diện UART để người dùng kích hoạt từng nhóm chức năng, quan sát log và kiểm tra phản hồi của các khối như UART0, SPI0, I2C0, GPIO, Timer, HyperRAM và Video Streaming. Đây là hướng quan trọng trong giai đoạn phần cứng còn thay đổi, vì firmware cần giúp phát hiện nhanh lỗi kết nối, lỗi timing hoặc lỗi cấu hình của từng IP.

Hướng thứ hai là **quản lý dữ liệu cho mô hình AI**. Vì weights của CNN lớn hơn bộ nhớ nội của CPU, application tổ chức luồng đưa dữ liệu từ host computer xuống SPI Flash, sau đó nạp sang HyperRAM khi cần chạy inference. Trong luồng hiện tại, HyperRAM0 được dùng cho weights và các tham số của mô hình, còn HyperRAM1 được dùng cho ảnh đầu vào, feature map trung gian và output.

Hướng thứ ba là **điều phối quá trình inference**. Application không trực tiếp thực hiện toàn bộ phép tính CNN bằng CPU, mà chuẩn bị dữ liệu, cấu hình tham số layer thông qua HAL, kích hoạt CNN accelerator và theo dõi trạng thái hoàn tất bằng Interrupt hoặc status register. Nhờ vậy, CPU giữ vai trò điều phối hệ thống, còn các phép tính tích chập được chuyển sang accelerator và hệ thống bộ nhớ ngoài.

Hướng thứ tư là **đánh giá kết quả và hỗ trợ mở rộng**. Application có thể chạy một lớp kiểm thử riêng, chạy toàn bộ mô hình ALL-CNN-C hoặc nhận nhiều ảnh CIFAR-10 từ host computer để đánh giá accuracy qua UART. Cách tổ chức này tạo cơ sở để thay đổi mô hình trong tương lai: phần cần cập nhật chủ yếu là weight blob, metadata của mô hình và trình tự cấu hình ở mức application, trong khi bootloader, HAL, driver bộ nhớ và cơ chế điều phối accelerator vẫn có thể tái sử dụng.



Hình 4.21: Các hướng chức năng của application mẫu trên BK-AISoC

4.11.6.2 Kiểm chứng hoạt động end-to-end

Để kiểm chứng từng bước, application hỗ trợ chạy riêng layer Conv1 và so sánh kết quả với reference output do CPU tính. Sau đó, luồng được mở rộng để chạy toàn bộ 9 layer của ALL-CNN-C, thực hiện global average pooling và xuất kết quả dự đoán [6]. Firmware cũng hỗ trợ chế độ gửi nhiều ảnh CIFAR-10 từ host computer qua UART để đánh giá accuracy mà không cần nạp lại firmware [24].

Các luồng trên cho thấy application đã kiểm chứng được đường chạy end-to-

end của hệ thống: host computer nạp dữ liệu, firmware quản lý bộ nhớ ngoài, CPU cấu hình accelerator thông qua HAL, accelerator xử lý dữ liệu và kết quả được trả về qua UART. Vì vậy, phần application trong luận văn được xem như minh chứng thực nghiệm rằng nền tảng BK-AISoC có thể được dùng để triển khai một mô hình AI hoàn chỉnh, thay vì chỉ kiểm thử riêng lẻ từng peripheral.

4.11.7 Kết quả hiện thực Firmware

Kết quả đầu tiên của phần firmware là xây dựng được một nền tảng bare-metal hoàn chỉnh cho BK-AISoC, bao gồm startup code, linker script, bootloader, HAL và application mẫu. Boot flow đã được chuyển sang dạng segmented boot image BAI1, cho phép bootloader nạp riêng các segment IMEM, DMEM và zero BSS trước khi chuyển quyền điều khiển cho application. Đồng thời, vùng DMEM 64 KB được phân chia thành 56 KB cho application và 8 KB cho bootloader, nhờ đó tránh trường hợp application ghi đè vùng RAM mà bootloader vẫn cần sử dụng.

Ở lớp truy cập phần cứng, firmware đã hoàn thiện HAL cho các peripheral chính của SoC hiện tại, gồm UART0, SPI0, I2C0, GPIO, Timer, HyperRAM0, HyperRAM1, Video Streaming, Interrupt và CNN accelerator. Mặc dù application hiện tại chủ yếu sử dụng UART0, SPI0 và I2C0, cấu trúc driver vẫn được tổ chức để có thể mở rộng thêm UART1, SPI1 và I2C1 khi các kênh này được kích hoạt trong firmware và memory map.

Ở hướng nạp dữ liệu và vận hành mô hình, firmware đã hỗ trợ giao thức flash firmware từ host computer với cơ chế retry khi lỗi, timeout, NACK và kiểm tra CRC cuối. Bên cạnh firmware image, luồng nạp weights cũng đã được hiện thực để đưa dữ liệu mô hình từ host computer xuống SPI Flash, sau đó copy từ Flash sang HyperRAM khi application cần chạy accelerator. Cách tách riêng firmware image và weight blob giúp hệ thống linh hoạt hơn khi thay đổi model hoặc cập nhật bộ weights.

Ở hướng kiểm chứng application, firmware đã chạy thành công layer Conv1

của ALL-CNN-C trên accelerator với kết quả so sánh cùng CPU reference đạt 0 mismatch trên 98304 phần tử. Sau bước kiểm thử cục bộ này, application tiếp tục chạy được toàn bộ ALL-CNN-C gồm 9 layer trên accelerator, thực hiện global average pool và in profiling theo layer. Ngoài ra, firmware cũng hỗ trợ chế độ đánh giá accuracy qua UART, cho phép host computer gửi nhiều ảnh kiểm thử xuống bo mạch và nhận prediction result trả về.

Những kết quả trên cho thấy firmware không chỉ đóng vai trò chương trình kiểm thử peripheral, mà đã trở thành lớp điều phối hệ thống cho BK-AISoC. Firmware cho phép CPU RISC-V cấu hình và giám sát các hardware IP, quản lý dữ liệu trong bộ nhớ ngoài, đồng thời chuyển các tác vụ tính toán nặng sang CNN accelerator.

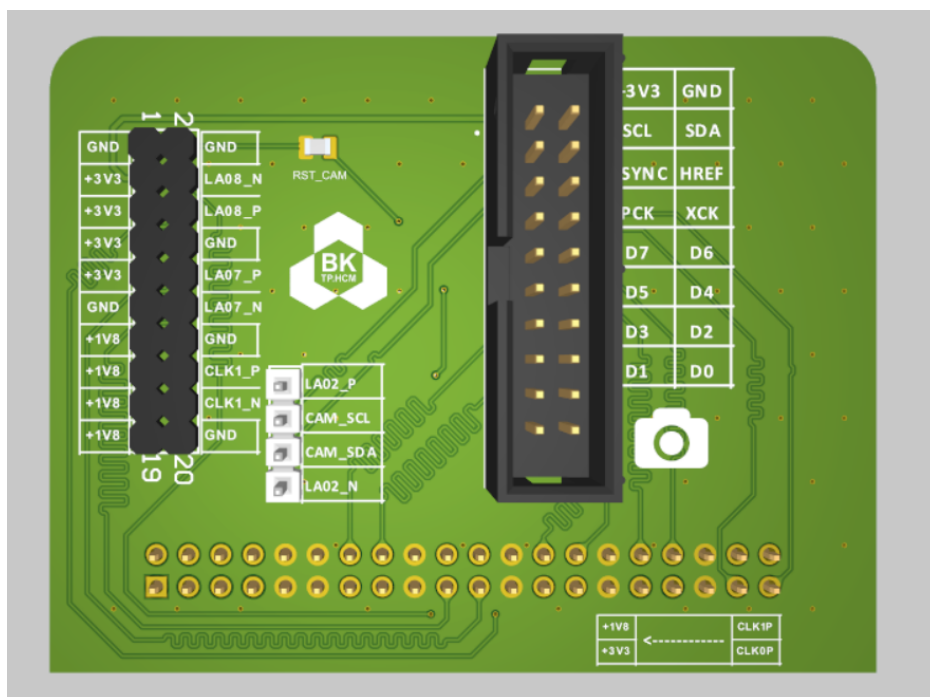
4.12 Triển khai hệ thống trên FPGA và kiểm chứng thực nghiệm

Chương này trình bày phần hiện thực hệ thống trên nền tảng phần cứng thật, bao gồm bo mạch VC707, mạch mở rộng I/O, quy trình build phần cứng trên FPGA và kết quả kiểm thử trực tiếp trên mạch. Mục tiêu của chương không chỉ là mô tả bo mạch được sử dụng, mà còn chứng minh rằng các khối phần cứng, Firmware và ngoại vi có thể phối hợp thành một hệ thống hoạt động được trên FPGA thực tế [9], [17].

4.12.1 Nền tảng phần cứng thực nghiệm

Hệ thống được triển khai trên bo mạch **Virtex-7 FPGA VC707 Evaluation Kit**. Đây là nền tảng FPGA chính dùng để nạp bitstream của SoC, kết nối với các ngoại vi bên ngoài và quan sát hoạt động thực tế của hệ thống sau khi hoàn tất Synthesis, Implementation và Bitstream Generation [17].

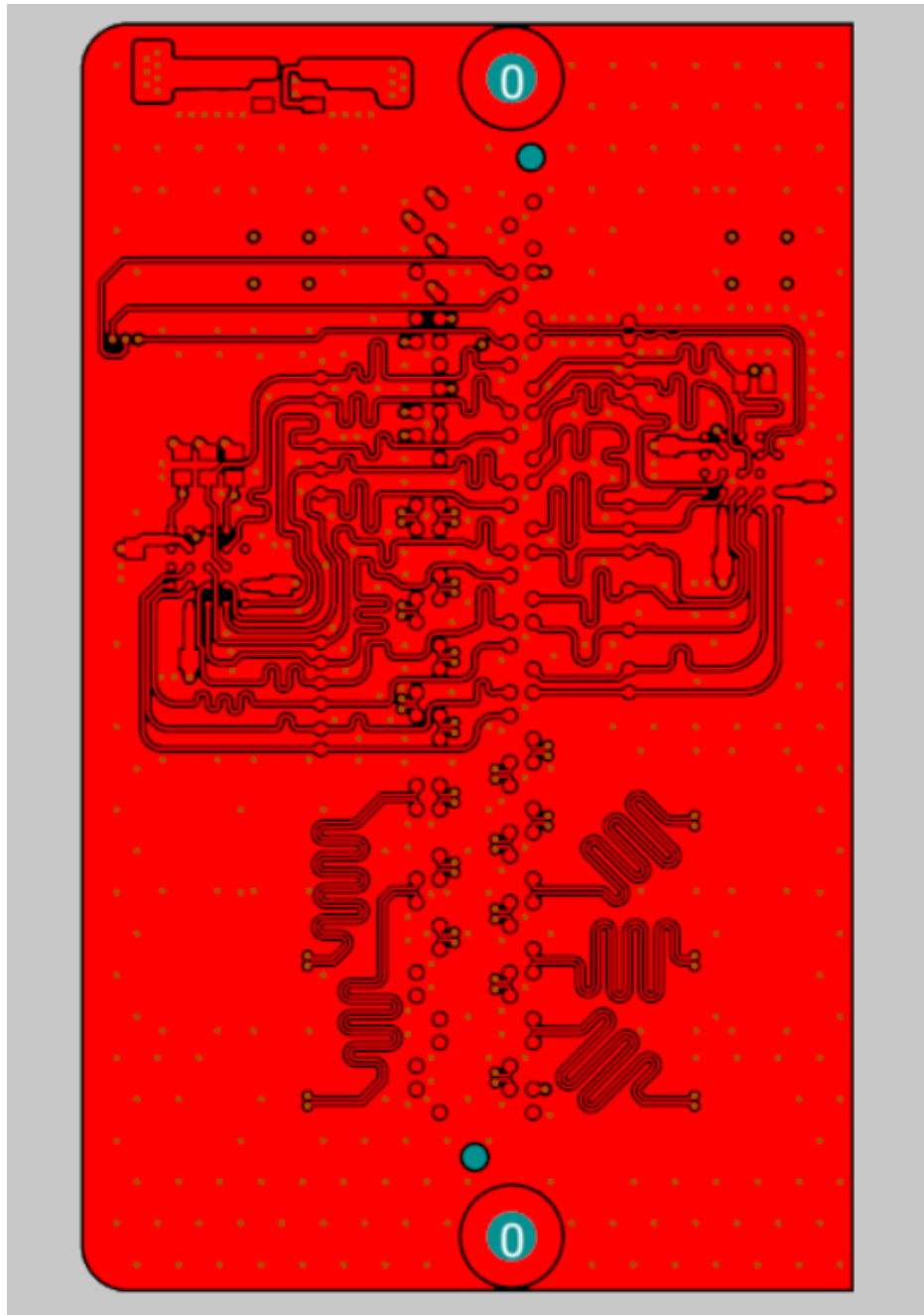
Để kết nối thuận tiện với camera, bộ nhớ ngoài và các chân tín hiệu gỡ lỗi, một bo mạch mở rộng **VC707 I/O Extension Board** được thiết kế thêm cho hệ thống. Bo mạch này đóng vai trò chuyển đổi tín hiệu từ chuẩn **FMC (FPGA Mezzanine Card)** của VC707 sang các cụm chân cắm và giao diện ngoại vi phù hợp với thiết kế SoC. Nhờ đó, các tín hiệu từ FPGA có thể được đưa ra mạch thật một cách có tổ chức, đồng thời giảm rủi ro đầu nối sai khi kiểm thử phần cứng.



Hình 4.24: Bo mạch mở rộng cho camera OV5640 với giao diện DVP

4.12.2 Thiết kế PCB và Signal Integrity

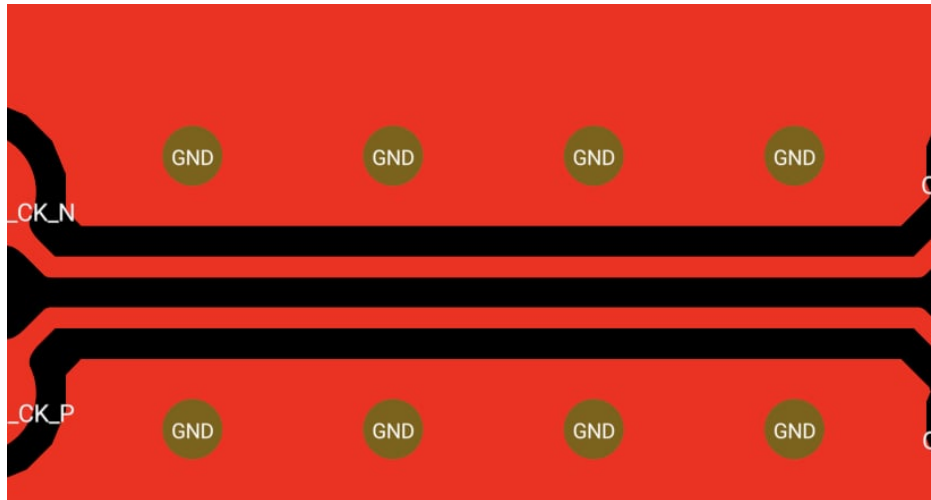
Do bo mạch mở rộng phải vận hành với các tín hiệu số tốc độ cao từ Hyper-RAM, camera và các đường Clock, quá trình thiết kế PCB cần chú ý đến **Signal Integrity** và nhiễu điện từ. Các đường tín hiệu quan trọng được định tuyến theo nguyên tắc kiểm soát trở kháng, giữ đường hồi dòng ngắn và hạn chế thay đổi đột ngột trên đường truyền.



Hình 4.25: Layout PCB thể hiện kỹ thuật Meander routing và mặt phẳng Ground

Kỹ thuật quan trọng đầu tiên là **Impedance Control** kết hợp với Stackup phù hợp. Các đường tín hiệu **Single-ended** và **Differential pair** được định tuyến theo cấu trúc **Coplanar Waveguide**, trong đó đường tín hiệu được bao bởi vùng đồng tham chiếu ở hai bên. Cách định tuyến này giúp các đường giao tiếp HyperRAM và camera duy trì trở kháng đặc tính ổn định, hạn chế Reflection khi tín hiệu chuyển mức ở tốc độ cao.

Bên cạnh đó, lớp **Ground Plane** được bố trí liền kề với lớp tín hiệu để tạo đường **Return Path** ngắn. Đối với các đường Clock, kỹ thuật **Via Shielding** được dùng dọc theo đường truyền để tăng khả năng chống nhiễu và kiểm soát EMI. Cách bố trí này đặc biệt quan trọng với các tín hiệu PCLK của camera và Clock liên quan đến bộ nhớ ngoài.



Hình 4.26: Via Shielding được áp dụng cho đường Clock nhằm kiểm soát EMI

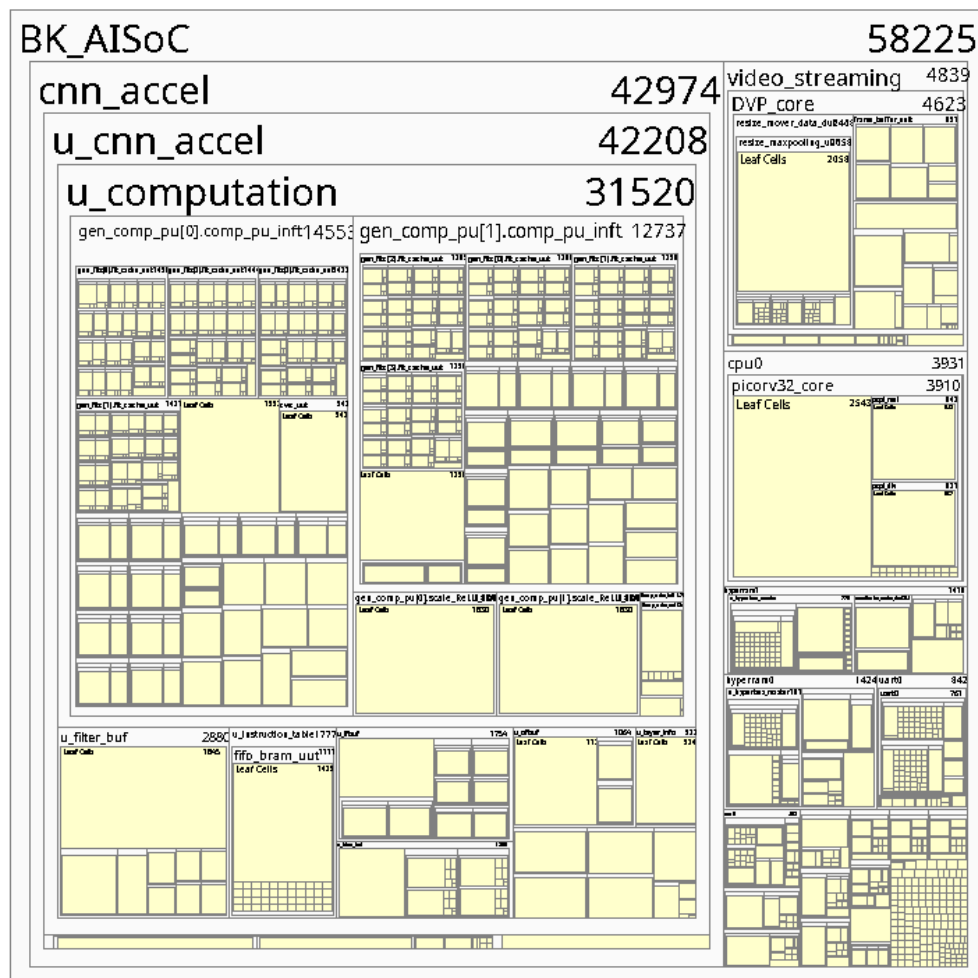
Kỹ thuật thứ hai là **Length Matching** nhằm kiểm soát Skew giữa các đường trong cùng một bus. Với các bus song song như HyperRAM hoặc camera D[7:0], sai khác chiều dài đường mạch có thể làm tín hiệu đến FPGA tại các thời điểm khác nhau, gây vi phạm Setup/Hold time. Vì vậy, các đường ngắn hơn được bù chiều dài bằng **Meander routing**, như minh họa trong Hình 4.25.

4.12.3 Quy trình build phần cứng trên FPGA

Sau khi các khối RTL của SoC, Video Streaming, bộ nhớ ngoài và accelerator được tích hợp ở mức hệ thống, thiết kế được build bằng **Xilinx Vivado 2024.2** cho FPGA **xc7vx485tffg1761-2** trên bo mạch VC707. Mục tiêu chính của bước triển khai là toàn bộ SoC phải đáp ứng Timing tại tần số hệ thống **200 MHz**, tương ứng chu kỳ Clock 5 ns . Quy trình build được chia thành hai bước chính: **Synthesis** và **Implementation**. Bước Synthesis chuyển RTL thành netlist, trong khi bước Implementation thực hiện tối ưu, ánh xạ, đặt và đi dây thiết kế lên tài

nguyên vật lý của FPGA [9], [17].

Để đạt được mục tiêu Timing 200 MHz, quá trình Implementation không chỉ dùng thiết lập mặc định của Vivado mà còn kết hợp **floorplanning thủ công** với strategy **Performance ExplorePostRoutePhysOpt**. Floorplanning được dùng để định hướng vị trí tương đối của các khối lớn như CNN accelerator, Video Streaming, CPU và các bộ nhớ, từ đó giảm độ dài đường đi dây giữa các cụm logic có giao tiếp nhiều. Strategy Performance ExplorePostRoutePhysOpt tiếp tục tối ưu sau route, hỗ trợ cải thiện các đường Timing tối hạn trước khi tạo Bitstream.



Hình 4.27: Phân bố tài nguyên theo phân cấp BK-AISoC

Các miền Clock sử dụng trong thiết kế sau Implementation được tổng hợp trong Bảng 4.3. Trong đó, Clock 200 MHz là miền Clock chính cho phần SoC,

còn các Clock phụ được dùng cho camera, video output và cấu hình ngoại vi.

Bảng 4.3: Các miền Clock được sử dụng trong thiết kế sau Implementation

Clock	Period	Frequency	Vai trò
clk_p	5.000ns	200MHz	Clock đầu vào chính
clk_out200MHz_clk_wiz_0	5.000ns	200MHz	Clock hệ thống của SoC
clk_out50MHz_clk_wiz_0	20.000ns	50MHz	Clock phụ cho ngoại vi/video
cam_pclk_i	20.000ns	50MHz	Pixel Clock từ camera
clk_out25MHz_clk_wiz_0	40.000ns	25MHz	Clock phục vụ video output
clk_out24MHz_clk_wiz_0	41.667ns	24MHz	Clock cấu hình camera
clkfbout_clk_wiz_0	5.000ns	200MHz	Feedback Clock nội bộ MMCM

Các báo cáo sau mỗi bước được dùng để kiểm tra mức sử dụng tài nguyên và đánh giá khả năng tiếp tục tạo Bitstream. Bảng 4.4 trình bày tài nguyên tổng của thiết kế **BK-AISoC** sau Synthesis và sau Implementation. Kết quả sau Implementation là số liệu cuối cùng sau khi thiết kế đã đi qua các bước tối ưu vật lý và route.

Bảng 4.4: Tổng hợp tài nguyên sau Synthesis và Implementation

Tài nguyên	Synthesis	Implementation	Khả dụng
Slice LUTs	40 764 (13.43%)	39 540 (13.02%)	303 600
LUTRAM	—	3 014 (2.30%)	130 800
Slice Registers	25 666 (4.23%)	25 744 (4.24%)	607 200
Block RAM Tile	414 (40.19%)	414 (40.19%)	1 030
DSP48E1	19 (0.68%)	19 (0.68%)	2 800
Bonded IOB	89 (12.71%)	89 (12.71%)	700
MMCM	1 (7.14%)	1 (7.14%)	14

Từ Bảng 4.4, có thể thấy lượng LUT sau Implementation giảm từ 40 764 xuống 39 540 do các bước tối ưu vật lý của Vivado. Số lượng Register tăng nhẹ

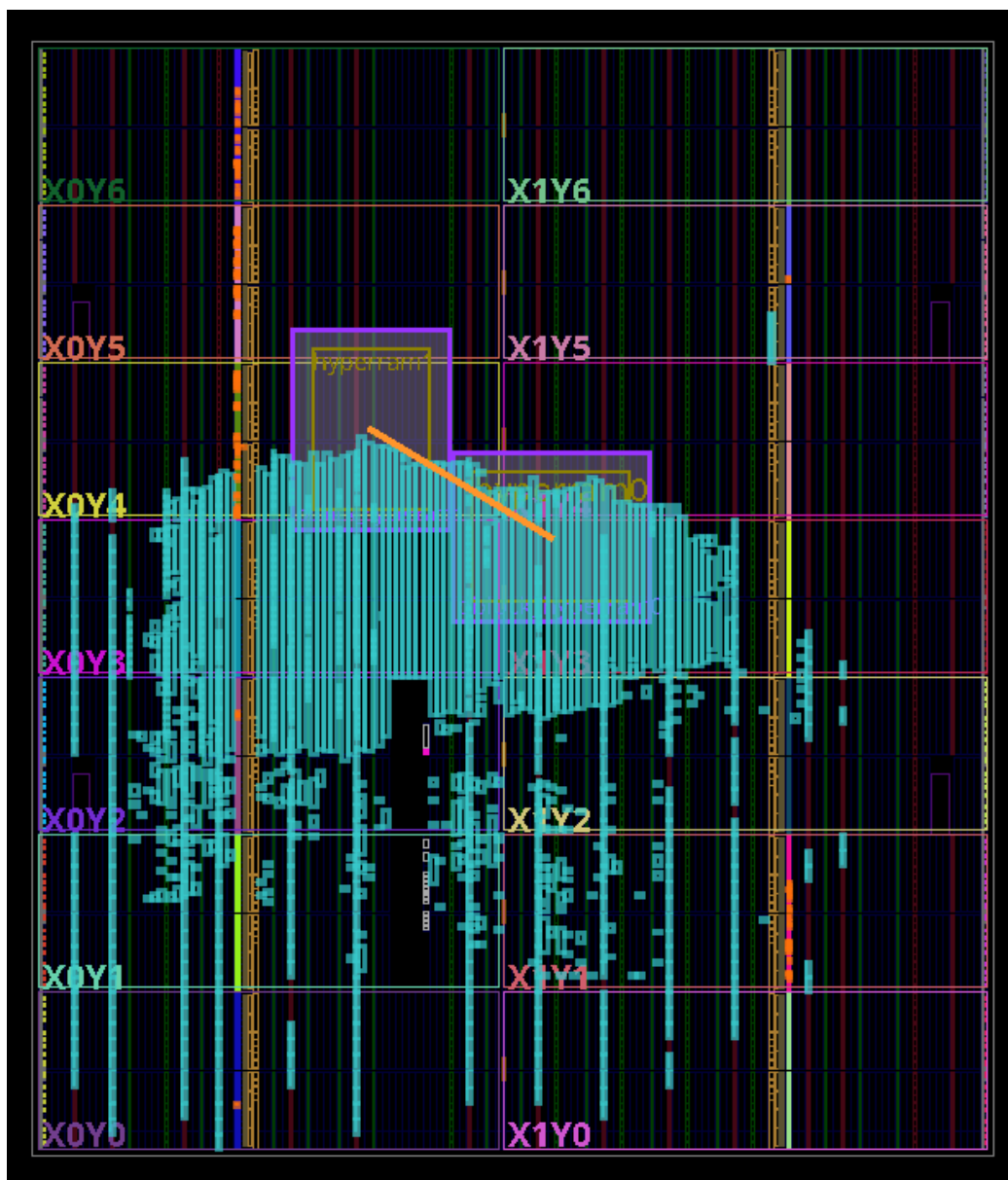
từ 25 666 lên 25 744, phản ánh các thay đổi sau tối ưu và ánh xạ vật lý. Báo cáo Synthesis không tách riêng thành phần LUTRAM, trong khi sau Implementation hệ thống sử dụng 3 014/130 800 LUTRAM, tương ứng 2.30%. Tài nguyên chiếm tỷ lệ lớn nhất vẫn là **Block RAM Tile**, với 414/1 030 tile, tương ứng 40.19%. Trong phiên bản báo cáo mới, **DSP48E1** sử dụng 19/2 800 khối, tương ứng 0.68%, cho thấy cấu hình accelerator hiện tại phụ thuộc chủ yếu vào LUT, Register và BRAM hơn là các khối DSP chuyên dụng.

Bảng 4.5 liệt kê mức sử dụng tài nguyên sau Implementation theo các IP chính trong hệ thống. Các số liệu này được dùng để xác định khối nào chiếm phần lớn tài nguyên FPGA và hỗ trợ định hướng tối ưu cho các bước phát triển tiếp theo.

Bảng 4.5: Tài nguyên sử dụng theo từng IP sau Implementation

IP / khối chức năng	Slice LUTs	Slice Registers	BRAM Tile	DSP
CNN accelerator	32 151	21 024	45	19
Video Streaming	1 439	1 527	321	0
PicoRV32 CPU	2 230	1 140	0	0
HyperRAM1 controller	691	530	0	0
HyperRAM0 controller	685	545	0	0
UART0	342	142	0	0
Timer0	86	105	0	0
I2C0	59	106	0	0
SPI0	36	101	0	0
SPI1	36	101	0	0
GPIO	31	91	0	0
IMEM	7	106	16	0
DMEM	7	106	16	0
BMEM	8	106	16	0
AXI-Lite Interconnect	35	13	0	0

Kết quả ở Bảng 4.5 cho thấy **CNN accelerator** là khối chiếm nhiều LUT, Register và toàn bộ DSP nhất trong hệ thống, với 32 151 LUT, 21 024 Register, 45 BRAM tile và 19 DSP. Đây là điều phù hợp vì accelerator chứa datapath tính toán tích chập, các bộ đệm trung gian và logic điều khiển cho quá trình xử lý mạng CNN. **Video Streaming** là khối sử dụng BRAM lớn nhất, với 321 BRAM tile, do phải lưu trữ và điều phối dữ liệu ảnh qua Frame Buffer, FIFO và các khối xử lý resize. Các ngoại vi như UART0, SPI0, SPI1, I2C0, GPIO và Timer0 chiếm lượng tài nguyên nhỏ hơn đáng kể, cho thấy phần giao tiếp điều khiển không phải là giới hạn chính của thiết kế.



Hình 4.28: Phân bố tài nguyên thực tế của BK-AISoC sau Implementation

4.12.4 Thực nghiệm trên mạch thật

Sau khi tạo Bitstream, hệ thống được nạp xuống bo mạch VC707 để kiểm thử ở mức phần cứng thật. Mô hình thực nghiệm bao gồm bo mạch VC707, VC707 I/O Extension Board, camera OV5640 và màn hình HDMI. Camera cung cấp luồng ảnh đầu vào, IP Video Streaming thực hiện thu nhận và điều phối dữ liệu ảnh, sau đó hệ thống xuất tín hiệu video ra màn hình ngoài.

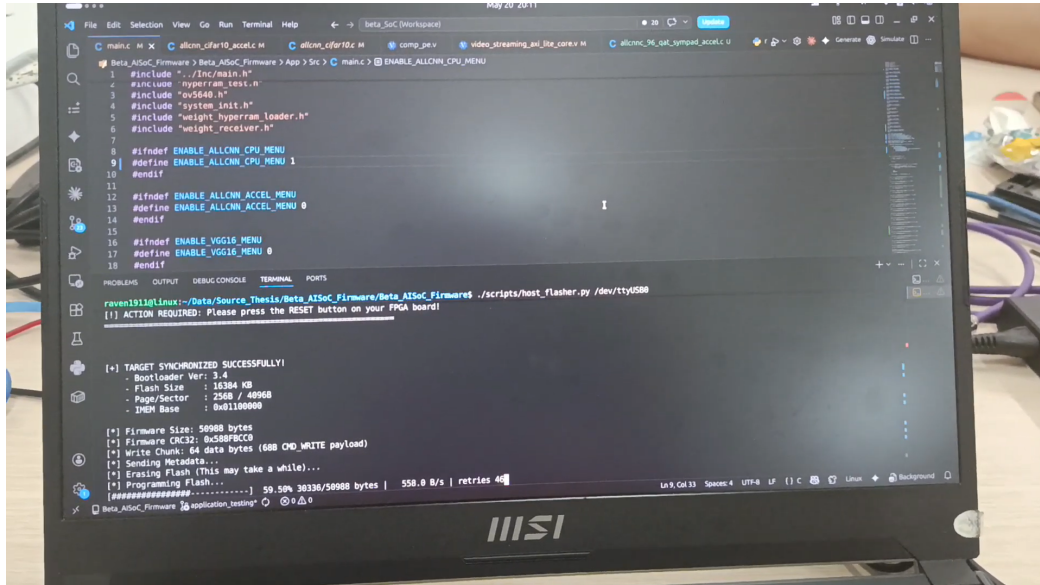


Hình 4.29: Mô hình thực nghiệm gồm VC707, mạch mở rộng I/O, camera OV5640 và HDMI

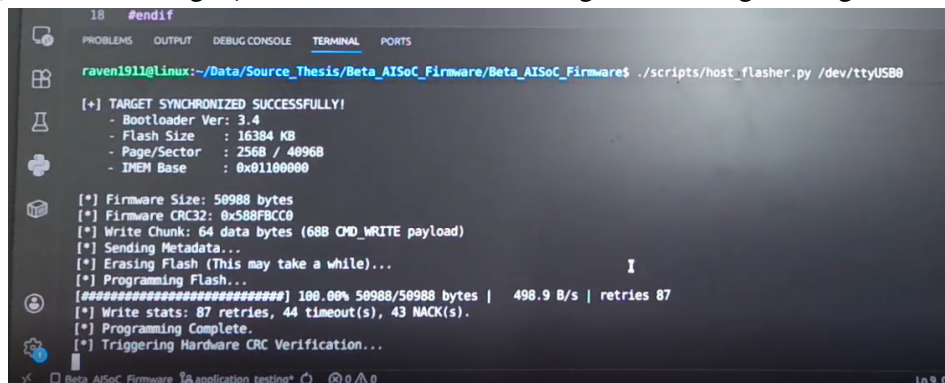
Như thể hiện trong Hình 4.29, các khối phần cứng và ngoại vi đã được kết nối thành một hệ thống hoàn chỉnh. Firmware chạy trên lõi RISC-V thực hiện các bước cấu hình cần thiết cho camera và các ngoại vi, trong khi phần cứng xử lý luồng dữ liệu ảnh theo thời gian thực. Kết quả này xác nhận rằng phần cứng SoC, Firmware và mạch mở rộng có thể phối hợp trên nền FPGA thật.

Một luồng kiểm thử khác được thực hiện với application chạy trực tiếp trên SoC sau khi nạp Bitstream. Trong luồng này, host computer giao tiếp với BK-

AISoC thông qua UART: trước hết UART bootloader nhận boot image từ laptop và ghi xuống SPI Flash, sau đó application trên SoC trao đổi dữ liệu trực tiếp với chương trình trên laptop qua cùng kênh UART. Thử nghiệm này kiểm chứng đồng thời bootloader, đường truyền UART, SPI Flash và khả năng chạy application thật trên hệ thống phần cứng.



(a) Host tool đồng bộ với bootloader và bắt đầu ghi boot image xuống SPI Flash

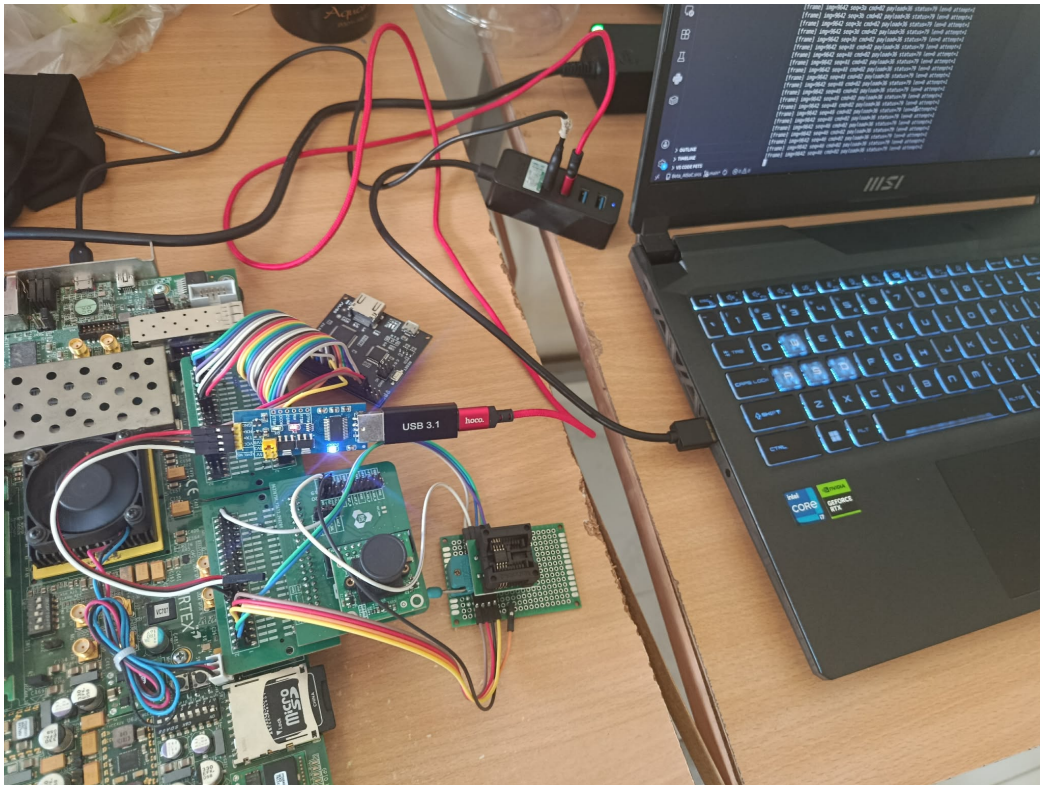


(b) Quá trình nạp hoàn tất và bootloader kích hoạt bước kiểm tra CRC phần cứng

Hình 4.30: Quá trình nạp boot image từ host computer xuống BK-AISoC thông qua UART bootloader

Như minh họa trong Hình 4.30, host tool trước hết yêu cầu người dùng reset FPGA để đưa bootloader vào trạng thái nhận firmware. Sau khi đồng bộ thành công, bootloader trả về thông tin phiên bản, kích thước Flash và vùng IMEM đích; host tiếp tục gửi metadata, xóa Flash, ghi từng chunk dữ liệu và kiểm tra

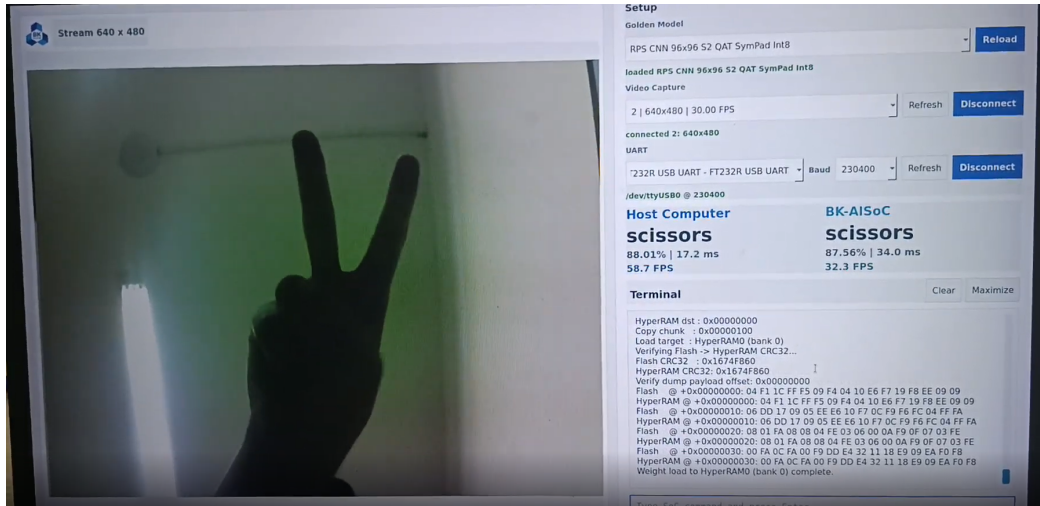
CRC. Kết quả nạp thành công cho thấy giao thức UART bootloader có thể dùng để cập nhật application mà không cần build lại Bitstream.



Hình 4.31: Thực nghiệm chạy application thật với UART bootloader và truyền dữ liệu trực tiếp giữa laptop và BK-AISoC

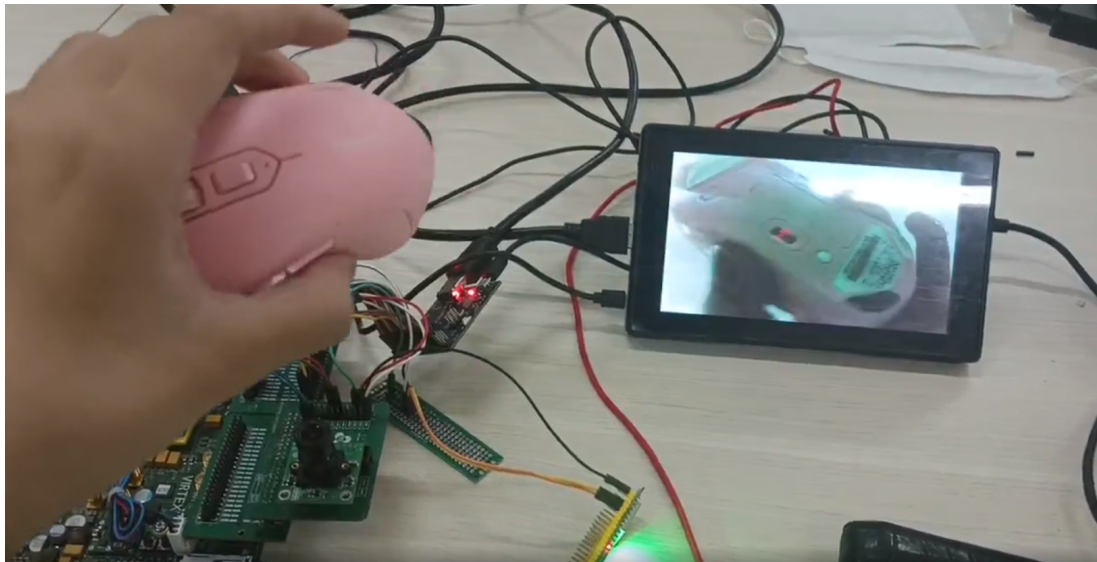


Hình 4.32: Hệ thống BK-AISoC chạy thực tế với camera, màn hình hiển thị và host application



Hình 4.33: Kết quả host application nhận luồng ảnh và so sánh dự đoán giữa host computer và BK-AISoC

Hình 4.32 và Hình 4.33 ghi lại trạng thái chạy thực tế sau khi firmware đã được nạp. Luồng camera được hiển thị trên màn hình ngoài, đồng thời host application nhận dữ liệu ảnh, gửi lệnh qua UART và hiển thị kết quả phân loại. Việc kết quả phía host computer và phía BK-AISoC cùng nhận diện lớp *scissors* cho thấy pipeline phần mềm–phần cứng đã vận hành được từ camera, firmware điều phối, accelerator cho đến giao diện quan sát trên host.



Hình 4.34: Kết quả xuất tín hiệu video theo thời gian thực lên màn hình

Kết quả hiển thị trong Hình 4.34 cho thấy luồng video từ camera được thu nhận và xuất ra màn hình thành công. Hình ảnh hiển thị ổn định, không xuất

hiện lỗi mất đồng bộ rõ rệt trong quá trình quan sát. Điều này chứng minh rằng các thành phần chính của hệ thống, bao gồm Clock Domain Crossing, bộ đệm khung hình, đường xuất HDMI và cấu hình camera, đã hoạt động đúng trên phần cứng thật.

Chương 5

Đánh giá

Chương này tổng hợp và phân tích các kết quả đánh giá của CNN accelerator ở hai mức: mô phỏng RTL và chạy thực tế trên board VC707. Phần mô phỏng được sử dụng để kiểm chứng hoạt động của kiến trúc ở mức tín hiệu, tính toán các chỉ số MACs, ConvOps và GOPS, đồng thời đối chiếu với mô hình thời gian đã xây dựng. Phần thực nghiệm trên board tiếp tục đánh giá mức sai lệch giữa mô hình hóa và triển khai thực tế đối với VGG16, sau đó phân tích hiệu quả tăng tốc khi chạy ALL CNN C bằng CPU kết hợp CNN accelerator [5], [6]. Cách trình bày này giúp liên kết trực tiếp giữa kết quả lý thuyết, kết quả mô phỏng và kết quả thực tế, từ đó chỉ ra rõ các thành phần overhead còn ảnh hưởng đến hiệu năng hệ thống.

Phần đầu của chương tập trung vào kết quả mô phỏng RTL của khối accelerator trên hai lớp tích chập đại diện: lớp tích chập đầu của VGG16 và lớp Conv1 của ALL CNN C. Mục tiêu của phép mô phỏng là kiểm tra tính đúng đắn về trình tự điều khiển sau khi hiện thực RTL, đồng thời đối chiếu thời gian thực thi thu được từ waveform với mô hình thời gian đã xây dựng trước đó. Khác với phép đo trên FPGA, thời điểm bắt đầu và kết thúc trong mô phỏng được xác định trực tiếp từ các tín hiệu điều khiển, do đó kết quả phản ánh tương đối rõ phần thời gian phát sinh từ FSM, truyền dữ liệu và đồng bộ giữa các khối.

5.1 Thiết lập đánh giá

Trong quá trình đánh giá, số lượng phép tính của một lớp tích chập được xác định dựa trên số phép nhân–cộng tích lũy cần thiết để tạo ra toàn bộ output feature map. Đây là cách quy đổi thường dùng khi đánh giá khối lượng tính toán của các lớp convolution trong accelerator CNN [3]. Với một lớp tích chập chuẩn, số phép MAC được tính theo công thức:

$$MACs = W_o \times H_o \times C_o \times K_w \times K_h \times C_i \quad (5.1)$$

trong đó W_o và H_o lần lượt là chiều rộng và chiều cao của output feature map, C_o là số kênh đầu ra, K_w và K_h là kích thước kernel, còn C_i là số kênh đầu vào. Do mỗi phép MAC bao gồm một phép nhân và một phép cộng, tổng số phép toán tích chập được quy đổi như sau:

$$ConvOps = 2 \times MACs \quad (5.2)$$

Hiệu năng tính toán của accelerator được đánh giá thông qua chỉ số GOPS:

$$GOPS = \frac{ConvOps}{t \times 10^9} \quad (5.3)$$

trong đó t là thời gian thực thi của lớp tích chập tính theo giây. Công thức này cho phép đánh giá số lượng phép toán mà accelerator có thể xử lý trong một giây, từ đó phản ánh hiệu quả khai thác phần cứng trong từng cấu hình lớp khác nhau.

5.2 Phân tích kết quả mô phỏng

5.2.1 Kết quả mô phỏng lớp tích chập đầu của VGG16

Trường hợp đầu tiên được đánh giá là lớp tích chập đầu của VGG16 [5]. Lớp này có kích thước đầu vào $224 \times 224 \times 3$, kernel 3×3 , stride bằng 1, padding bằng 1 và số kênh đầu ra là 64. Do padding được sử dụng để giữ nguyên kích thước không gian, output feature map có kích thước $224 \times 224 \times 64$.

Số lượng phép MAC của lớp này được tính như sau:

$$MACs_{VGG16} = 224 \times 224 \times 64 \times 3 \times 3 \times 3 \quad (5.4)$$

$$MACs_{VGG16} = 86,704,128 \quad (5.5)$$

Tổng số phép toán tích chập tương ứng là:

$$ConvOps_{VGG16} = 2 \times 86,704,128 = 173,408,256 \quad (5.6)$$

Kết quả waveform mô phỏng được trình bày trong Hình 5.1. Từ waveform, thời gian thực thi của lớp VGG16 được xác định xấp xỉ $25.74633ms$. Đây là trường hợp có thời gian chạy lớn hơn trong hai lớp được khảo sát, do kích thước feature map đầu vào lớn và tổng số phép MAC cao.



Hình 5.1: Waveform mô phỏng lớp tích chập đầu của VGG16

Dựa trên thời gian mô phỏng thu được, hiệu năng GOPS của lớp VGG16 được tính như sau:

$$GOPS_{VGG16} = \frac{173,408,256}{0.02574633 \times 10^9} \quad (5.7)$$

$$GOPS_{VGG16} \approx 6.7353 \quad (5.8)$$

Như vậy, hiệu năng tính toán của accelerator đối với lớp VGG16 đạt xấp xỉ:

$$GOPS_{VGG16} \approx 6.74 \quad (5.9)$$

Kết quả này cho thấy với lớp có khối lượng tính toán lớn, accelerator có khả năng khai thác tốt mức song song của kiến trúc. Do số lượng phép MAC lớn, phần thời gian dành cho tính toán chiếm tỷ trọng đáng kể trong tổng thời gian thực thi. Vì vậy, các chi phí cố định như khởi tạo, cấp dữ liệu và đồng bộ tín hiệu ít làm suy giảm hiệu năng trung bình hơn so với các lớp nhỏ.

5.2.2 Kết quả mô phỏng lớp Conv1 của ALL CNN C

Trường hợp thứ hai là lớp Conv1 của ALL CNN C [6]. Lớp này có kích thước đầu vào $32 \times 32 \times 3$, kernel 3×3 , stride bằng 1, padding bằng 1 và số kênh đầu ra là 96. Output feature map tương ứng có kích thước $32 \times 32 \times 96$.

Số lượng phép MAC của lớp này được tính như sau:

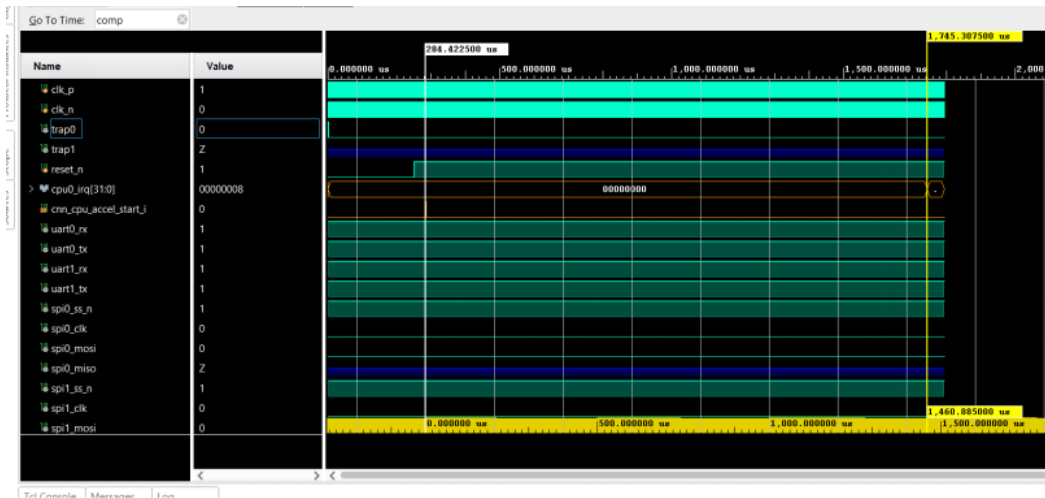
$$MACs_{ALLCNN} = 32 \times 32 \times 96 \times 3 \times 3 \times 3 \quad (5.10)$$

$$MACs_{ALLCNN} = 2,654,208 \quad (5.11)$$

Tổng số phép toán tích chập tương ứng là:

$$ConvOps_{ALLCNN} = 2 \times 2,654,208 = 5,308,416 \quad (5.12)$$

Kết quả waveform mô phỏng của lớp Conv1 được thể hiện trong Hình 5.2. Thời gian thực thi đo được từ waveform xấp xỉ 1.460885ms . Mặc dù lớp này có số phép toán nhỏ hơn đáng kể so với VGG16, thời gian thực thi không giảm theo cùng tỷ lệ với số lượng MAC. Điều này cho thấy chi phí cố định của quá trình điều khiển, nạp dữ liệu và giao tiếp bộ nhớ có ảnh hưởng rõ rệt hơn ở các lớp có kích thước nhỏ.



Hình 5.2: Waveform mô phỏng lớp Conv1 của ALL CNN C

Hiệu năng GOPS của lớp ALL CNN C Conv1 được tính như sau:

$$GOPS_{ALLCNN} = \frac{5,308,416}{0.001460885 \times 10^9} \quad (5.13)$$

$$GOPS_{ALLCNN} \approx 3.6337 \quad (5.14)$$

So với VGG16, hiệu năng GOPS của lớp ALL CNN C Conv1 thấp hơn. Nguyên nhân chính không đến từ sai khác chức năng của accelerator, mà đến từ tỷ lệ giữa thời gian tính toán hữu ích và thời gian phụ trợ. Với lớp nhỏ, số lượng phép MAC chỉ đạt 2,654,208, thấp hơn nhiều so với 86,704,128 MAC của lớp VGG16. Trong khi đó, các thao tác như nạp cấu hình, truyền dữ liệu, đồng bộ luồng tính toán và ghi kết quả vẫn cần một khoảng thời gian gần như cố định. Do đó, phần overhead chiếm tỷ trọng lớn hơn, làm giảm GOPS trung bình.

5.2.3 So sánh với thời gian mô hình hóa

Bên cạnh việc đánh giá hiệu năng thông qua chỉ số GOPS, kết quả simulation còn được dùng để kiểm chứng mức độ phù hợp của mô hình thời gian đã xây dựng ở mục trước. Trong mô hình hóa, thời gian xử lý của một lớp tích chập được ước lượng từ các thành phần chính gồm thời gian nạp dữ liệu, thời gian tính toán và thời gian ghi kết quả. Tổng thời gian xử lý một lớp được xây dựng dựa trên thời gian của từng pass, từng tile và từng cụm bộ lọc. Do đó, mô hình này phản ánh được xu hướng thay đổi thời gian khi kích thước feature map, số kênh đầu vào, số kênh đầu ra hoặc kích thước kernel thay đổi.

Tuy nhiên, mô hình hóa không bao gồm toàn bộ các chi phí phụ trợ xuất hiện trong simulation RTL. Cụ thể, mô hình chủ yếu xét thời gian tải, nạp và tính toán dữ liệu, trong khi simulation còn bao gồm thời gian nhận lệnh, khởi tạo trạng thái điều khiển, đồng bộ giữa các FSM, latency của giao tiếp bộ nhớ, cũng như các chu kỳ chờ phát sinh khi các tín hiệu điều khiển chưa sẵn sàng. Vì vậy, thời gian simulation thường lớn hơn thời gian mô hình hóa, đặc biệt đối với các lớp

có kích thước nhỏ.

Đối với lớp tích chập đầu của VGG16, thời gian mô hình hóa thu được là:

$$t_{model,VGG16} = 22.0774ms \quad (5.15)$$

Trong khi đó, thời gian đo được từ simulation là:

$$t_{sim,VGG16} = 25.74633ms \quad (5.16)$$

Sai lệch tuyệt đối giữa simulation và mô hình hóa được tính như sau:

$$\begin{aligned} \Delta t_{VGG16} &= t_{sim,VGG16} - t_{model,VGG16} \\ &= 25.74633 - 22.0774 \\ &= 3.66893ms \end{aligned} \quad (5.17)$$

Sai lệch tương đối là:

$$\begin{aligned} Error_{VGG16} &= \frac{t_{sim,VGG16} - t_{model,VGG16}}{t_{model,VGG16}} \times 100\% \\ &= \frac{25.74633 - 22.0774}{22.0774} \times 100\% \\ &\approx 16.62\% \end{aligned} \quad (5.18)$$

Tỷ lệ giữa thời gian simulation và thời gian mô hình hóa là:

$$\begin{aligned} Ratio_{VGG16} &= \frac{t_{sim,VGG16}}{t_{model,VGG16}} \\ &= \frac{25.74633}{22.0774} \\ &\approx 1.1662 \end{aligned} \quad (5.19)$$

Kết quả này cho thấy đối với lớp VGG16, thời gian simulation chỉ lớn hơn thời gian mô hình hóa khoảng 16.62%. Đây là mức sai lệch tương đối chấp nhận được trong giai đoạn ước lượng kiến trúc, vì lớp VGG16 có kích thước feature map lớn và tổng số phép toán cao. Khi khối lượng tính toán đủ lớn, các chi phí cố định như nhận lệnh, khởi tạo FSM và chờ giao tiếp bộ nhớ chỉ chiếm một

phần nhỏ trong tổng thời gian thực thi. Do đó, mô hình hóa vẫn dự đoán được tương đối tốt thời gian xử lý của accelerator đối với lớp có kích thước lớn.

Đối với lớp Conv1 của ALL CNN C, thời gian mô hình hóa là:

$$t_{model,ALLCNN} = 0.67584ms \quad (5.20)$$

Trong khi đó, thời gian đo được từ simulation là:

$$t_{sim,ALLCNN} = 1.460885ms \quad (5.21)$$

Sai lệch tuyệt đối được tính như sau:

$$\begin{aligned} \Delta t_{ALLCNN} &= t_{sim,ALLCNN} - t_{model,ALLCNN} \\ &= 1.460885 - 0.67584 \\ &= 0.785045ms \end{aligned} \quad (5.22)$$

Sai lệch tương đối là:

$$\begin{aligned} Error_{ALLCNN} &= \frac{t_{sim,ALLCNN} - t_{model,ALLCNN}}{t_{model,ALLCNN}} \times 100\% \\ &= \frac{1.460885 - 0.67584}{0.67584} \times 100\% \\ &\approx 116.16\% \end{aligned} \quad (5.23)$$

Tỷ lệ giữa thời gian simulation và thời gian mô hình hóa là:

$$\begin{aligned} Ratio_{ALLCNN} &= \frac{t_{sim,ALLCNN}}{t_{model,ALLCNN}} \\ &= \frac{1.460885}{0.67584} \\ &\approx 2.1616 \end{aligned} \quad (5.24)$$

Kết quả trên cho thấy đối với lớp ALL CNN C Conv1, thời gian simulation lớn hơn khoảng 2.16 lần so với thời gian mô hình hóa. Sự chênh lệch này xuất phát từ việc lớp Conv1 của ALL CNN C có kích thước nhỏ hơn đáng kể so với VGG16. Tổng số phép MAC của lớp này chỉ là 2,654,208, trong khi lớp

VGG16 cần 86,704,128 **MAC**. Vì vậy, phần thời gian tính toán hữu ích giảm mạnh, nhưng các chi phí cố định như nhận lệnh, khởi động điều khiển, đồng bộ dữ liệu và giao tiếp bộ nhớ không giảm theo cùng tỷ lệ. Khi đó, overhead chiếm tỷ trọng lớn trong tổng thời gian simulation, làm cho sai lệch so với mô hình hóa tăng lên rõ rệt.

Từ hai trường hợp trên, có thể rút ra rằng mô hình thời gian đã xây dựng có khả năng ước lượng tốt xu hướng thực thi của accelerator, đặc biệt đối với các lớp có khối lượng tính toán lớn. Mô hình phản ánh đúng rằng lớp VGG16 cần thời gian xử lý lớn hơn lớp ALL CNN C Conv1 do có số lượng phép MAC cao hơn. Tuy nhiên, mô hình hiện tại vẫn mang tính lý tưởng vì chưa xét đầy đủ các thành phần overhead của hệ thống điều khiển và giao tiếp. Do đó, để cải thiện độ chính xác trong các bước tiếp theo, mô hình thời gian có thể được mở rộng bằng cách bổ sung một thành phần chi phí cố định:

$$t_{sim} \approx t_{model} + t_{overhead} \quad (5.25)$$

Trong đó, $t_{overhead}$ biểu diễn các chi phí không trực tiếp thuộc về quá trình tính toán MAC, bao gồm thời gian nhận lệnh, khởi tạo trạng thái, đồng bộ giữa các khối, chờ giao tiếp bộ nhớ và ghi nhận tín hiệu hoàn tất. Với các lớp lớn, $t_{overhead}$ nhỏ so với t_{model} nên sai lệch tương đối thấp. Ngược lại, với các lớp nhỏ, $t_{overhead}$ có thể cùng bậc với t_{model} , dẫn đến sai lệch tương đối cao hơn.

Bảng 5.1 tổng hợp kết quả so sánh giữa thời gian mô hình hóa và thời gian simulation.

Bảng 5.1: So sánh thời gian mô hình hóa và thời gian simulation

Lớp mô phỏng	t_{model}	t_{sim}	Sai lệch tuyệt đối	Sai lệch tương đối
VGG16 Conv1	22.0774 ms	25.74633 ms	3.66893 ms	16.62%
ALL CNN C Conv1	0.67584 ms	1.460885 ms	0.785045 ms	116.16%

Nhìn chung, kết quả so sánh cho thấy mô hình hóa không nhằm thay thế hoàn toàn simulation RTL, mà đóng vai trò là công cụ ước lượng nhanh ở giai đoạn thiết kế kiến trúc. Mô hình giúp dự đoán xu hướng thay đổi thời gian thực thi

khi thay đổi kích thước lớp, trong khi simulation phản ánh đầy đủ hơn hành vi thực tế của phần cứng sau khi bổ sung logic điều khiển, giao tiếp và đồng bộ. Vì vậy, việc kết hợp cả hai phương pháp cho phép đánh giá accelerator một cách chặt chẽ hơn: mô hình hóa dùng để định hướng thiết kế và lựa chọn tham số kiến trúc, còn simulation dùng để kiểm chứng độ chính xác và chỉ ra các thành phần overhead cần tiếp tục tối ưu.

5.2.4 Đánh giá tổng hợp kết quả mô phỏng RTL

Bảng 5.2 tổng hợp kết quả mô phỏng của hai lớp tích chập được khảo sát.

Bảng 5.2: Tổng hợp kết quả mô phỏng RTL trên hai lớp tích chập

Lớp mô phỏng	Kích thước đầu vào	MACs	Thời gian mô phỏng	GOPS
VGG16 Conv1	$224 \times 224 \times 3$	86,704,128	25.74633 ms	6.74
ALL CNN C Conv1	$32 \times 32 \times 3$	2,654,208	1.460885 ms	3.6337

Từ bảng kết quả có thể thấy lớp VGG16 có thời gian mô phỏng lớn hơn do khối lượng tính toán lớn hơn đáng kể. Tuy nhiên, chính vì có số lượng phép toán lớn, lớp này đạt hiệu năng GOPS cao hơn. Ngược lại, ALL CNN C Conv1 có thời gian thực thi ngắn hơn về giá trị tuyệt đối, nhưng hiệu năng GOPS thấp hơn vì overhead của quá trình giao tiếp và điều khiển chiếm tỷ trọng lớn hơn trong tổng thời gian.

Kết quả trên cho thấy hiệu năng của accelerator không chỉ phụ thuộc vào số lượng phần tử tính toán hay số lượng phép MAC, mà còn phụ thuộc mạnh vào khả năng duy trì luồng dữ liệu liên tục cho khối computation. Đối với các lớp có kích thước lớn, kiến trúc có thể khai thác tốt hơn mức song song và tái sử dụng dữ liệu, từ đó đạt hiệu năng trung bình cao hơn. Đối với các lớp nhỏ, cần tiếp tục tối ưu cơ chế khởi tạo, nạp dữ liệu và điều phối bộ nhớ để giảm overhead, đặc biệt trong trường hợp accelerator phải xử lý nhiều lớp nhỏ liên tiếp.

5.3 Phân tích kết quả thực tế VGG16 trên board VC707

5.3.1 Kết quả thực nghiệm và mô hình hoá thời gian chạy VGG16

Nội dung trong mục này trình bày kết quả đo thời gian thực tế khi chạy các lớp tích chập của mô hình VGG16 trên board VC707, đồng thời đối chiếu với kết quả thời gian được ước lượng từ mô hình hoá trước đó [5], [17]. Mục tiêu của phép so sánh không chỉ là kiểm tra sai lệch giữa mô hình lý thuyết và triển khai thực tế, mà còn nhằm xác định các thành phần gây chênh lệch thời gian, đặc biệt là độ trễ giao tiếp giữa DMA, bộ nhớ ngoài và các buffer dữ liệu của accelerator.

Trong thí nghiệm, toàn bộ các lớp convolution của VGG16 được thực thi trên accelerator. Thời gian thực tế được ghi nhận theo số chu kỳ xung clock và quy đổi sang mili giây. Bảng 5.3 tổng hợp kết quả chạy thực tế trên board đối với 13 lớp convolution của VGG16.

Bảng 5.3: Thời gian chạy thực tế của các lớp convolution trong VGG16 trên board VC707

Layer	Kích thước vào	Kích thước ra	Cycles thực tế	Time thực tế (ms)	Time ước lượng (ms)	N_{ip}	N_{fp}	N_{pass}	Workload (MACs)	Cost (cycles/1000 MACs)
Conv1_1	$224 \times 224 \times 3$	$224 \times 224 \times 64$	5,251,595	26	22.0774	2	8	1	86,704,128	61
Conv1_2	$224 \times 224 \times 64$	$224 \times 224 \times 64$	34,242,170	171	144.507	4	8	1	1,849,688,064	19
Conv2_1	$112 \times 112 \times 64$	$112 \times 112 \times 128$	19,791,230	98	72.2534	4	8	1	924,844,032	21
Conv2_2	$112 \times 112 \times 128$	$112 \times 112 \times 128$	37,395,905	186	136.479	4	8	1	1,849,688,064	20
Conv3_1	$56 \times 56 \times 128$	$56 \times 56 \times 256$	23,747,810	118	68.4954	4	8	1	924,844,032	26
Conv3_2	$56 \times 56 \times 256$	$56 \times 56 \times 256$	46,112,045	230	133.048	4	8	1	1,849,688,064	25
Conv3_3	$56 \times 56 \times 256$	$56 \times 56 \times 256$	46,112,045	230	133.048	4	8	1	1,849,688,064	25
Conv4_1	$28 \times 28 \times 256$	$28 \times 28 \times 512$	32,978,300	164	70.0416	4	8	1	924,844,032	36
Conv4_2	$28 \times 28 \times 512$	$28 \times 28 \times 512$	86,880,695	434	180.563	3	6	1	1,849,688,064	47
Conv4_3	$28 \times 28 \times 512$	$28 \times 28 \times 512$	86,880,695	434	180.563	3	6	1	1,849,688,064	47
Conv5_1	$14 \times 14 \times 512$	$14 \times 14 \times 512$	34,914,065	174	52.5811	4	6	1	462,422,016	76
Conv5_2	$14 \times 14 \times 512$	$14 \times 14 \times 512$	34,914,065	174	52.5811	4	6	1	462,422,016	76
Conv5_3	$14 \times 14 \times 512$	$14 \times 14 \times 512$	34,914,065	174	52.5811	4	6	1	462,422,016	76
Tổng	–	–	524,134,685	2,613	1,298.8191	–	–	–	–	–

Kết quả thực nghiệm cho thấy tổng thời gian chạy của 13 lớp convolution là 524,134,685 **chu kỳ**, tương ứng xấp xỉ 2,613 **ms** nếu lấy theo thời gian quy đổi từ log chạy thực tế. Các lớp có khối lượng tính toán lớn như Conv4_2 và Conv4_3 đạt thời gian 434ms cho mỗi lớp, cao hơn đáng kể so với các lớp đầu mạng. Tuy nhiên, khi xét chỉ số chi phí trên mỗi 1000 phép MAC, có thể nhận thấy hiệu quả không chỉ phụ thuộc vào tổng số phép MAC, mà còn bị ảnh hưởng mạnh bởi kích thước feature map, số kênh, mức song song thực tế và chi phí nạp/xả dữ liệu qua bộ nhớ.

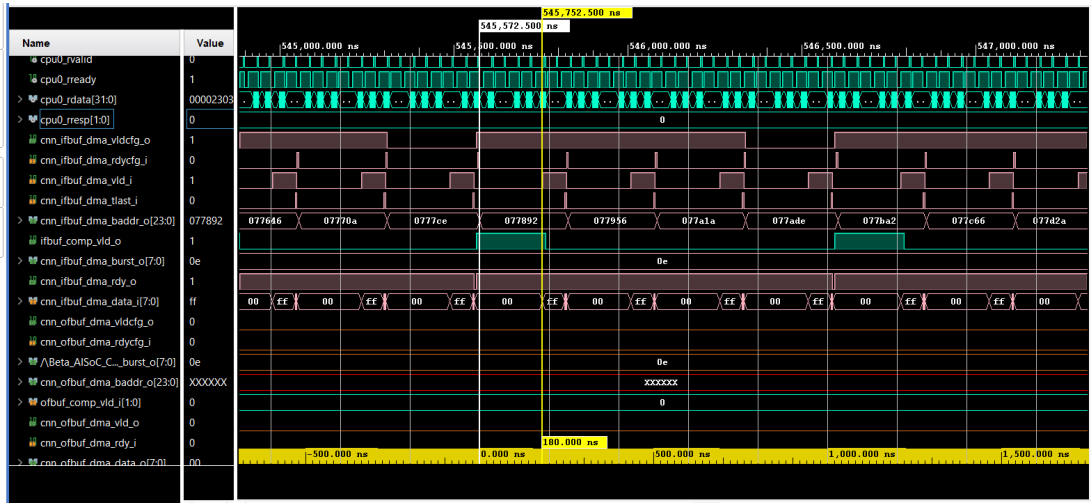
Tổng thời gian theo mô hình ước lượng là khoảng 1,298.8191 **ms**, trong khi tổng thời gian thực tế đo được là 2,613 **ms**. Như vậy, thời gian thực tế lớn hơn mô hình ước lượng khoảng 1,314.1809 **ms**, tương đương tăng khoảng 101.18%. Sai lệch này là hợp lý vì mô hình ước lượng chủ yếu phản ánh thời gian tính toán và thời gian truyền dữ liệu lý tưởng, trong khi kết quả thực tế còn bao gồm độ trễ cấu hình DMA, độ trễ truy cập bộ nhớ ngoài, thời gian chờ giữa các burst và các chi phí điều phối hệ thống trên board.

Một thông tin quan trọng trong quá trình phân tích là số lần accelerator yêu

cầu đọc dữ liệu từ *input feature buffer* tại một layer. Với ký hiệu C_o là số kênh đầu ra, H_i là chiều cao feature map đầu vào, C_i là số kênh đầu vào, $N_{of tile}$ là số tile filter đầu ra và $N_{of parr}$ là số filter đầu ra được xử lý song song, số lần yêu cầu đọc *ifbuf* có thể được mô tả bởi công thức

$$N_{read,ifbuf} = \left\lceil \frac{C_o}{N_{of tile} \times N_{of parr}} \right\rceil \times H_i \times C_i. \quad (5.26)$$

Theo kết quả mô phỏng, thời gian từ lúc DMA nhận cấu hình đến khi dữ liệu đầu tiên được đưa vào accelerator là 36 **chu kỳ**, tương ứng khoảng 180ns. Ở giai đoạn này, đường truyền được dùng chung cho *ifbuf* và *ofbuf*; tuy nhiên, *ofbuf* chưa sử dụng đường truyền nên chi phí chờ chủ yếu xuất hiện ở phía *ifbuf*. Cần lưu ý rằng đây là số liệu thu được trong môi trường mô phỏng, do đó khi chạy trên phần cứng thực tế, độ trễ có thể còn lớn hơn do ảnh hưởng của giao tiếp với bộ nhớ ngoài và cơ chế điều phối DMA.



Hình 5.3: Thời gian chờ DMA cấu hình memory

Để phân tích sâu hơn nguyên nhân sai lệch, nhóm chọn hai lớp đại diện của VGG16. Lớp Conv1_2 đại diện cho nhóm layer đầu mạng, nơi feature map có kích thước không gian lớn. Lớp Conv5_1 đại diện cho nhóm layer gần cuối mạng, nơi kích thước không gian nhỏ hơn nhưng số kênh đầu vào và đầu ra lớn, dẫn đến số lần cấu hình và đọc dữ liệu có đặc điểm khác biệt.

5.3.1.1 Phân tích layer Conv1_2

Layer Conv1_2 có kích thước đầu vào $W_i = 224$, $H_i = 224$, $C_i = 64$, kernel 3×3 , stride bằng 1, padding bằng 1, và đầu ra có kích thước $W_o = 224$, $H_o = 224$, $C_o = 64$. Với cấu hình $N_{of tile} = 1$ và $N_{of parr} = 8$, số lần accelerator cần đọc dữ liệu ở layer này là

$$N_{read,ifbuf} = \left\lceil \frac{64}{1 \times 8} \right\rceil \times 224 \times 64 = 114,688. \quad (5.27)$$

Nếu nhân với thời gian chờ 180 ns cho mỗi lần DMA nhận cấu hình đến khi dữ liệu đầu tiên được trả về, tổng thời gian chờ do giao tiếp *ifbuf* gây ra có thể ước lượng là

$$T_{wait,Conv1_2} = 114,688 \times 180 \text{ ns} \approx 20.64 \text{ ms}. \quad (5.28)$$

Thời gian thực tế đo được của Conv1_2 là 171 ms, trong khi thời gian ước lượng từ mô hình hoá là 144.507 ms. Độ chênh lệch giữa hai giá trị này xấp xỉ 26.493 ms. Con số này tương đối gần với phần thời gian chờ 20.64 ms được tính từ độ trễ DMA trong mô phỏng. Điều này cho thấy sai lệch giữa mô hình và thực nghiệm không xuất phát chủ yếu từ khối tính toán MAC, mà nhiều khả năng đến từ chi phí giao tiếp bộ nhớ và độ trễ khi nạp dữ liệu vào accelerator.

Xét theo góc nhìn burst dữ liệu, với $W_i = 224$, thời gian tải một burst trong mô hình lý tưởng có thể xem xấp xỉ 224 chu kỳ. Tuy nhiên, trên thực tế, burst này phải cộng thêm độ trễ giao tiếp bộ nhớ. Nếu lấy độ trễ tối thiểu là 36 chu kỳ, thời gian cho một burst có thể tăng lên khoảng 260 chu kỳ. Tỷ lệ tăng khi đó là

$$\frac{260}{224} \approx 1.16. \quad (5.29)$$

Trong khi đó, tỷ lệ giữa thời gian thực tế và thời gian mô hình hoá của Conv1_2 là

$$\frac{171}{144.507} \approx 1.18. \quad (5.30)$$

Hai tỷ lệ này có độ tương quan cao. Vì vậy, đối với Conv1_2, mô hình ước lượng

đã phản ánh đúng xu hướng thời gian chạy, còn phần sai lệch chủ yếu có thể giải thích bằng chi phí chờ dữ liệu và giao tiếp bộ nhớ chưa được đưa đầy đủ vào mô hình ban đầu.

5.3.1.2 Phân tích layer Conv5_1

Layer Conv5_1 có kích thước đầu vào $W_i = 14$, $H_i = 14$, $C_i = 512$, kernel 3×3 , stride bằng 1, padding bằng 1, và đầu ra có kích thước $W_o = 14$, $H_o = 14$, $C_o = 512$. Với cấu hình $N_{of tile} = 1$ và $N_{of parr} = 6$, số lần accelerator cần đọc dữ liệu ở layer này là

$$N_{read,ifbuf} = \left\lceil \frac{512}{1 \times 6} \right\rceil \times 14 \times 512 = 616,448. \quad (5.31)$$

Từ đó, thời gian chờ nhận dữ liệu từ *ifbuf*, tính theo độ trễ 180ns trong mô phỏng, là

$$T_{wait,Conv5_1} = 616,448 \times 180\text{ns} \approx 110.96\text{ms}. \quad (5.32)$$

Trong kết quả thực nghiệm, thời gian chạy của Conv5_1 là 174ms, trong khi thời gian mô hình hoá chỉ là 52.5811ms. Độ chênh lệch tương ứng là khoảng 121.4189ms. Giá trị này khá gần với phần thời gian chờ 110.96ms được ước lượng từ số lần yêu cầu đọc *ifbuf*. Do đó, có thể kết luận rằng đối với các layer cuối của VGG16, chi phí giao tiếp DMA và bộ nhớ ngoài đóng vai trò rất lớn trong tổng thời gian chạy.

Nguyên nhân của hiện tượng này nằm ở đặc điểm của layer. So với Conv1_2, Conv5_1 có kích thước không gian nhỏ hơn rất nhiều, nhưng số kênh đầu vào và đầu ra đều là 512. Vì vậy, mặc dù mỗi burst dữ liệu ngắn hơn, số lần phải khởi tạo và chờ giao tiếp dữ liệu lại tăng đáng kể. Với $W_i = 14$, thời gian tải dữ liệu lý tưởng cho một burst chỉ khoảng 14 chu kỳ. Nếu cộng thêm độ trễ giao tiếp tối thiểu 36 chu kỳ, thời gian thực tế cho một burst có thể xấp xỉ 50 chu kỳ. Tỷ lệ tăng khi đó là

$$\frac{50}{14} \approx 3.57. \quad (5.33)$$

Trong khi đó, tỷ lệ giữa thời gian thực tế và thời gian mô hình hoá của Conv5_1 là

$$\frac{174}{52.5811} \approx 3.31. \quad (5.34)$$

Hai giá trị này tiếp tục cho thấy mối tương quan rõ ràng giữa chi phí burst thực tế và sai lệch thời gian chạy. Khi kích thước không gian của feature map giảm, phần thời gian tính toán hữu ích trong mỗi burst trở nên nhỏ hơn, khiến độ trễ cố định của DMA chiếm tỷ trọng lớn hơn trong tổng thời gian. Đây là lý do các layer Conv5 có chỉ số chi phí lên tới 76 chu kỳ trên 1000 MAC, cao hơn đáng kể so với các layer như Conv1_2, Conv2_1 hoặc Conv2_2.

5.3.2 Nhận xét và đánh giá chung đối với VGG16

Từ các kết quả trên, có thể nhận thấy mô hình hoá thời gian đã dự đoán tương đối tốt xu hướng vận hành của accelerator. Đối với các layer có feature map lớn như Conv1_2, sai lệch giữa mô hình và thực tế chủ yếu đến từ phần độ trễ giao tiếp bộ nhớ được cộng thêm vào mỗi burst. Do kích thước burst lớn, độ trễ cố định này chỉ làm thời gian tăng ở mức vừa phải. Ngược lại, ở các layer cuối như Conv5_1, kích thước burst nhỏ nhưng số lần truy cập và cấu hình tăng mạnh, làm cho độ trễ DMA trở thành thành phần chi phối trong tổng thời gian chạy.

Một điểm tích cực là kiến trúc accelerator hoạt động ổn định trong quá trình chạy VGG16 trên board VC707. Kết quả thực nghiệm không ghi nhận hiện tượng tràn bộ nhớ ở *obuf* trong lúc chờ ghi dữ liệu ra bộ nhớ ngoài. Điều này cho thấy cơ chế điều phối giữa tính toán, lưu partial sum, đọc *ifbuf* và ghi *obuf* đã đáp ứng được yêu cầu vận hành cơ bản của mô hình. Nói cách khác, giới hạn chính của hệ thống ở giai đoạn này không nằm ở tính đúng đắn của datapath hay dung lượng buffer, mà nằm ở hiệu quả giao tiếp dữ liệu giữa accelerator và bộ nhớ ngoài.

Tuy nhiên, kết quả cũng cho thấy nếu chỉ tối ưu khối MAC hoặc tăng số lượng PE thì chưa chắc đem lại mức tăng tốc tương ứng, đặc biệt đối với các layer có burst ngắn và số lần truy cập lớn, mặc dù đã có cơ chế double buffering.

Khi độ trễ DMA chiếm tỷ trọng cao, hiệu năng tổng thể sẽ bị giới hạn bởi khả năng cấp phát dữ liệu thay vì năng lực tính toán thuần. Vì vậy, các hướng cải thiện quan trọng trong giai đoạn tiếp theo nên tập trung vào giảm thời gian giao tiếp với memory off-chip, gom nhóm các yêu cầu đọc nhỏ thành các burst hiệu quả hơn, tăng khả năng tái sử dụng dữ liệu trong on-chip buffer.

Nhìn chung, phép đối chiếu giữa kết quả mô hình hoá và kết quả chạy thực tế đã xác nhận rằng mô hình ước lượng có giá trị trong việc dự đoán xu hướng thời gian của accelerator. Dù chưa bao phủ đầy đủ các chi phí hệ thống, mô hình vẫn giúp chỉ ra rõ nguyên nhân sai lệch và định hướng tối ưu. Kết quả thực nghiệm trên VGG16 cho thấy kiến trúc hiện tại đã hoạt động đúng và ổn định trên board VC707, nhưng hiệu năng cuối cùng còn phụ thuộc lớn vào độ trễ DMA và hiệu quả truy cập bộ nhớ ngoài. Đây là cơ sở quan trọng để tiếp tục tối ưu kiến trúc điều phối dữ liệu trong các phiên bản tiếp theo của accelerator.

5.4 Phân tích kết quả thực tế ALL CNN C trên board VC707

Trong phần này, nhóm tiến hành đánh giá hiệu suất thực thi mô hình ALL CNN C trên board VC707 trong hai trường hợp: mô hình được chạy hoàn toàn bằng CPU và mô hình được chạy với sự kết hợp giữa CPU và khối gia tốc CNN [6], [17]. Mục tiêu chính của phép đo là xác định mức cải thiện về thời gian xử lý khi các lớp convolution được chuyển sang thực thi trên accelerator, đồng thời phân tích phần chi phí còn lại do CPU và các overhead hệ thống gây ra trong quá trình chạy end-to-end.

Mô hình ALL CNN C được đánh giá trên cùng ảnh đầu vào CIFAR-10 trong cả hai trường hợp [6], [24]. Đối với cấu hình CPU-only, toàn bộ pipeline bao gồm nạp dữ liệu đầu vào, thực thi các lớp convolution, thực hiện Global Average Pooling và xuất kết quả đều được xử lý bởi CPU. Đối với cấu hình có accelerator, các lớp convolution được chuyển sang khối gia tốc CNN, trong khi một số tác

vụ còn lại như Global Average Pooling và điều phối luồng thực thi vẫn do CPU đảm nhiệm. Việc so sánh hai cấu hình này cho phép đánh giá trực tiếp vai trò của accelerator đối với phần tính toán chiếm tỷ trọng lớn nhất trong mô hình.

Bảng 5.4: So sánh thời gian thực thi mô hình ALL CNN C giữa CPU-only và CPU kết hợp CNN accelerator

Chỉ số	CPU-only	CPU + CNN accelerator	Ghi chú
Conv/compute cycles	30,831,184,413	20,603,434	CPU-only gồm Conv + GlobalAvgPool; accelerator gồm conv trên accelerator và GlobalAvgPool trên CPU
Conv/compute time	154,155 ms	103 ms	Tính theo timer SoC, phản ánh thời gian xử lý chính của mô hình
Wall section cycles	30,833,755,629	44,387,845	Bao gồm thêm overhead UART/log trong quá trình chạy
Wall section time	154,168 ms	221 ms	Phản ánh thời gian end-to-end quan sát được trong vùng đo wall section
Predicted class	3 (cat)	3 (cat)	Hai cấu hình cho cùng kết quả phân loại trên ảnh test hiện tại

Kết quả trong Bảng 5.4 cho thấy sự khác biệt rất lớn giữa hai phương án thực thi. Khi toàn bộ mô hình được chạy bằng CPU, riêng phần compute-only cần 30,831,184,413 chu kỳ, tương ứng khoảng 154,155 ms. Trong khi đó, khi sử dụng CNN accelerator cho các lớp convolution, tổng thời gian đo được cho phần model measured stages chỉ còn 20,603,434 chu kỳ, tương ứng khoảng 103 ms. Như vậy, nếu xét trên phần compute/model measured stages, mô hình khi có accelerator nhanh hơn xấp xỉ:

$$\text{Speedup}_{\text{compute}} = \frac{30,831,184,413}{20,603,434} \approx \mathbf{1496}.$$

Kết quả này cho thấy khối gia tốc CNN đã cải thiện rất mạnh thời gian xử lý của mô hình, đặc biệt ở các lớp convolution. Đây là điều phù hợp với đặc trưng của mô hình ALL CNN C, trong đó phần lớn khối lượng tính toán đến từ các phép nhân–cộng trong convolution. Khi chạy trên CPU, các phép toán MAC phải được thực hiện tuần tự hoặc với mức song song rất hạn chế, dẫn đến thời gian xử lý rất lớn. Ngược lại, accelerator được thiết kế chuyên biệt cho convolution nên có thể khai thác song song dữ liệu, tái sử dụng dữ liệu trong các buffer nội bộ và giảm đáng kể số chu kỳ cần thiết cho cùng một nhóm phép toán.

Để đánh giá sát hơn thời gian end-to-end, cần xét thêm chỉ số wall section.

Với CPU-only, wall section mất 30,833,755,629 chu kỳ, tương ứng 154,168 ms. Khi sử dụng accelerator, wall section giảm xuống còn 44,387,845 chu kỳ, tương ứng 221 ms. Tốc độ tăng tốc khi xét toàn bộ wall section được tính như sau:

$$\text{Speedup}_{\text{wall}} = \frac{30,833,755,629}{44,387,845} \approx \mathbf{695}.$$

Có thể thấy speedup theo wall section nhỏ hơn speedup theo compute/model measured stages. Nguyên nhân là wall section không chỉ bao gồm thời gian tính toán thuần của mô hình, mà còn bao gồm các overhead hệ thống như in log qua UART, điều phối giữa CPU và accelerator, cấu hình DMA, truyền dữ liệu và các thao tác phụ trợ khác. Trong log thực nghiệm, thời gian model measured stages của cấu hình có accelerator là 103 ms, trong khi wall section là 221 ms. Phần chênh lệch này cho thấy mặc dù accelerator xử lý convolution rất nhanh, thời gian end-to-end vẫn còn bị ảnh hưởng đáng kể bởi các chi phí ngoài phần tính toán chính.

Trong quá trình đánh giá, workload được thống nhất là workload của mô hình ALL CNN C khi chạy end-to-end trên cùng một ảnh đầu vào. Trong báo cáo này, workload end-to-end của mô hình ALL CNN C được lấy là:

$$\text{Workload}_{\text{ALL_CNN_C}} = 281,174,016 \text{ MACs}.$$

Giá trị workload này được dùng làm cơ sở để tính toán throughput và cost cho cả hai lần chạy. Với cấu hình CPU-only, toàn bộ workload được xử lý bởi CPU. Với cấu hình CPU kết hợp accelerator, phần lớn workload, đặc biệt là các lớp convolution, được xử lý bởi khối gia tốc CNN, trong khi CPU đảm nhiệm các bước điều phối và một số phép xử lý phụ trợ còn lại.

Bảng 5.5: So sánh hiệu suất xử lý theo cùng workload end-to-end của mô hình ALL CNN C

Cấu hình	Workload (MACs)	Compute/model cycles	Compute/model time (ms)	Cost (cycles/1000 MACs)
CPU-only	281,174,016	30,831,184,413	154,155	109,649
CPU + CNN accelerator	281,174,016	20,603,434	103	73

Kết quả phân loại cũng là một yếu tố quan trọng trong quá trình đánh giá. Cả hai cấu hình CPU-only và CPU kết hợp accelerator đều dự đoán cùng một lớp là class 3, tương ứng với nhãn cat. Điều này cho thấy việc chuyển các lớp convolution sang accelerator không làm thay đổi kết quả phân loại đối với ảnh test hiện tại. Nói cách khác, trong phép thử này, accelerator không chỉ cải thiện mạnh thời gian xử lý mà còn duy trì được tính đúng đắn về mặt chức năng của mô hình. Khi sử dụng cùng workload end-to-end là 281,174,016 **MACs**, cấu hình CPU-only cần khoảng 109,649 **chu kỳ** để xử lý mỗi 1000 MACs. Trong khi đó, cấu hình CPU kết hợp CNN accelerator chỉ cần khoảng 73 **chu kỳ** cho mỗi 1000 MACs nếu xét theo thời gian model measured stages. Điều này cho thấy accelerator giúp giảm chi phí xử lý trên mỗi đơn vị tính toán khoảng:

$$\frac{109,649}{73} \approx \mathbf{1502 \text{ lần.}}$$

Giá trị này tương đối gần với speedup theo thời gian compute/model measured stages là khoảng 1496 lần. Sự sai khác nhỏ đến từ việc làm tròn số liệu thời gian và cost trong quá trình trình bày. Tuy nhiên, khi phân tích kết quả end-to-end, không nên chỉ nhìn vào thời gian convolution riêng lẻ. Trong cấu hình có accelerator, thời gian convolution là 103 ms nhưng wall section lên đến 221 ms. Điều này cho thấy phần overhead ngoài accelerator chiếm tỷ lệ đáng kể trong tổng thời gian quan sát được. Các overhead này có thể bao gồm thời gian cấu hình accelerator, thời gian truyền dữ liệu giữa bộ nhớ ngoài và buffer nội bộ, đồng bộ giữa CPU và accelerator, cũng như thời gian in log UART trong từng giai đoạn. Do đó, nếu mục tiêu là tối ưu hiệu năng end-to-end, cần tiếp tục giảm các chi phí điều phối và truyền dữ liệu, không chỉ tối ưu riêng datapath tính toán của accelerator.

Tổng hợp lại, kết quả thực nghiệm cho thấy CNN accelerator mang lại hiệu quả rất lớn khi chạy mô hình ALL CNN C trên board VC707. Nếu xét phần xử lý chính của mô hình, cấu hình có accelerator đạt mức tăng tốc khoảng 1496 **lần** so với CPU-only; nếu xét toàn bộ wall section, tức bao gồm cả overhead hệ

thống và log, mức tăng tốc vẫn đạt khoảng 695 **lần**. Các số liệu này khẳng định accelerator hoạt động hiệu quả đối với mô hình CNN có tỷ trọng convolution cao, đồng thời chỉ ra rằng giới hạn hiệu năng còn lại không nằm chủ yếu ở datapath tính toán mà ở luồng truyền dữ liệu và điều phối hệ thống. Vì vậy, hướng tối ưu tiếp theo cần tập trung vào giảm overhead UART/log, giảm chi phí cấu hình DMA, tăng hiệu quả truy cập bộ nhớ ngoài và tổ chức lại cơ chế trao đổi dữ liệu giữa CPU, DMA, buffer nội bộ và accelerator để đưa thời gian end-to-end tiến gần hơn đến thời gian compute thực tế.

5.5 Phân tích sự đánh đổi khi triển khai mô hình INT8 trên CNN Accelerator

Trong chương này, nhóm phân tích ảnh hưởng của quá trình lượng tử hóa và môi trường thực thi đến độ chính xác của mô hình khi triển khai trên phần cứng chuyên dụng. Mô hình được sử dụng là ALL-CNN-C ở định dạng số thực 32-bit dấu phẩy động, đã được huấn luyện trước và được chọn làm mô hình tham chiếu [6]. Khi đánh giá trên tập kiểm thử CIFAR-10 gồm **10.000** ảnh, mô hình gốc FP32 đạt độ chính xác 90.25% [24].

Sau đó, mô hình được lượng tử hóa sang định dạng số nguyên 8-bit bằng phương pháp post-training quantization, không thực hiện thêm bước huấn luyện hậu lượng tử hóa [7], [22]. Cùng một mô hình INT8, bao gồm cùng weight, bias, scale và zero-point, được sử dụng để đánh giá trên hai môi trường thực thi khác nhau: TensorFlow Lite và CNN accelerator. Kết quả thực nghiệm được trình bày trong Bảng 5.6.

Bảng 5.6: So sánh độ chính xác của mô hình trên các môi trường thực thi

Môi trường thực thi	Accuracy
Mô hình FP32 chạy trên framework tham chiếu	90.25%
Mô hình INT8 chạy trên TensorFlow Lite	84.50%
Mô hình INT8 chạy trên CNN accelerator	87.85%

Kết quả cho thấy quá trình lượng tử hóa từ FP32 sang INT8 làm suy giảm độ chính xác của mô hình. Khi chạy bằng TensorFlow Lite, accuracy giảm từ 90.25% xuống còn 84.50%, tương ứng mức giảm 5.75 điểm phần trăm. Tuy nhiên, khi cùng mô hình INT8 được triển khai trên CNN accelerator, accuracy đạt 87.85%, cao hơn TensorFlow Lite 3.35 điểm phần trăm. Điều này cho thấy độ chính xác của mô hình lượng tử hóa không chỉ phụ thuộc vào bản thân weight, bias, scale và zero-point đã được lưu trong mô hình, mà còn phụ thuộc đáng kể vào cách môi trường thực thi hiện thực các phép toán số học lượng tử hóa.

Về mặt nguyên lý, một phép convolution INT8 thường không tạo trực tiếp kết quả đầu ra ở INT8 ngay lập tức. Thay vào đó, phép nhân giữa activation và weight INT8 được tích lũy trong miền có độ rộng lớn hơn, thường là INT32 [7], [22]. Quá trình này có thể được mô tả khái niệm như sau:

$$acc_{int32} = \sum (x_q - zp_x)(w_q - zp_w) + b_{int32} \quad (5.35)$$

Trong đó x_q là activation đã lượng tử hóa, w_q là weight đã lượng tử hóa, zp_x và zp_w lần lượt là zero-point của activation và weight, còn b_{int32} là bias được biểu diễn trong miền INT32. Sau bước tích lũy, giá trị INT32 cần được đưa trở lại miền biểu diễn INT8 thông qua quá trình requantization:

$$y_q = clamp(round(acc_{int32} \cdot S_{eff}) + zp_y) \quad (5.36)$$

với hệ số scale hiệu dụng:

$$S_{eff} = \frac{S_x S_w}{S_y} \quad (5.37)$$

Trong đó S_x , S_w và S_y lần lượt là scale của input, weight và output. Công thức trên cho thấy ngay cả khi hai môi trường thực thi sử dụng cùng weight, bias, scale và zero-point, kết quả cuối cùng vẫn có thể khác nhau nếu cách hiện thực các bước trung gian không hoàn toàn giống nhau. Các khác biệt có thể xuất hiện ở cách xấp xỉ hệ số scale bằng fixed-point multiplier, độ rộng bit của giá trị trung gian, quy tắc làm tròn, thứ tự cộng zero-point, cách áp dụng activation và quy tắc clamp về miền INT8.

Nguyên nhân hợp lý cho việc CNN accelerator đạt accuracy cao hơn TensorFlow Lite trong thí nghiệm này có thể nằm ở cách phần cứng xử lý các giá trị trung gian trong quá trình convolution và requantization. Thay vì làm tròn và ép kết quả về INT8 quá sớm sau bước nhân với hệ số scale, accelerator có thể giữ kết quả trung gian ở độ rộng lớn hơn là INT32. Sau khi hoàn tất các bước scale, cộng zero-point, áp dụng activation và xử lý saturation, kết quả cuối cùng mới

được clamp về miền INT8. Cách xử lý này giúp giảm số lần làm tròn và hạn chế mất mát thông tin trung gian.

Sự khác biệt này đặc biệt có ý nghĩa đối với mô hình ALL-CNN-C, do mô hình gồm nhiều lớp convolution nối tiếp. Trong mạng nhiều tầng, sai số lượng tử hóa phát sinh ở các lớp đầu có thể tiếp tục lan truyền sang các lớp sau. Nếu mỗi tầng đều phát sinh sai số do làm tròn hoặc clamp quá sớm, sai số tích lũy có thể làm thay đổi đáng kể đặc trưng ở các tầng sâu, từ đó ảnh hưởng đến kết quả phân lớp cuối cùng. Do đó, việc giữ độ chính xác trung gian cao hơn trong accelerator có thể giúp đầu ra của mô hình INT8 gần hơn với mô hình FP32, dẫn đến accuracy cao hơn so với khi thực thi bằng một pipeline có sai số làm tròn lớn hơn.

Tuy nhiên, cần lưu ý rằng việc sử dụng cùng mô hình đã lượng tử hóa không đồng nghĩa với việc hai môi trường thực thi sẽ cho kết quả bit-exact. Cùng weight, bias, scale và zero-point chỉ đảm bảo hai môi trường sử dụng cùng bộ tham số lượng tử hóa. Để cho ra kết quả hoàn toàn giống nhau, cả hai môi trường còn phải giống nhau ở cách thực hiện số học bên trong, bao gồm tích lũy, nhân scale, dịch bit, làm tròn, saturation, activation và clamp. Chỉ cần output của một layer lệch một vài mức lượng tử hóa, sai khác này vẫn có thể lan truyền qua các tầng convolution tiếp theo và làm thay đổi nhãn dự đoán cuối cùng.

Vì vậy, chênh lệch accuracy giữa TensorFlow Lite và CNN accelerator trong thí nghiệm này là có cơ sở về mặt kỹ thuật. Kết quả không nên được hiểu là phần cứng tự làm mô hình trở nên chính xác hơn, mà nên được hiểu là pipeline số học của accelerator có thể bảo toàn thông tin trung gian tốt hơn trong quá trình suy luận INT8. Đây là một yếu tố quan trọng khi thiết kế phần cứng tăng tốc CNN, vì lựa chọn độ rộng bit và vị trí làm tròn trong datapath có thể ảnh hưởng trực tiếp đến độ chính xác suy luận.

Mặc dù vậy, để khẳng định chắc chắn nguyên nhân của chênh lệch trên, cần thực hiện thêm bước đối chiếu output trung gian giữa TensorFlow Lite và CNN accelerator sau từng lớp convolution. Nếu accumulator INT32 của hai môi

trường giống nhau nhưng output INT8 sau requantization khác nhau, nguyên nhân nhiều khả năng nằm ở bước nhân scale, rounding, activation hoặc clamp. Ngược lại, nếu sai khác đã xuất hiện ngay tại accumulator INT32, cần kiểm tra thêm các yếu tố như padding, thứ tự kênh, layout của weight, bias addition hoặc dữ liệu input sau preprocessing.

Từ các phân tích trên, có thể kết luận rằng khi triển khai mô hình lượng tử hóa 8-bit trên phần cứng chuyên dụng, độ chính xác không chỉ phụ thuộc vào mô hình đã lượng tử hóa mà còn phụ thuộc mạnh vào cách phần cứng hiện thực các phép toán trung gian. Việc CNN accelerator đạt accuracy 87.85%, cao hơn TensorFlow Lite 84.50%, cho thấy thiết kế datapath có khả năng giữ giá trị trung gian ở độ rộng lớn hơn trước khi clamp về INT8 có thể giúp giảm sai số lượng tử hóa tích lũy. Đây là một minh chứng cho sự đánh đổi quan trọng giữa tài nguyên phần cứng, tốc độ xử lý và độ chính xác khi thiết kế accelerator cho mạng nơ-ron tích chập.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Đồ án đã trình bày quá trình thiết kế và hiện thực hệ thống BK-AISoC, một SoC RISC-V tích hợp CNN accelerator hướng đến các ứng dụng EdgeAI trên nền tảng FPGA [2], [8]. Hệ thống kết hợp CPU RISC-V cho control path với các khối phần cứng chuyên dụng như DMAC, Video Streaming, HyperRAM interface và CNN accelerator cho data path có cường độ tính toán cao [3], [4].

Kết quả hiện thực và đánh giá cho thấy kiến trúc accelerator có khả năng tăng tốc đáng kể các lớp tích chập trong mô hình CNN, đồng thời vẫn giữ được khả năng tích hợp vào một SoC hoàn chỉnh. Các phân tích về mô phỏng RTL, thực nghiệm trên board VC707 và độ chính xác khi triển khai mô hình INT8 cho thấy giá trị chính của thiết kế nằm ở việc cân bằng giữa hiệu năng, tài nguyên phần cứng và độ chính xác suy luận [7], [17].

6.2 Hướng phát triển

Chương này trình bày các hướng phát triển tiếp theo của hệ thống BK-AISoC sau giai đoạn hiện thực và đánh giá ban đầu. Các nội dung được tập trung vào việc hoàn thiện kiến trúc phần cứng, tối ưu luồng phần mềm và mở rộng khả

năng triển khai mô hình AI trên hệ thống thực.

6.2.1 Hoàn thiện kiến trúc phần cứng

Ở mức phần cứng, hướng phát triển chính là tiếp tục tối ưu luồng truyền dữ liệu giữa CPU, DMA, bộ nhớ ngoài và CNN accelerator. Các điểm cần cải thiện bao gồm giảm thời gian chờ khi truy cập HyperRAM, tối ưu cơ chế cấp dữ liệu cho các buffer nội bộ và giảm overhead trong quá trình cấu hình từng lớp của mô hình.

Bên cạnh đó, hệ thống có thể được mở rộng thêm các khối ngoại vi hoặc tăng mức song song của accelerator khi tài nguyên FPGA còn cho phép. Việc mở rộng này cần đi kèm với phân tích timing, floorplanning và đánh giá tài nguyên để bảo đảm toàn bộ SoC vẫn đáp ứng tần số mục tiêu.

6.2.2 Tối ưu firmware và application

Ở mức firmware, các luồng điều khiển accelerator, nạp weights và quản lý bộ nhớ có thể tiếp tục được chuẩn hóa để hỗ trợ nhiều mô hình hơn. Một hướng quan trọng là giảm overhead trong quá trình điều phối từ CPU, đặc biệt ở các thao tác cấu hình DMA, kiểm tra trạng thái accelerator và ghi log qua UART.

Đối với application, hệ thống có thể được mở rộng từ chương trình kiểm thử mẫu sang các pipeline hoàn chỉnh hơn, bao gồm thu nhận dữ liệu từ camera, tiền xử lý ảnh, chạy inference và xuất kết quả theo thời gian thực.

6.2.3 Định hướng đánh giá thực nghiệm

Trong các giai đoạn tiếp theo, hệ thống cần được đánh giá trên nhiều mô hình CNN và nhiều dạng dữ liệu đầu vào hơn. Ngoài thời gian thực thi, các chỉ số như băng thông bộ nhớ, mức sử dụng tài nguyên, khả năng đáp ứng timing và độ ổn định khi chạy lâu trên board cũng cần được phân tích chi tiết hơn.

Các kết quả này sẽ là cơ sở để xác định giới hạn hiện tại của kiến trúc, từ đó lựa chọn hướng tối ưu phù hợp cho phiên bản tiếp theo của BK-AISoC.

Tài liệu tham khảo

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [2] Z. Liu, J. Jiang, G. Lei, K. Chen, B. Qin, and X. Zhao, “A heterogeneous processor design for cnn-based ai applications on iot devices,” *Procedia Computer Science*, vol. 174, pp. 2–8, 2020.
- [3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [4] L. Du et al., “A reconfigurable streaming deep convolutional neural network accelerator for internet of things,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 1, pp. 198–208, 2018.
- [5] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [6] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, “Striving for simplicity: The all convolutional net,” *arXiv preprint arXiv:1412.6806*, 2014.
- [7] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [8] A. Waterman and K. Asanovic, *The risc-v instruction set manual, volume i: Unprivileged isa*, Version 20191213, 2019.

- [9] P. P. Chu, *FPGA Prototyping by SystemVerilog Examples: Xilinx MicroBlaze MCS SoC Edition*. Wiley, 2018.
- [10] C. Wolf and YosysHQ, *Picorv32: A size-optimized risc-v cpu*, <https://github.com/YosysHQ/picorv32>, 2024.
- [11] Arm Limited, *Introduction to amba axi*, 2022.
- [12] Arm Limited, *Amba axi-stream protocol specification*, IHI0051B, 2010.
- [13] E. Pena and M. G. Legaspi, “Uart: A hardware communication protocol understanding universal asynchronous receiver/transmitter,” *Analog Dialogue*, vol. 54, 2020.
- [14] P. Dhaker, “Introduction to spi interface,” *Analog Dialogue*, vol. 52, 2018.
- [15] M. G. Legaspi and E. Pena, “I2c communication protocol: Understanding i2c primer, pmbus, and smbus,” *Analog Dialogue*, vol. 55, 2021.
- [16] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill Education, 2013.
- [17] Xilinx, Inc., *Vc707 evaluation board for the virtex-7 fpga user guide*, UG885, v1.8, 2019.
- [18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [19] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37, 2015, pp. 448–456.
- [20] Z. Li et al., “A high-performance pixel-level fully pipelined hardware accelerator for neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, no. 5, pp. 7970–7983, 2025.

- [21] J. Zhang and J. Li, “Improving the performance of opencl-based fpga accelerator for convolutional neural network,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2017, pp. 25–34.
- [22] TensorFlow, *Tensorflow lite 8-bit quantization specification*, https://www.tensorflow.org/lite/performance/quantization_spec, 2024.
- [23] Xilinx, Inc., *Logicore ip axi interconnect data sheet*, DS768, 2012.
- [24] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.